

COAST

Coastal Overtopping Analysis and Simulation Toolkit

User and Technical Manual

Version 1.0

Ali Mahdi

ali.mahdi@univ-rouen.fr

Important notice

COAST is research software.

The author makes **no warranty of any kind** regarding the accuracy, reliability, or fitness for any purpose of anything this software produces: geometries, meshes, OpenFOAM cases, simulation results, discharges, wave statistics, spectra, reflection coefficients, energy fluxes, flow depths and velocities, forces, reports, or figures.

The author accepts no responsibility for the results obtained with this software, nor for anything the software does on your system, nor for any decision taken on the basis of its output. Every result must be independently verified by the user.

COAST must **not** be used as the sole or primary basis for the design, assessment, or certification of any coastal structure or flood defence, nor for any decision affecting public safety, property, or the environment. Any such use is entirely at the user's own risk and remains the responsibility of the user and of the qualified engineer signing off on the work.

The full licence text and the complete disclaimer are given in Chapter 15 and in the `LICENSE` file at the root of the repository. By using this software you accept those terms.

Contents

Important notice i

1 Introduction 1
- 1.1 Purpose . . . 1
- 1.2 How to use this manual . . . 1
- 1.3 Software stack . . . 2
- 1.4 Conventions and notation . . . 3

2 Installation and Quick Start 4
- 2.1 Prerequisites . . . 4
- 2.2 Installation . . . 4
- 2.3 Environment set-up . . . 4
- 2.4 Launching the application . . . 5
- 2.5 A first case, step by step . . . 6
- 2.6 Command-line tools . . . 6
- 2.7 Running the analysis outside the application . . . 7

3 Application Overview 8
- 3.1 Architecture . . . 8
- 3.2 Sidebar and case-generation tabs . . . 8
- 3.3 Post-processing page . . . 10
- 3.4 Case directory layout . . . 11
- 3.5 Reproducibility: metadata and reports . . . 12

4 Step-by-Step Guide 14
- 4.1 The demonstration structure . . . 14
- 4.2 Before starting: how the interface keeps the inputs . . . 14
- 4.3 The window and the sidebar . . . 15
- 4.4 Tab *Geometry* . . . 17
  - 4.4.1 Variant: overtopping collector . . . 17
  - 4.4.2 Variant: breakwater mode . . . 19
  - 4.4.3 Variant: a structure with porous layers . . . 21
  - 4.4.4 Variant: adding an obstacle . . . 22
- 4.5 Tab *Wave and Domain* . . . 23
- 4.6 Tab *Mesh* . . . 26
- 4.7 Tab *Simulation and Job* . . . 30
- 4.8 Tab *Run* . . . 31
- 4.9 Tab *Monitor* . . . 31
- 4.10 An example of an analysed run . . . 34
- 4.11 Post-processing page: case, gauges and settings . . . 34

4.12 Tabs 1 to 5: the waves at each gauge . . . 34
4.13 Tab 6: reflection . . . 40
4.14 Tab 7: overtopping . . . 40
4.15 Tabs 8 to 10: force, summary and reports, comparison . . . 42
4.16 The case folder at the end . . . 44

5 Structure Geometry **51**
5.1 EurOtop parameterisation . . . 51
5.1.1 From a record of the EurOtop database to the inputs of COAST . . . 52
5.2 Profile construction . . . 53
5.2.1 Vertical levels . . . 53
5.2.2 Horizontal distances . . . 53
5.2.3 STL extrusion . . . 54
5.3 Structure type: breakwater mode and collector . . . 54
5.4 Obstacles and imported geometry . . . 54
5.5 Porous layers . . . 56
5.5.1 How many armour layers: the roughness factor γ_f . . . 57
5.6 Profile segmentation and force patches . . . 58

6 Wave Conditions and Computational Domain **59**
6.1 Wave parameters . . . 59
6.2 Deep-water wavelength . . . 60
6.3 Domain sizing . . . 60
6.3.1 Horizontal extent . . . 60
6.3.2 Vertical extent . . . 60
6.3.3 Lateral extent . . . 60
6.4 Relaxation zones . . . 61
6.5 Breakwater mode . . . 61
6.6 Initial conditions . . . 61

7 Mesh Generation and Measurement Devices **62**
7.1 Background mesh: blockMesh . . . 62
7.2 Surface refinement: snappyHexMesh . . . 62
7.2.1 Interface band . . . 62
7.2.2 Run-up band . . . 62
7.3 Extrusion to pseudo-2D . . . 63
7.4 Resolution guidance . . . 63
7.5 Measurement devices written at build time . . . 64

8 Numerical Model **65**
8.1 Solver and governing equations . . . 65
8.1.1 Continuity . . . 65
8.1.2 Momentum . . . 65
8.1.3 VOF transport . . . 65
8.1.4 Mixture properties . . . 65
8.2 Discretisation and solution . . . 65
8.3 Boundary conditions . . . 66
8.4 Turbulence treatment . . . 66
8.5 Wall roughness . . . 67
8.6 Porous zone modelling . . . 67

8.7 Wave generation and absorption . . . 67
8.8 Time control and output . . . 68
8.9 Parallel execution . . . 69

9 Building and Running a Case **70**
9.1 Build pipeline . . . 70
9.2 The generated job script . . . 70
9.3 Execution and monitoring . . . 71

10 Post-Processing Pipeline **72**
10.1 Free-surface signal extraction . . . 72
10.2 Transient trimming . . . 73
10.2.1 Stationarity detection . . . 73
10.3 Wave detection by zero down-crossing . . . 74
10.3.1 Mean removal . . . 74
10.3.2 Zero crossings by linear interpolation . . . 74
10.3.3 Individual wave properties . . . 74
10.4 Wave statistics . . . 75
10.4.1 Characteristic heights . . . 75
10.4.2 Empirical distributions . . . 75
10.4.3 Rayleigh reference . . . 75
10.5 Spectral analysis . . . 76
10.5.1 Welch power spectral density . . . 76
10.5.2 Spectral moments and parameters . . . 76
10.5.3 Target JONSWAP spectrum . . . 77
10.6 Reflection analysis . . . 77
10.6.1 Formulation . . . 78
10.6.2 Procedure . . . 78
10.7 Overtopping discharge and events (Step 1) . . . 79
10.7.1 Discharge . . . 80
10.7.2 Averaging window . . . 80
10.7.3 Event detection by hysteresis . . . 81
10.7.4 Detection parameters . . . 81
10.7.5 Volume budget as a quality indicator . . . 82
10.7.6 Outputs . . . 82
10.8 Kinetic and potential energy flux (Step 2) . . . 82
10.9 Energy–volume analysis (Step 3) . . . 84
10.9.1 Median regression . . . 84
10.9.2 Power law . . . 84
10.9.3 Medians . . . 84
10.10 Flow depth and flow velocity (Step 4) . . . 85
10.10.1 Quantities at each output time . . . 85
10.10.2 Values per event . . . 86
10.10.3 Values exceeded by 2 % of the incident waves . . . 86
10.10.4 Parameters, outputs and figures . . . 86
10.10.5 Figures of one overtopping event . . . 88
10.11 Force extraction . . . 90
10.12 Summary, comparison and reports . . . 91

11 Demonstration Against the EurOtop Database **92**

11.1 Purpose and status of this comparison . . . 92
11.2 Campaign overview . . . 92
11.3 Surface treatment and porous parameterisation . . . 93
11.4 Dimensionless parameterisation . . . 94
11.5 Results . . . 94
11.5.1 Sources of residual scatter . . . 94
11.6 Case-by-case results . . . 96

12 Limitations and Good Practice **98**
12.1 Limitations . . . 98
12.2 Recommended verification sequence . . . 99

13 Questions and Troubleshooting **100**
13.1 Installation and start . . . 100
13.2 Inputs and set-up . . . 100
13.3 Running . . . 102
13.4 Post-processing . . . 103

14 Repository Structure **105**

15 Licence and Disclaimer **107**
15.1 CeCILL v2.1 . . . 107
15.2 Author's additional notice . . . 107
15.3 How to cite . . . 108

A Input Parameter Reference **109**

B Output File Reference **113**

Bibliography **115**

List of Figures

3.1 COAST workflow . . . 9

4.1 The COAST window . . . 16
4.2 Tab Geometry: inputs . . . 18
4.3 Tab Geometry: generated section . . . 19
4.4 Collector: input . . . 19
4.5 Collector: generated section . . . 20
4.6 Breakwater mode: input . . . 20
4.7 Breakwater mode: domain sizing . . . 21
4.8 Breakwater mode: domain . . . 21
4.9 Tab Geometry: porous layers . . . 22
4.10 Tab Geometry: section with porous layers . . . 23
4.11 Tab Geometry: obstacle editor . . . 24
4.12 Tab Geometry: current obstacles . . . 25
4.13 Tab Geometry: section with an obstacle . . . 26
4.14 Tab Wave and Domain . . . 27
4.15 Numerical flume of the demonstration structure . . . 28
4.16 Tab Mesh . . . 29
4.17 Tab Simulation and Job . . . 30
4.18 Force segments . . . 31
4.19 Tab Run . . . 32
4.20 Tab Monitor . . . 33
4.21 Numerical flume of the example run . . . 35
4.22 Post-processing page: settings . . . 36
4.23 Tab 1 Signals . . . 37
4.24 Tab 2 Trimming . . . 37
4.25 Tab 3 Wave Detection . . . 38
4.26 Tab 4 Statistics . . . 38
4.27 Tab 5 Spectrum . . . 39
4.28 Tab 6 Reflection . . . 41
4.29 Tab 7 Step 1: parameters . . . 42
4.30 Tab 7 Step 1: summary . . . 43
4.31 Tab 7 Step 1: events and figures . . . 44
4.32 Tab 7 Step 2 . . . 45
4.33 Tab 7 Step 3 . . . 46
4.34 Tab 7 Step 3: figures . . . 47
4.35 Tab 7 Step 4 . . . 48
4.36 Tab 7: figures of one event . . . 49
4.37 Tab 8 Force . . . 49
4.38 Tab 9 Summary . . . 50
4.39 Tab 10 Compare . . . 50

5.1 Generated cross-section 52
5.2 Geometry catalogue 55
5.3 Layered porous structure 57

7.1 Mesh refinement regions 63

10.1 Raw free-surface signal 73
10.2 Zero-crossing wave detection 74
10.3 Wave-height statistics 76
10.4 Measured versus target spectrum 77
10.5 Incident/reflected separation 79
10.6 Step 1 overtopping analysis 82
10.7 Step 2 energy flux 84
10.8 Event energy versus volume 85
10.9 Definition of flow depth and velocity 87
10.10 Flow depth and velocity of all events 87
10.11 Snapshots of one overtopping event 89
10.12 Overtopping section of one event 90
10.13 Force analysis 91

11.1 COAST versus EurOtop parity 95

List of Tables

1.1 Where to find what. . . . 2
1.2 Software dependencies. . . . 2
1.3 Principal symbols used throughout this manual. . . . 3

2.1 Prerequisites. . . . 4

3.1 Case-generation tabs. . . . 10
3.2 Post-processing tabs. . . . 11

4.1 Inputs of the demonstration structure (case name `Demo_structure`). . . . 15
4.2 Example run: values of the EurOtop database (test D-1.1) and inputs of the run. 34

5.1 Structure geometry parameters (EurOtop convention, plus the placement parameters of COAST). . . . 51
5.2 Columns of the EurOtop database and the corresponding inputs of COAST. . . . 52
5.3 Roughness factor of rubble mound armour (EurOtop Table 6.2) 57

8.1 Boundary conditions of the templates. "No-slip" is a fixed zero velocity. . . . 66
8.2 Time-control and output parameters (*Simulation and Job* tab). . . . 68

10.1 The four steps of the overtopping chain. . . . 72
10.2 Hysteresis event-detection state machine. . . . 81
10.3 Step 1 parameters. . . . 81

11.1 EurOtop database structure-type labels. . . . 93
11.2 COAST versus EurOtop database for the 50 reference cases. R_c^* and q^* as in Eq. (11.1); subscripts E and C denote EurOtop and COAST; RF and CF are the reliability and complexity factors of the EurOtop database. q in $\mathrm{m^3\,s^{-1}\,m^{-1}}$. . . . 96

A.1 Inputs of the case-generation page. . . . 109
A.2 Inputs of the post-processing page. . . . 111

B.1 Files written by COAST. . . . 113

Chapter 1

Introduction

1.1 Purpose

COAST (*Coastal Overtopping Analysis and Simulation Toolkit*) is an application that prepares, runs, and analyses numerical simulations of wave overtopping at coastal structures. It uses the two-phase Volume-of-Fluid (VOF) solvers of the open-source CFD library OpenFOAM together with the wave generation library waves2Foam (Jacobsen et al., 2012). The structure is described with the parameters of the EurOtop manual (EurOtop, 2018). From these parameters and the wave conditions, COAST writes a complete OpenFOAM case, runs it, and analyses the results with a fixed post-processing chain.

The motivation is practical. Assembling an OpenFOAM case for a coastal structure by hand means writing and cross-checking dozens of dictionaries: mesh, boundary conditions, wave generation, porous zones, sampling, output. Even for an experienced user this takes hours per case and is a well-known source of silent configuration errors. COAST derives every dictionary from one coherent parameter set, so the cases of a campaign differ only in the physics being studied, not in the set-up choices of the analyst.

COAST couples four stages:

- **Geometry generation**: parameterised EurOtop cross-sections, optional crown wall, collector, merged obstacles and multi-layer porous armour, extruded to watertight 3D STL surfaces;
- **Case set-up**: automated mesh generation (`blockMesh` and `snappyHexMesh`), boundary and initial conditions, wave generation, measurement devices, and domain decomposition;
- **Execution**: parallel `waveFoam` or `porousWaveFoam` runs in terminal-multiplexer (tmux) sessions with live monitoring;
- **Post-processing**: wave-by-wave and spectral analysis, incident/reflected wave separation, mean overtopping discharge and individual overtopping events, energy fluxes, flow depth and flow velocity of each event, forces on selected structure faces, and multi-case comparison.

1.2 How to use this manual

The manual is written for two kinds of reader: users who want to run a case and read its results, and users who need to know exactly how each quantity is computed. Table 1.1 indicates where to start.

Table 1.1: Where to find what.

If you want to…	**Read**
Install COAST and run a first case	Chapter 2
Find your way in the interface	Chapter 3
Go through every tab, with screenshots	Chapter 4
Choose the porous layers from the EurOtop database (γ_f)	Section 5.5.1
Describe the structure, the waves and the mesh	Chapters 5–7
Know which equations are solved, with which boundary conditions	Chapter 8
Build, launch and follow a simulation	Chapter 9
Understand every output and its formula	Chapter 10
Judge how far a result can be trusted	Chapters 11 and 12
Find the answer to a practical question or a problem	Chapter 13
Look up an input of the interface or an output file	Appendices A and B

Default values. The application proposes a default value for most inputs. These values are shown in the input fields of the interface and are *not* repeated in this manual: they are starting points that may change between versions, not recommendations for a given structure or sea state. For each parameter, the manual explains its role, its effect on the result, and how to choose it.

This manual documents the software and the methods it implements. It is self-contained: no other publication is needed to use COAST. Every input, button and output of the interface is described: the inputs in Appendix A, the outputs in Appendix B, and their use, tab by tab, in Chapter 4.

1.3 Software stack

Table 1.2: Software dependencies.

Component	**Tool / Library**	**Version**
CFD solver	OpenFOAM (`waveFoam`, `porousWaveFoam`)	v2206 (v2112 also used)
Wave library	waves2Foam	compiled against the OpenFOAM version
Turbulence	k-ω SST	OpenFOAM built-in
User interface	Streamlit	$\geq$ 1.51
Numerics	NumPy, SciPy, pandas	see `requirements.txt`
CAD	numpy-stl, Shapely	see `requirements.txt`
Plotting	Matplotlib, Seaborn, Plotly	see `requirements.txt`
Reports	ReportLab	see `requirements.txt`
Execution	tmux	installed locally by `install.sh`

The stack is entirely open source. OpenFOAM and waves2Foam are third-party software with their own licences and their own warranty disclaimers; COAST orchestrates them but does not include or license them (Chapter 15).

1.4 Conventions and notation

Coordinates. x is horizontal and positive in the direction of wave propagation (from the offshore boundary towards the structure and landward), z is vertical and positive upward, and y is the spanwise direction. All elevations are absolute values of z; the still-water level (SWL) is an input, written z_{SWL} in equations. The free-surface elevation η is measured from the SWL.

Units. SI units throughout. Simulations are pseudo-two-dimensional (Section 7.3); discharges, volumes, and energies are given per metre of flume width.

Table 1.3: Principal symbols used throughout this manual.

Symbol	Unit	Meaning
$\eta(t)$	m	Free-surface elevation at a gauge, relative to the SWL
H_{m0}	m	Spectral significant wave height, $4\sqrt{m_0}$
$H_{1/3}$, $H_{1/10}$	m	Mean of the highest third and tenth of the zero-crossing wave heights
T_p, f_p	s, Hz	Spectral peak period and peak frequency, $f_p = 1/T_p$
$T_{m-1,0}$	s	Spectral period m_{-1}/m_0
L_0	m	Deep-water wavelength, $gT_p^2/2\pi$
L_p	m	Linear wavelength at T_p and the local depth
h	m	Water depth at the structure toe
h_{deep}	m	Water depth at the offshore (wave generation) boundary
A_c, R_c	m	Armour crest level and crest freeboard above the SWL
$q^+(t)$	$\mathrm{m^3\,s^{-1}\,m^{-1}}$	Landward overtopping rate per unit width
$\overline{q}$	$\mathrm{m^3\,s^{-1}\,m^{-1}}$	Mean overtopping discharge
V_e	$\mathrm{m^3/m}$	Volume of overtopping event e
F_{KE}, F_{PE}	W/m	Kinetic and potential energy flux through the overtopping section
E_e, e_e	J/m, $\mathrm{J/m^3}$	Energy and specific energy (E_e/V_e) of event e
κ	–	Exponent of the energy–volume power law $E_e = aV_e^{\kappa}$
h_f, u_f	m, m/s	Flow depth and depth-averaged flow velocity at the overtopping section
K_r	–	Reflection coefficient $H_{m0,\mathrm{ref}}/H_{m0,\mathrm{inc}}$
q^*, R_c^*	–	Dimensionless discharge $\overline{q}/\sqrt{gH_{m0}^3}$ and freeboard R_c/H_{m0}
α	–	Water volume fraction (VOF). Slope angles are written α_u, α_d, or θ for a generic slope
ρ_w, ρ_a	$\mathrm{kg/m^3}$	Densities of water and air
n	–	Porosity of a porous layer
α_{vG}, β_{vG}	–	Darcy and Forchheimer resistance coefficients (van Gent)
Δ_{cell}	m	Cell size in the run-up refinement band

If COAST contributes to published work, please cite it as indicated in Section 15.3.

Chapter 2

Installation and Quick Start

2.1 Prerequisites

Table 2.1: Prerequisites.

Requirement	Details
Operating system	Linux (the job scripts are bash scripts)
OpenFOAM	v2112 or v2206, with the waves2Foam library compiled against the same version
Python	3.9 or newer
Python packages	`numpy`, `numpy-stl`, `shapely`, `scipy`, `pandas`, `matplotlib`, `seaborn`, `plotly`, `streamlit`, `reportlab` (installed from `requirements.txt`)
tmux	Installed locally by `install.sh` (no `sudo` required)
Hardware	A multi-core workstation or a cluster allocation. Depending on the mesh, the simulated duration, and the number of cores, a case runs for hours to days
Disk space	A long simulation can occupy tens of gigabytes (field output, stored iso-surfaces, sampling line)

2.2 Installation

Listing 2.1: Installation.

```
git clone <repository-url> COAST
cd COAST

./install.sh           # local tmux (no sudo), Python packages, bin/ on
   PATH
source ~/.bashrc
```

`install.sh` builds tmux locally in `tools/tmux/` (through `tools/install_tmux.sh`), installs the Python packages of `requirements.txt` with `pip`, and adds the command-line tools of `bin/` to the `PATH` in the shell start-up file.

2.3 Environment set-up

COAST launches every simulation inside a **tmux** session. For that session to inherit a working OpenFOAM environment, both OpenFOAM and waves2Foam must be sourced in the shell start-up file:

Listing 2.2: Additions to `~/.bashrc`.

```
source /path/to/OpenFOAM-v2206/etc/bashrc
```

```
source /path/to/waves2Foam/bin/bashrc
export PATH="$HOME/.local/bin:$PATH"
```

Installation paths are detected automatically. The generated job script (`job_Run.sh`) re-sources OpenFOAM and waves2Foam from the installation active in the shell where COAST is started (`$WM_PROJECT_DIR`, `$WAVES_DIR` or `$WM_PROJECT_USER_DIR`). For an installation in a non-standard location, set the paths explicitly before launching COAST:

```
export COAST_OPENFOAM_BASHRC=/path/to/OpenFOAM-v2206/etc/bashrc
export COAST_WAVES2FOAM_BASHRC=/path/to/waves2Foam/bin/bashrc
```

If the solver is still not found, the job stops immediately with an explicit message in `TMUX_OUT.log`.

2.4 Launching the application

Listing 2.3: Launching COAST from the repository root.

```
export PYTHONPATH="$(pwd):$(pwd)/python_tools"
streamlit run coast_app.py
# opens at http://localhost:8501
```

Start COAST from the repository root: case folders and the generated STL files (`STL_Geos/`) are created in the directory from which the application is started, while the post-processing page and the *Monitor* tab look for cases in the repository root. The post-processing page is reached from the Streamlit sidebar, or directly from the *Monitor* tab of any case. Stop the application with `Ctrl-C` in the terminal; running simulations are not affected, because they run in their own tmux sessions.

On a remote machine. When COAST runs on a server or a workstation reached by SSH, start it there and forward the two ports it uses, 8501 (the interface) and 8502 (the file server that opens the case reports of the *Monitor* tab), then open `http://localhost:8501` in the local browser:

```
ssh -L 8501:localhost:8501 -L 8502:localhost:8502 user@server
cd COAST && export PYTHONPATH="$(pwd):$(pwd)/python_tools"
streamlit run coast_app.py --server.headless true
```

If port 8501 is busy, Streamlit takes the next free port and prints it; forward that port instead.

On a desktop session. Step 1 of the overtopping analysis (`OV_Test.py`) ends with `plt.show()`. When COAST is started in a desktop session, with a display, its figures open in separate windows, and Step 1 does not finish until they are closed. To avoid it, start COAST with `export MPLBACKEND=Agg`: the figures are then only saved to files, as on a server without a display (Section 10.7).

2.5 A first case, step by step

The same sequence is shown with screenshots and real values, tab by tab, in Chapter 4.

1. **Sidebar.** Enter a *Case name* (it becomes the name of the case folder) and choose the *Flow model*: `rans` (k-ω SST) or `laminar`.
2. **Geometry.** Enter the EurOtop parameters of the structure (Chapter 5); optionally enable porous layers or add obstacles. Press *Generate STL*. The annotated cross-section appears: check that it is the structure you intended before going on.
3. **Wave and Domain.** Enter the deep-water wave conditions and the random-phase seed, the expected toe conditions, and the domain sizing (Chapter 6). Press *Size Domain*. COAST computes L_0 and the flume extent and shows the domain.
4. **Mesh.** Set the background cell size, the refinement levels, and the extent of the refinement bands (Chapter 7). Press *Preview Refinement* to see the bands and the position of the overtopping section.
5. **Simulation and Job.** Set the simulated duration, the time-step controls, the output intervals, the wall roughness (RANS), and the number of processes. Select structure segments if forces are wanted.
6. **Run.** Press *Build Case*: the log lists each dictionary as it is written. Then press *Launch Simulation*.
7. **Monitor.** Press *Refresh*, select the case, and follow its status and progress. Open the case report or the post-processing page from there.
8. **Post-processing.** When the run has finished, extract the wave gauges and work through the analysis tabs in order (Chapter 10).

Case names must be unique. Building a case whose name is already used by a folder in the working directory *deletes that folder* and replaces it with the new case, including any results it contains.

Before trusting a first result, check the incident waves: compare the measured spectrum with the target spectrum (Section 10.5), compare the wave-height statistics with Rayleigh (Section 10.4), and inspect K_r (Section 10.6). If the incident waves are wrong, every quantity derived from the run is wrong with them.

2.6 Command-line tools

For headless operation on a remote server, three scripts in `bin/` complement the *Monitor* tab:

Listing 2.4: Command-line tools.

```
joblist              # list all cases with status, run time, CPU, RAM,
   phase
joblist ./my_dir/    # restrict the scan to a directory
```

```
killcase <name>      # terminate a running case (partial name accepted)
rmcase   <dir> ...   # fast deletion of one or several case directories
```

2.7 Running the analysis outside the application

The analysis scripts can be run directly on a finished case. They read their settings from the top of each script (the application writes its own settings into the copies it runs; see Section 10.7.4).

Listing 2.5: Free-surface gauges, the four-step overtopping chain and the figures of one event from the command line.

```
cd Case_XXX/
R=<repo_root>/Rans_OF_Main

python $R/extract_from_freesurface.py --xgauge -8.0   # one gauge signal
python $R/OV_Test.py                # Step 1: discharge and events
python $R/kinetic.py                # Step 2: kinetic and potential
    energy flux
python $R/merge_OV_energy.py        # Step 3: event energy and energy-
    volume fits
python $R/flow_depth_velocity.py    # Step 4: flow depth and velocity per
     event
python $R/event_figures.py 1        # figures of one event (here event 1)
```

Chapter 3

Application Overview

3.1 Architecture

COAST is a multi-page Streamlit application backed by a library of geometry, meshing, and analysis modules. The entry point `coast_app.py` contains the case-generation and monitoring tabs; the analysis page is `pages/postprocessing.py`. Shared utilities live in `python_tools/`. The OpenFOAM case templates `Rans_OF_Main/` and `Laminar_OF_Main/` contain the dictionaries that COAST edits; `Rans_OF_Main/` also holds the analysis scripts and the wave-analysis library `wave_analysis_py_functions/`.

The workflow (Figure 3.1) has three phases. *Phase 1* (case preparation) translates physical parameters into a complete OpenFOAM case. *Phase 2* (execution) runs the meshing and solver chain in parallel. *Phase 3* (analysis) applies the post-processing pipeline of Chapter 10. Every case follows the same directory layout, so all simulations of a campaign are analysed identically.

3.2 Sidebar and case-generation tabs

The sidebar holds the *Case name*, the *Flow model*, and a status checklist (STL generated, domain sized, case built, simulation launched). A button at the bottom of the sidebar switches between the day and night themes.

The main page presents six tabs, worked through from left to right (Table 3.1). An action is refused until its prerequisite is complete: the domain cannot be sized before an STL exists, and a case cannot be built before the domain is sized. Regenerating the STL invalidates the domain and the built case; resizing the domain invalidates the built case. The status follows the buttons, not the inputs: an input changed after *Generate STL* or *Size Domain* is used only when that button is pressed again (Section 4.2).

Behaviour common to all tabs.

- The inputs are kept in the session of the browser tab until the case is launched; reloading the page restores the default values. There is no function to reload the inputs of a previous case: they are recorded in its `run_info.json` (Section 3.5).
- The symbol `?` next to an input shows its help text.
- The figures of the case-generation page are zoomable: mouse wheel to zoom, drag to move, double-click to restore, and the coordinates under the mouse are displayed.
- While a button is working, "Running…" is shown in the top right corner of the window.

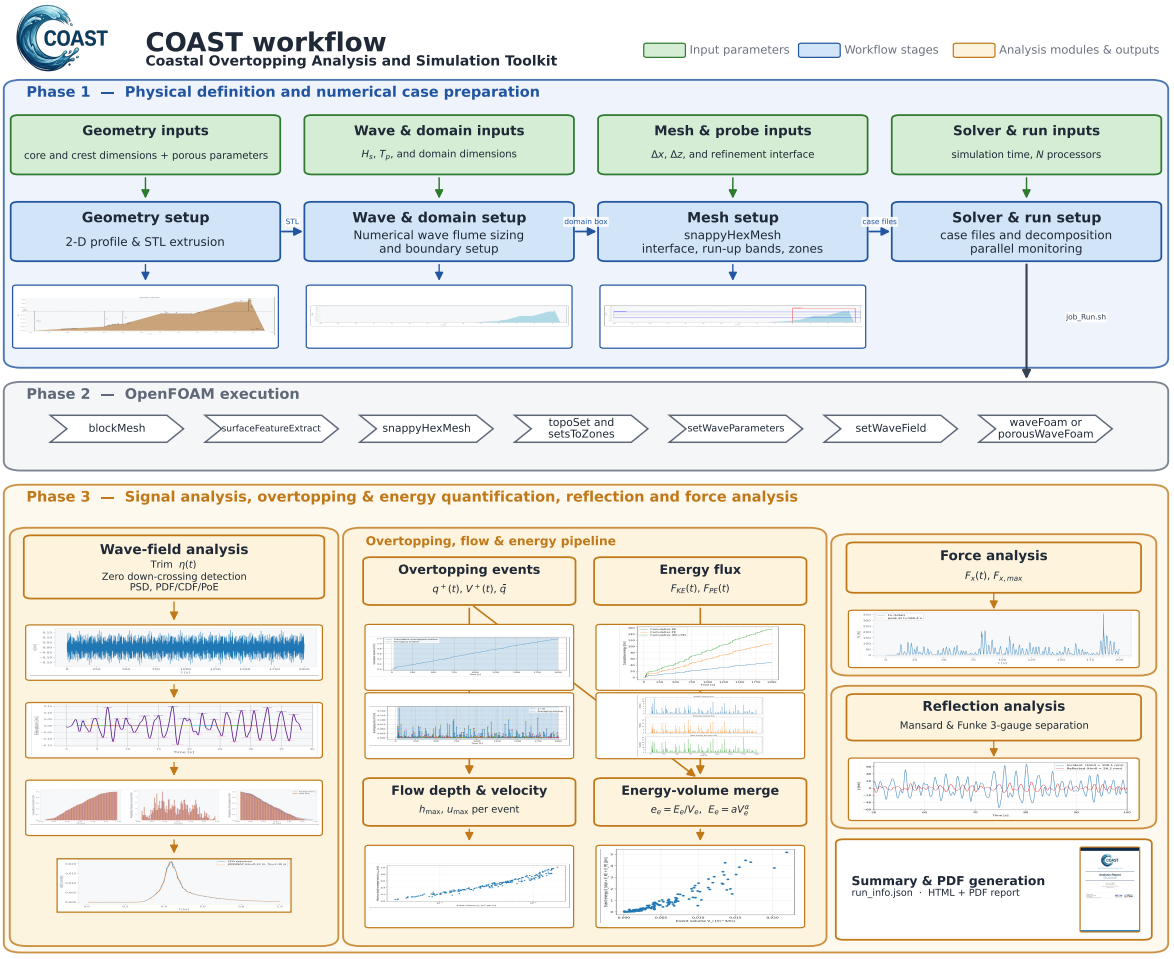


Figure 3.1: The COAST workflow, from the physical parameters (Phase 1) through the OpenFOAM execution (Phase 2) to the analysis of the wave field, overtopping, flow depth and velocity, energy, reflection and forces (Phase 3).

Table 3.1: Case-generation tabs.

Tab	Content and action
Geometry	EurOtop structure parameters, structure type (breakwater mode, collector), connector ramps, porous layers, obstacles. *Generate STL* writes the surfaces and shows the annotated cross-section
Wave and Domain	Deep-water wave conditions and random-phase seed, expected toe conditions, domain sizing (automatic or in metres). *Size Domain* computes and shows the flume
Mesh	Background cell size, refinement levels, interface band, run-up band. *Preview Refinement* shows the bands and the overtopping section
Simulation and Job	Time control, output intervals, interval of the overtopping sampling line, wall roughness (RANS), number of processes and decomposition method, structure segments for force measurement
Run	*Build Case* writes the case and its job script; *Launch Simulation* starts it in a tmux session
Monitor	All cases found under a scan directory, with status, run time, processes, memory, solver progress, and the last lines of the solver log; buttons to open the case report, open the post-processing page, or kill the run

3.3 Post-processing page

The post-processing page first asks for the case: every folder of the repository root that contains `run_info.json`, `TMUX_SOCKET.txt` or a `constant/` directory is listed, and any other path can be entered by hand. When the page is opened from the *Monitor* tab, the case is preselected.

Two panels above the tabs apply to the whole wave analysis:

- *Gauge and extraction settings*: the x-positions of the free-surface gauges, their y-position, the SWL elevation, and an option to overwrite existing gauge files. *Extract free-surface gauges* creates the signals (Section 10.1);
- *Analysis settings*: the trimming method and cut time (Section 10.2) and the segment length of the Welch spectrum (Section 10.5).

The ten analysis tabs (Table 3.2) are ordered as a workflow: the trimming chosen in the analysis settings applies to every wave-analysis tab, so it should be settled before reading any statistic.

Table 3.2: Post-processing tabs.

#	Tab	Purpose	Main outputs
1	Signals	Free-surface elevation at each gauge, with the target wave parameters	$\eta(t)$
2	Trimming	Removal of the start-up transient	t_{cut}
3	Wave Detection	Zero down-crossing decomposition	$\{H_j, T_j\}$, N_w
4	Statistics	Wave-height statistics and distributions	$H_{1/3}$, $H_{1/10}$, $H_{\max}$, PDF/CDF/PoE
5	Spectrum	Welch spectrum versus the target JONSWAP spectrum	$S(f)$, H_{m0}, T_p, $T_{m-1,0}$, T_{01}, T_{02}
6	Reflection	Three-gauge incident/reflected separation	K_r, $H_{m0,\mathrm{inc}}$, $H_{m0,\mathrm{ref}}$
7	Overtopping	Steps 1–4: discharge and events, energy flux, energy–volume analysis, flow depth and velocity; figures of one chosen event	$\overline{q}$, $\{V_e, E_e\}$, κ, $\{h_{f,\max}, u_{f,\max}\}$, event figures
8	Force	Horizontal load on the selected structure segments	$F_x(t)$, peak $\lvert F_x \rvert$
9	Summary	Table of the wave parameters of all gauges; HTML and PDF reports	CSV table, reports
10	Compare	Overlay of several cases: cumulative volume, force, energy flux, or event energy versus volume	Comparative figures

3.4 Case directory layout

Every case built by COAST has the same structure, which is what allows the analysis to run identically over a whole campaign:

Listing 3.1: Case directory layout (files written by the analysis are listed in Appendix B).

```
Case_XXX/
  0.orig/                          initial and boundary conditions
  constant/                        transport, turbulence, waveProperties(.
     input),
                                   porosityZones, triSurface/*.stl
  system/                          blockMeshDict, snappyHexMeshDict,
     controlDict,
                                   topoSetDict, createPatchDict,
                                      decomposeParDict, ...
  job_Run.sh                       generated execution script
  TMUX_SESSION.txt  TMUX_SOCKET.txt  TMUX_STATUS.txt  TMUX_OUT.log
                                   tmux session bookkeeping and full job
                                      output
  run_info.json                    complete parameter record (written at
     launch)
  run_report.html                  case report (written at launch)
```

```
figures/                        set-up figures (cross-section, domain,
    bands)
log.*                           logs of snappyHexMesh, topoSet, the
    solver, ...
processor*/                     decomposed mesh and results
postProcessing/
  overtopping/0/overtopping.dat    water flux through the overtopping
      section
  line/<time>/x01_*.xy             profiles on the sampling line x01
  freeSurface/<time>/              alpha = 0.5 iso-surface
  forceImpact_<patch>/0/force.dat  forces on selected segments (if
      requested)
Wave_Analysis/
  data/                         extracted gauge signals
      eta_from_freeSurface_x_*.txt
  output/                       quick plots of the extracted signals
  figures/                      figures of the wave-analysis tabs
  reflection_results.json       results of the Reflection tab
  wave_report.html              wave-analysis report
OV_Test.py, flow_depth_velocity.py    copies run by Steps 1 and 4
overtopping_*, energyFlux_*, merge_*, flow_depth_velocity_*, fdv_*
                                outputs of Steps 1 to 4
event_figures/                  figures of single overtopping events (if
    drawn)
<case>_report.pdf               PDF report (if generated)
```

3.5 Reproducibility: metadata and reports

When the simulation is launched, COAST writes `run_info.json` into the case folder, a record of the inputs: case name, date and machine; flow model, breakwater mode, collector and force segments; all geometry inputs; deep-water and toe waves, random-phase seed, n_{pre}, n_{post} and air clearance; flume width, SWL, background cell sizes, refinement levels and the buffer of the interface band, start offset and height of the run-up band, position, number of points and output interval of the sampling line; time control; number of processes and decomposition method; porous zones, layers and core; obstacles; and the list of set-up figures. Five inputs are *not* recorded: the end of the run-up band `RUNUP_X_END`, the wall roughness K_s and C_s, and the sizing mode with the manual inlet and outlet lengths. They are found in the dictionaries of the case: `system/snappyHexMeshDict` (box `runupBand`), `0.orig/nut` (patch `coast`), and `system/blockMeshDict` (extent of the domain). The values are those of the interface at the moment of the launch; if an input was changed after *Generate STL* or *Size Domain* without pressing the button again, the record and the case differ (Section 4.2). Three reports can be generated:

- the **case report** `run_report.html`, written at launch, which summarises the configuration (overview, geometry, waves, domain and grid, numerical control, patches and sampling, porous zones) with the set-up figures;
- the **wave-analysis report** `Wave_Analysis/wave_report.html`, generated from the *Summary* tab, which contains the wave-analysis figures and the tables and figures of overtopping Steps 1–4;
- the **PDF report** `<case>_report.pdf`, generated from the *Summary* tab, with a cover page, table of contents, list of figures, and sections for the case parameters, geometry and

domain, wave analysis, spectra, reflection, overtopping and energy flux (Steps 1–3), and forces.

A campaign is therefore an archive of self-documenting cases that can be regenerated, extended, or re-analysed later. This addresses a persistent weakness of manually assembled CFD studies, and it is the mechanism by which a result can be audited after the fact.

Chapter 4

Step-by-Step Guide: from the Parameters to the Results

This chapter goes through every tab of the two pages of COAST, with screenshots of the interface. It shows what to enter, what each button does, what appears on the screen, which files are written, and what to check before going on. The methods behind each step are documented in the technical chapters that follow; each section points to them. Two examples are used:

- the **case-generation page** (Sections 4.1 to 4.9) is shown with a **demonstration structure** that contains every element of the parametric profile: foreshore, toe berm, lower slope, berm, upper slope, crest, crown wall and landward slope. Variants of the same structure show the collector, the breakwater mode, porous layers and an obstacle;
- the **post-processing page** (Sections 4.11 to 4.15) is shown with a completed run, a test of the EurOtop database taken as an example (Section 4.10), because the analysis needs the results of a simulation.

The values in the screenshots are **those of these examples**: they are neither the default values of the application nor recommendations.

4.1 The demonstration structure

The demonstration structure is the profile of Figure 5.1 with a value for every parameter (Table 4.1): a dike on a 1:10 foreshore, with a toe berm, a submerged berm between two 1:2 slopes, a crest with a crown wall, and a landward slope. Its dimensions and its waves are an **illustration** chosen for this chapter: they are not a test of the database, and the case is built but not run. The numerical settings (mesh, time control, processes) are those of the demonstration campaign, also used by the run analysed in the second part of the chapter; the wall roughness is the one of the smooth cases of the campaign (Section 11.3).

4.2 Before starting: how the interface keeps the inputs

- **The inputs live in the browser tab.** Every value typed in the interface is kept in the session of the browser tab, until the case is launched. Reloading the page, or opening COAST in a new browser tab, starts a new session in which every input is back to its default value. Nothing is saved before *Launch Simulation*, which writes `run_info.json` into the case folder. The interface has no "load case" button: to set up a case like a previous one, enter again the values recorded in its `run_info.json` (Section 3.5 lists what this file contains).
- **Each button uses the values present when it is pressed.** *Generate STL* stores the geometry, *Size Domain* stores the domain and L_0, and *Build Case* writes the case from these stored results and from the current values of the other tabs. Changing a geometry input after *Generate STL* does not change the stored STL. After any change

Table 4.1: Inputs of the demonstration structure (case name `Demo_structure`).

Element	**Inputs**
Foreshore	`m` = 10 (slope 1:10), from `h_deep` = 0.75 m offshore to `h` = 0.60 m at the toe, `x_toe` = 0
Toe berm	`h_t` = 0.50 m, `B_t` = 0.20 m; joined to the toe by the connector `dx_berm_connector` = 0.20 m
Lower slope	`cot_alpha_d` = 2
Berm	`h_b` = 0.20 m (below the SWL), `B` = 0.60 m, horizontal (`tan_alpha_B` = 0)
Upper slope	`cot_alpha_u` = 2
Crest	`A_c` = 0.35 m, `G_c` = 0.30 m
Crown wall	`R_c` = 0.55 m (wall 0.20 m above the crest), `t_wall` = 0.06 m
Landward slope	`L_back` = 0.50 m
Waves	Deep water: `Hm0_deep` = 0.12 m, `Tp_deep` = 1.80 s, `Tm_deep` = 1.50 s, seed 1234. Toe: `Hm0_toe` = 0.115 m, `Tp_toe` = 1.80 s, `Tm_toe` = 1.50 s, `Tm10_toe` = 1.65 s
Numerics	As the analysed run (Section 4.10); $K_s = 10^{-3}$ m, $C_s = 0.5$

in the *Geometry* tab, press *Generate STL* again, then *Size Domain*; after a change of `Tp_deep`, `n_pre`, `n_post` or `top_clearance`, press *Size Domain* again. The status list of the sidebar is reset only when an upstream button is pressed again: it does not detect that an input has been changed (Section 3.2).

- **The case name.** It becomes the name of the case folder, of the STL files (`STL_Geos/<name>.stl`), of the tmux session, and of its socket `/tmp/tmux_<name>.sock`. Use letters, digits, `-`, `_` and `.` only, and a name that is not used by another folder: *Build Case* deletes a folder of the same name.
- **Help.** The symbol `?` next to an input shows a short explanation when the mouse is placed on it.
- **Figures.** The figures of the case-generation page are shown in a frame in which the mouse wheel zooms, dragging moves the view, double-clicking restores it, and the coordinates under the mouse are displayed. The interactive views (obstacles, force segments) have the usual Plotly tool bar.
- **Busy indicator.** While a button is working, Streamlit shows "Running..." in the top right corner; wait until it disappears before pressing another button.

4.3 The window and the sidebar

The sidebar (Figure 4.1) is the same on every tab:

1. **Page selector.** `coast app` is the case-generation page described in Sections 4.4 to 4.9; `postprocessing` is the analysis page (Sections 4.11 to 4.15).
2. **Case name.** Here `Demo_structure`, a name that no other folder uses (*Build Case* would replace that folder).
3. **Flow model.** `rans` (k-ω SST) or `laminar` (Section 8.4); it selects the template that is copied into the case.
4. **Status.** Four marks: STL generated, domain sized, case built, simulation launched. A red cross means that the step has not been done since the last change upstream.

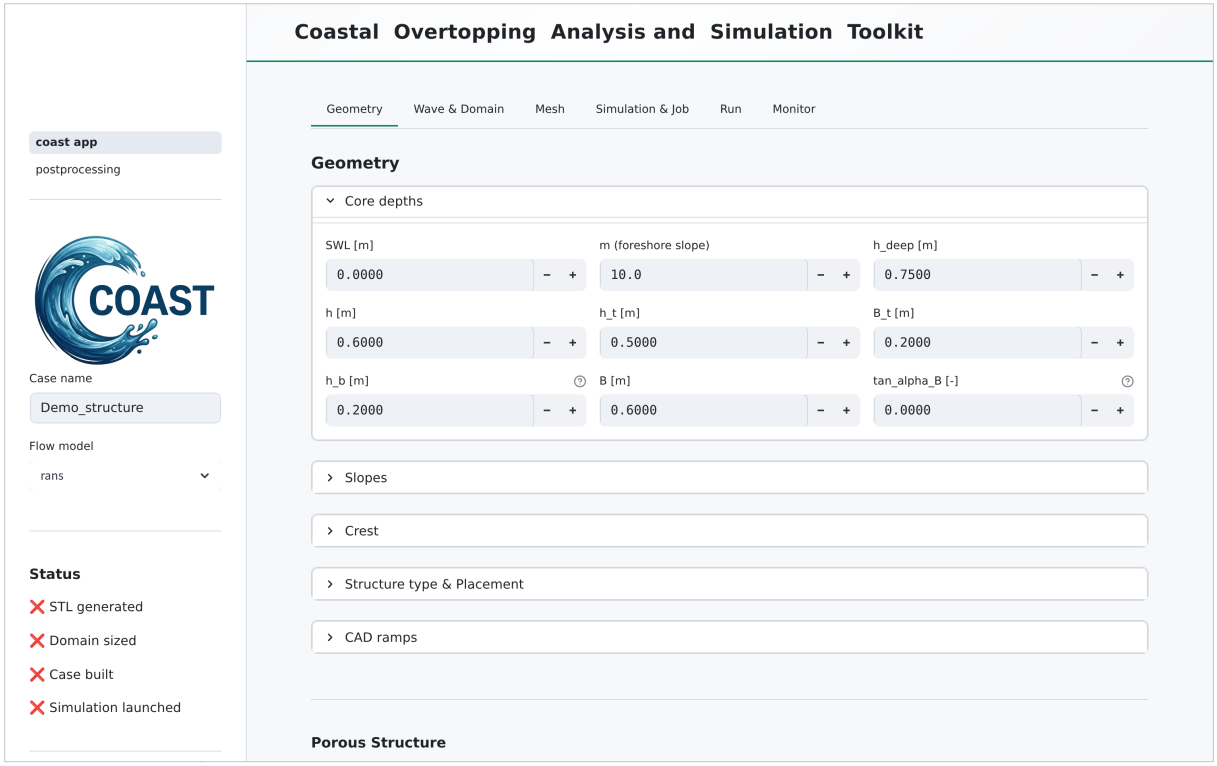


Figure 4.1: The COAST window with the inputs of the demonstration structure (case-generation page, *Geometry* tab). Sidebar, from the top: the two pages of the application (`coast app`: case generation and monitoring; `postprocessing`: analysis), the logo, the *Case name* and the *Flow model*, and the *Status* list. The six tabs of the page are above the inputs.

5. At the bottom of the sidebar: the contact and a button with a moon or a sun, which switches between the day and night themes.

4.4 Tab *Geometry*

1. Enter the structure. The inputs are grouped in five panels (Figure 4.2); click on the title of a panel to open or close it. For the demonstration structure (Table 4.1):

- *Core depths*: `SWL` = 0; foreshore slope `m` = 10 (1:10), from `h_deep` = 0.75 m offshore to `h` = 0.60 m at the toe; toe berm at `h_t` = 0.50 m, `B_t` = 0.20 m wide; berm at `h_b` = 0.20 m below the SWL (`h_b` is negative for a berm above the SWL), `B` = 0.60 m wide and horizontal (`tan_alpha_B` = 0; a positive value makes the berm rise landward, and `h_b` is then the depth at the middle of the berm). `m` = 1000 is the flat bed of a flume, without foreshore;
- *Slopes*: `cot_alpha_d` = 2 between the toe berm and the berm, `cot_alpha_u` = 2 between the berm and the crest. A cotangent 0 gives a vertical face;
- *Crest*: `A_c` = 0.35 m above the SWL, crest width `G_c` = 0.30 m. `R_c` = 0.55 m is larger than `A_c`: a crown wall 0.20 m high and `t_wall` = 0.06 m thick is placed at the landward end of the crest. With `R_c` = `A_c` there is no wall and `t_wall` is not used;
- *Structure type and Placement*: breakwater mode off; toe at `x_toe` = 0; `x_margin` = 0 (no flat bed seaward of the foreshore); width `Ly` = 1 m; `ZMIN_THICKNESS` = 0.001 m, the thickness of solid kept under the deepest bed level; landward slope over `L_back` = 0.50 m; no collector (`USE_COLLECTOR` unticked; Section 4.4.1);
- *CAD ramps*: `dx_foreshore_ramp` and `dx_toe_transition` are used on a flat flume (`m` = 1000); with a sloping foreshore, as here, the toe berm is joined to the toe by the connector `dx_berm_connector` = 0.20 m (Section 5.2).

The structure is impermeable: *Enable porous zones* stays unticked (variant in Section 4.4.3), and no obstacle is added (variant in Section 4.4.4).

2. Press *Generate STL*. COAST builds the cross-section, extrudes it over `Ly`, writes `STL_Geos/Demo_structure.stl`, and draws the section with the EurOtop symbols (Figure 4.3). Each symbol is drawn on the element it describes, as in Figure 5.1: h_{deep} at the offshore end, the foreshore slope m, h at the toe ($x = 0$), the connector ramp (`RAMP_BT`), B_t and h_t, α_d, h_b and B, α_u, G_c and A_c, R_c and t_{wall}, and L_{back}. When two parameters are equal, their labels are grouped (for example $h_{\text{deep}} = h = h_t$ on a flat flume without toe berm).

> **Check before going on.** Is the section the structure you intended? Compare the levels and lengths with the drawing of the test (the mouse gives the coordinates in the figure). A wrong sign of `h_b` (negative above the SWL), a slope entered as $\tan\alpha$ instead of $\cot\alpha$, or a crest width forgotten, are seen here and nowhere else.

4.4.1 Variant: overtopping collector

Tick `USE_COLLECTOR` in *Structure type and Placement* (Figure 4.4) and press *Generate STL* again. The landward slope is replaced by a box of length L_{back} behind the crown wall (behind

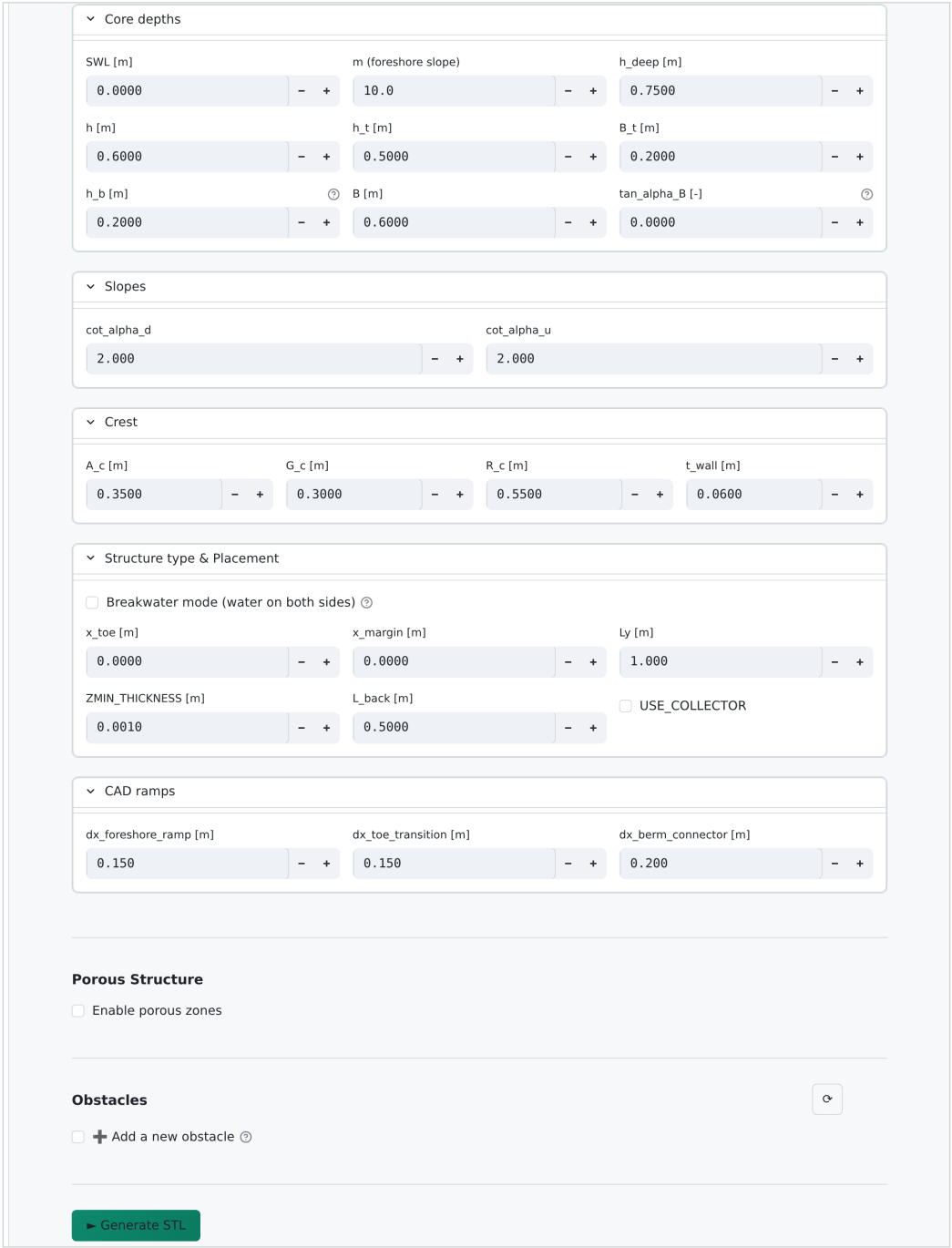


Figure 4.2: Tab *Geometry*, all panels open, with the inputs of the demonstration structure. Below the five panels: *Porous Structure*, *Obstacles*, and the button *Generate STL*.

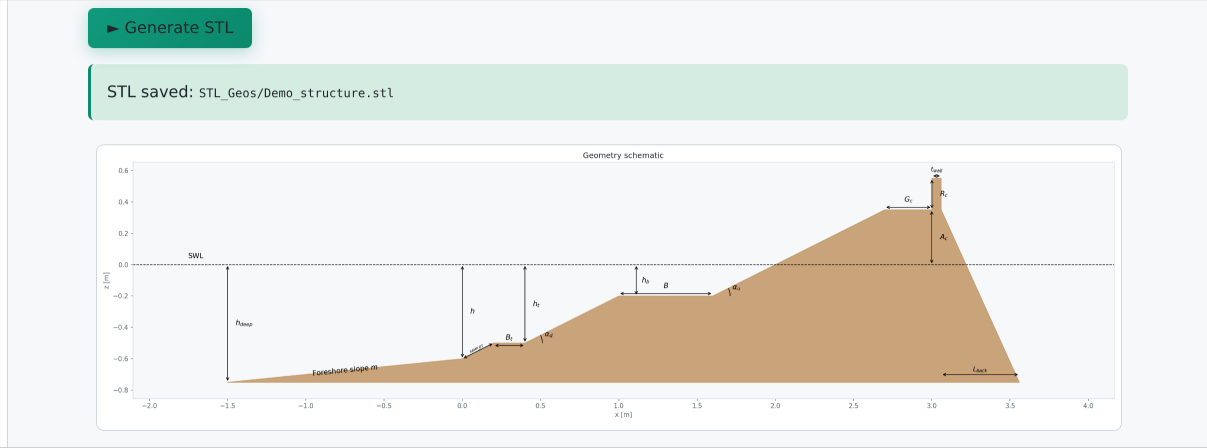


Figure 4.3: After *Generate STL*: the message with the STL file, and the cross-section of the demonstration structure: foreshore from $x = -1.5$ m to the toe at $x = 0$, toe berm from 0.20 to 0.40 m, berm from 1.00 to 1.60 m, crest from 2.70 to 3.00 m, crown wall from 3.00 to 3.06 m, landward slope down to the bed at $x = 3.56$ m.

the crest when there is no wall): its floor is just above the bottom of the domain and its landward side rises to the crest level (Figure 4.5). The box keeps the overtopped water, like the overtopping tank of a laboratory flume, and the volume it collects gives an independent estimate of the discharge (`LOOP_OVERTOPPING.py`, Section 5.3). The standard discharge of Step 1 does not need it. The overtopping section `x01` is placed at the entrance of the box, `x_container_start`; without the collector it is at the start of the landward slope. Here both are at the landward face of the wall, $x = 3.06$ m.

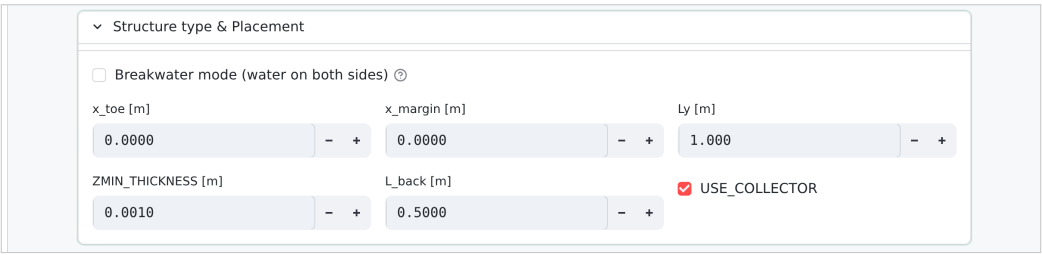


Figure 4.4: Panel *Structure type and Placement* with `USE_COLLECTOR` ticked; `L_back` is then the length of the box.

4.4.2 Variant: breakwater mode

For a breakwater or a low-crested structure with water on both sides, tick *Breakwater mode (water on both sides)* in *Structure type and Placement* (Figure 4.6). The message under the box recalls the two consequences: the collector is disabled (its box disappears from the panel), and an outlet relaxation zone will be added at the landward end of the flume to absorb the transmitted waves (Section 6.5). The section of the structure is the same as in Figure 4.3; at the start of the simulation the flume is filled with water up to the SWL on both sides of the structure, instead of being dry behind it.

The outlet relaxation zone needs room behind the structure. In the tab *Wave and Domain*, the panel *Domain sizing* raises `n_post` to 2 when it is smaller than 1.5 and says so (Figure 4.7); the message is shown only until the next action on the page. *Size Domain* then gives a domain that

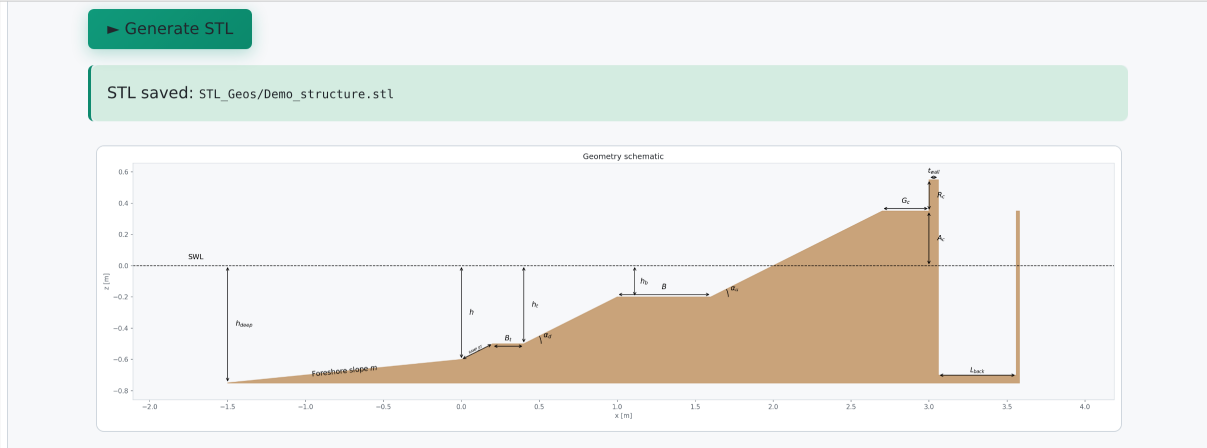


Figure 4.5: The demonstration structure with the collector: a box from $x = 3.06$ to 3.56 m behind the crown wall, whose landward side rises to the crest level ($z = 0.35$ m).

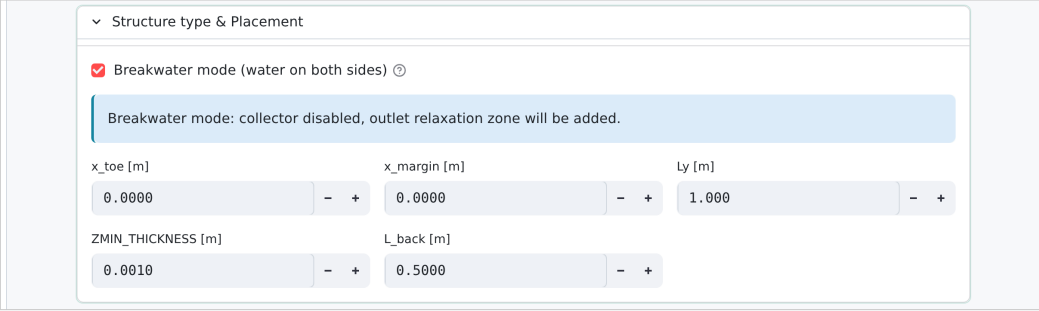


Figure 4.6: Panel *Structure type and Placement* with *Breakwater mode* ticked: the collector is disabled and an outlet relaxation zone will be added.

extends $2\,L_0 = 10.12$ m behind the structure, to $x = 13.677$ m (Figure 4.8), and *Build Case* places the outlet zone, $1.5\,L_0$ long, at the landward end. The mesh, the simulation time and the disk space grow with the length of the domain.

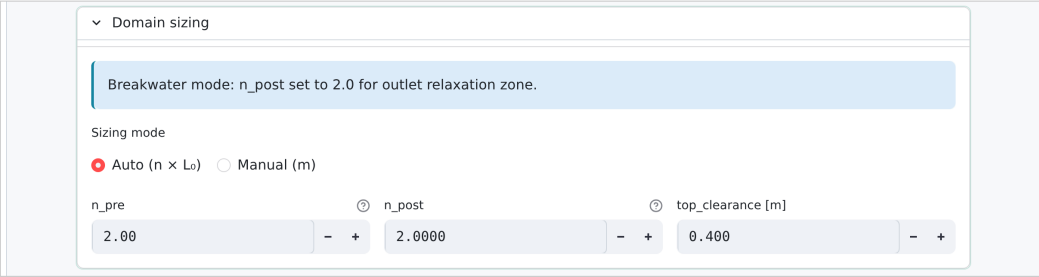


Figure 4.7: Tab *Wave and Domain* in breakwater mode: `n_post` raised to 2 for the outlet relaxation zone.

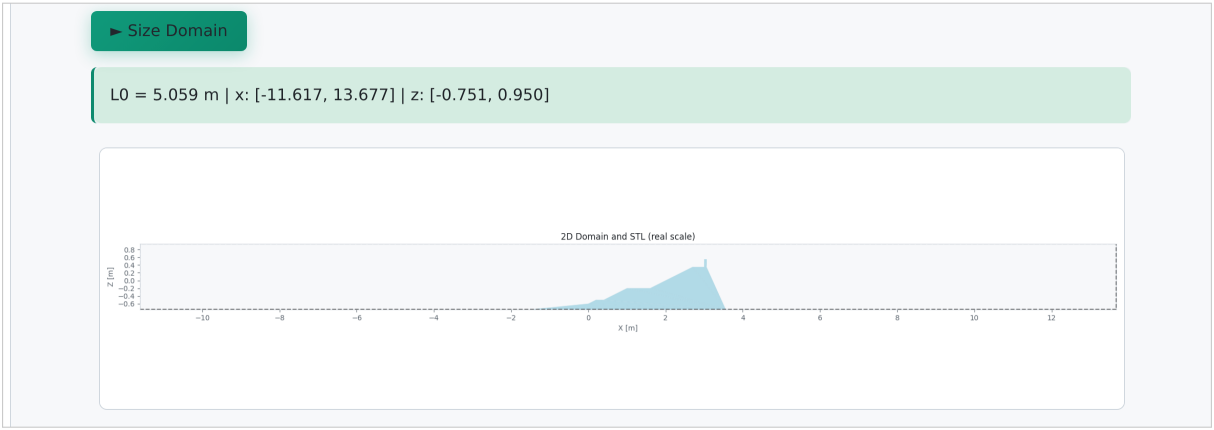


Figure 4.8: *Size Domain* in breakwater mode: the domain extends two wavelengths behind the structure.

4.4.3 Variant: a structure with porous layers

Figures 4.9 and 4.10 give the demonstration structure two porous layers, an armour and a filter, from the toe berm to the crest, on an impermeable core. The armour is taken as two layers of rock of $D_{n50} = 0.035$ m with $k_t = 1$: thickness $t = 2 \times 1 \times 0.035 = 0.07$ m normal to the slope, hence a vertical thickness $d = t/\cos\alpha = 1.12 \times 0.07 \approx 0.08$ m on the 1:2 slopes (Eq. 5.10), and $d_{50} = D_{n50}/0.84 = 0.042$ m. The filter is 0.04 m thick with $d_{50} = 0.015$ m. These values are an illustration; the resistance coefficients ($n = 0.416$, $\alpha = 200$, $\beta = 0.7$, KC $= 128$, $\gamma_{\mathrm{AM}} = 0.34$) are those of the demonstration campaign (Section 11.3). The steps are:

1. tick *Enable porous zones*;
2. tick the zones of the profile that carry the layers (here `toe_berm`, `lower_slope`, `berm`, `upper_slope` and `crest`). The zones are the elements of the profile: `foreshore`, `toe_slope` (the connector ramp), `toe_berm`, `lower_slope`, `berm`, `upper_slope`, `crest` and `back_slope`;
3. choose the *Number of layers* (here 2: `armor` and `filter`), from the outside to the inside;
4. for each layer, open its panel and enter its name, its *vertical* thickness d, and, in *Resistance parameters*, n, d_{50}, KC, α_{vG}, β_{vG} and γ_{AM} (Section 8.6). *Zone thickness overrides* and

Zone physics overrides give another thickness, porosity or d_{50} on one zone;

5. keep *Solid core* ticked for an impermeable core, or untick it and enter the porosity, d_{50}, α_{vG} and β_{vG} of a porous core;
6. press *Generate STL*.

The number of armour layers and the thickness of a test of the database follow from its roughness factor γ_f (Section 5.5.1, Table 5.3).

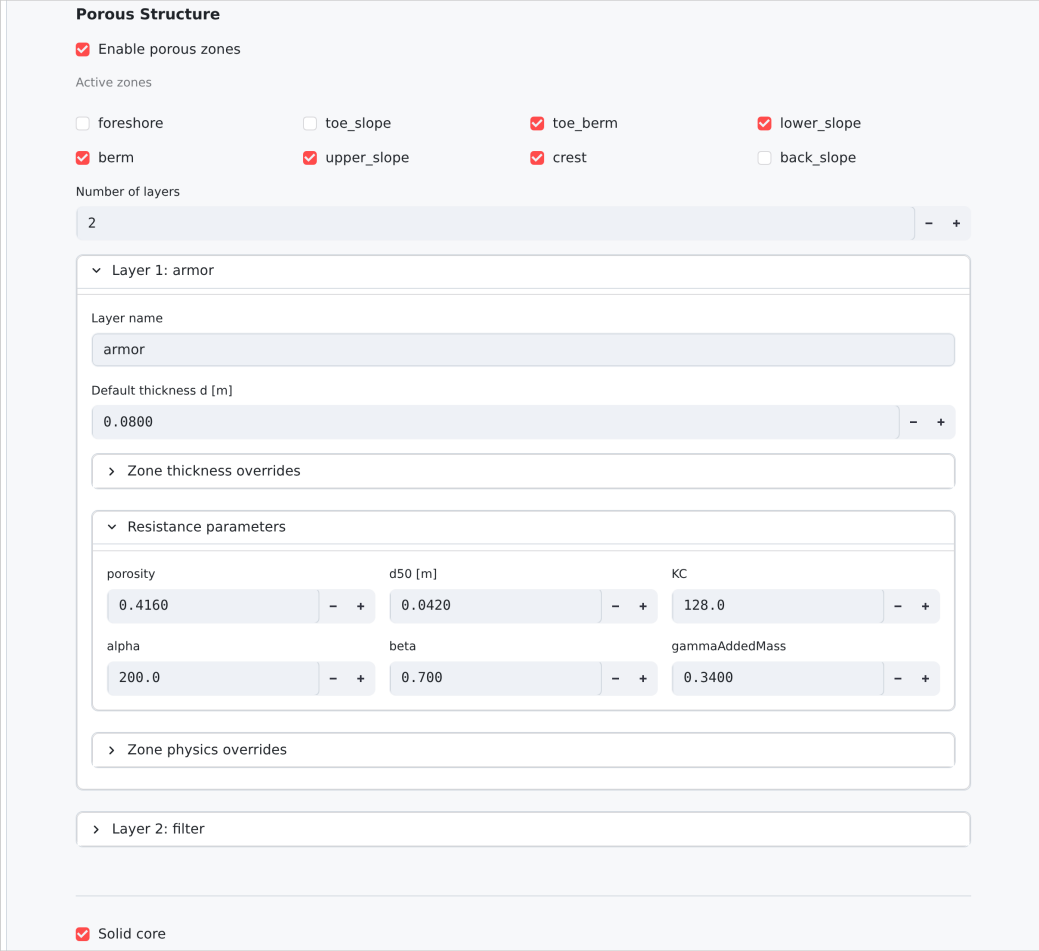


Figure 4.9: Panel *Porous Structure*: active zones, number of layers, and the panel of layer 1 (armour) with its resistance parameters.

A zone that does not exist in the profile is skipped: `crest` with $G_c = 0$, `berm` with $B = 0$, and `back_slope` when there is a crown wall or a collector. The message under *Generate STL* lists the zones that were built.

4.4.4 Variant: adding an obstacle

An obstacle is a solid added to the profile, for example a block, a storm wall or a recurve (Section 5.4). Figures 4.11 to 4.13 add, as an illustration, a block 0.10 m wide in the middle of the berm, whose top is 0.08 m above the berm:

1. tick *Add a new obstacle*: an interactive view of the profile appears; hover the mouse over the view to read coordinates;

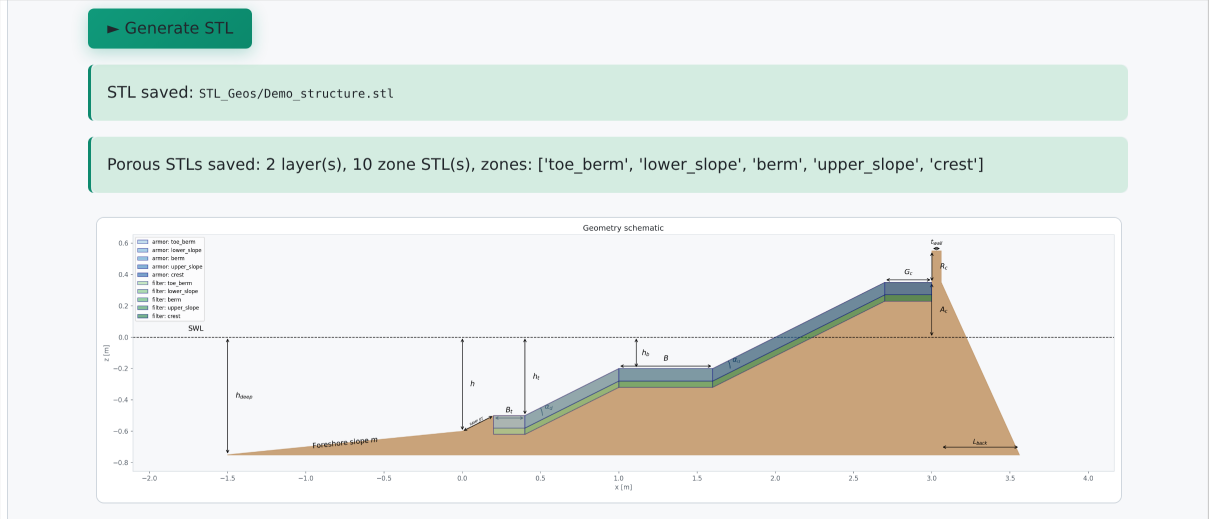


Figure 4.10: After *Generate STL* with porous layers: one STL per layer and zone (2 layers × 5 zones), and the section with the armour (blue) and the filter (green) from the toe berm to the crest. The layers stop at the crown wall.

2. choose *Rectangle* and enter *Left x*, *Right x*, *Bottom z* and *Top z* (here 1.25, 1.35, −0.25 and −0.12 m), or choose *Polygon vertices* and type one `x, z` pair per line; an existing STL can also be imported in *Import STL obstacle*;
3. press *Add rectangle obstacle*: the obstacle appears in the view and in the list *Current obstacles*, where its width, height and centre can be changed (*Apply dimensions*) or the obstacle removed;
4. press *Generate STL*: the obstacle is merged with the profile into a single solid.

The bottom of the rectangle is set 0.05 m below the berm, so that the block is anchored in the profile; the part below the surface disappears in the union.

4.5 Tab *Wave and Domain*

1. Enter the waves. *Deep-water conditions*: $H_{m0} = 0.12$ m, $T_p = 1.80$ s, $T_m = 1.50$ s, and the random-phase seed 1234 (Figure 4.14). The wave generation uses these values. *Toe conditions*: the expected values at the toe ($H_{m0} = 0.115$ m, $T_p = 1.80$ s, $T_m = 1.50$ s, $T_{m-1,0} = 1.65$ s); of these, only `Hm0_toe` acts on the case, through the height of the interface band (next tab). For a test of the database, enter its values at the toe (Table 5.2).

2. Size the domain. *Domain sizing*, mode *Auto*: $n_{\text{pre}} = 2$, $n_{\text{post}} = 0.02$, `top_clearance` = 0.4 m. Press *Size Domain*. The message gives

- $L_0 = gT_p^2/2\pi = 5.059$ m;
- $x \in [-11.617, 3.661]$ m: the domain starts $2L_0 = 10.12$ m seaward of the offshore end of the structure ($x = -1.5$ m) and ends $0.02\,L_0 = 0.10$ m behind its landward end ($x = 3.56$ m);
- $z \in [-0.751, 0.950]$ m: from the bottom of the STL to the top of the crown wall plus the clearance, $0.55 + 0.40 = 0.95$ m.

The mode *Manual* takes the inlet and outlet lengths in metres instead of multiples of L_0. The inlet relaxation zone occupies the first wavelength, $x \in [-11.617, -6.559]$ m (it is written in

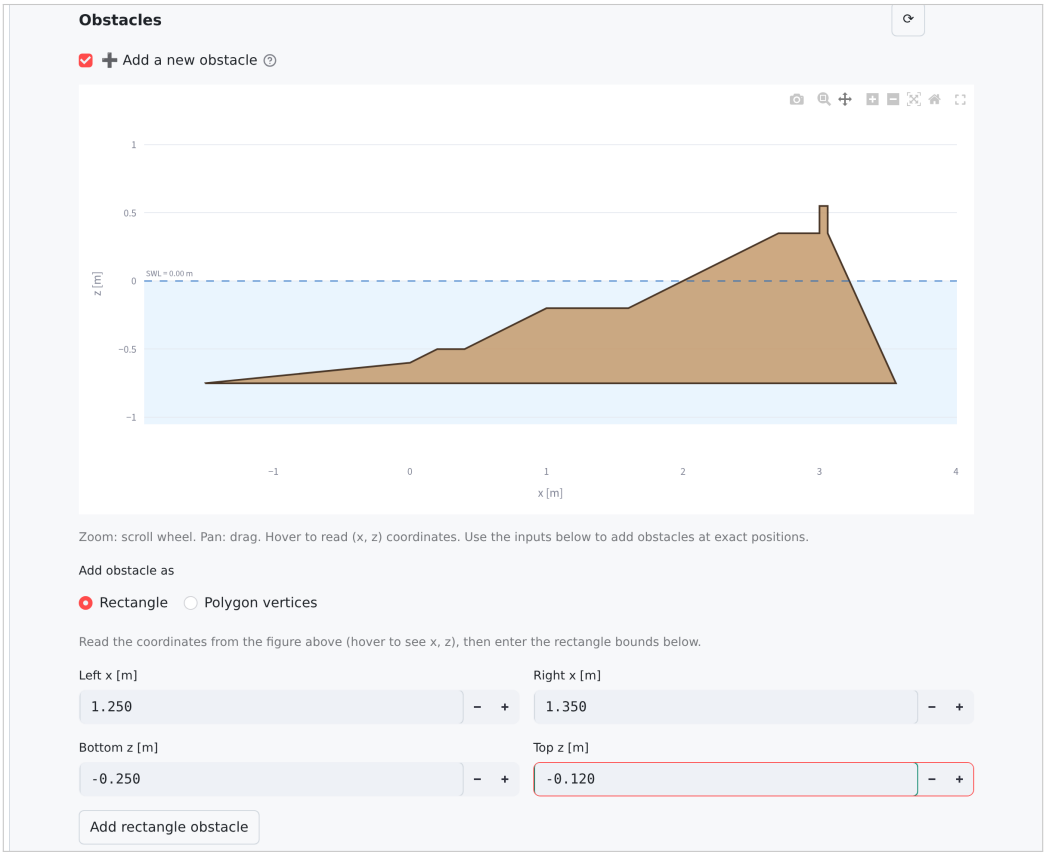


Figure 4.11: *Add a new obstacle*: interactive view of the profile, and the inputs of a rectangle.

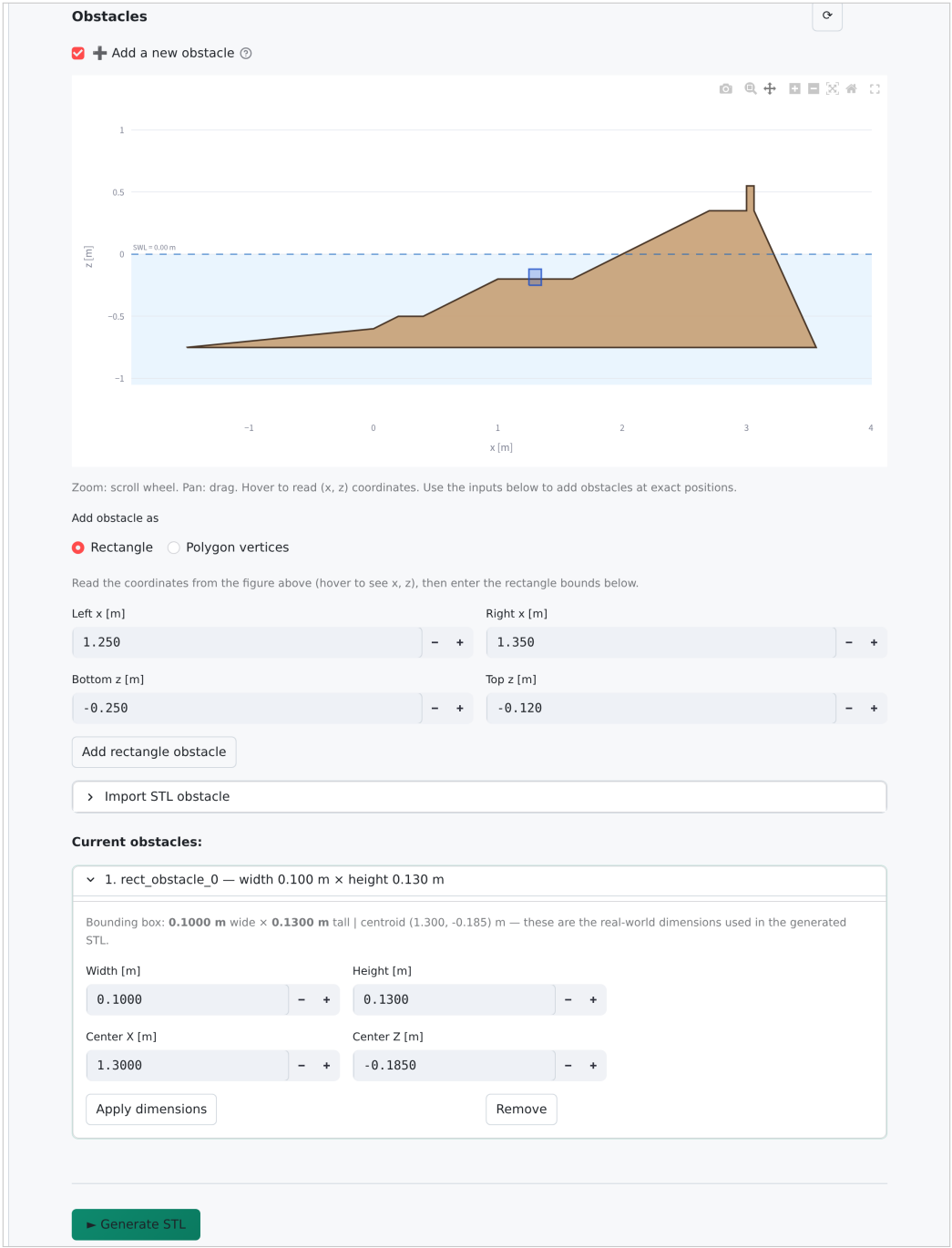


Figure 4.12: After *Add rectangle obstacle*: the obstacle in the view (blue) and its panel in *Current obstacles*, with its bounding box, width, height and centre.

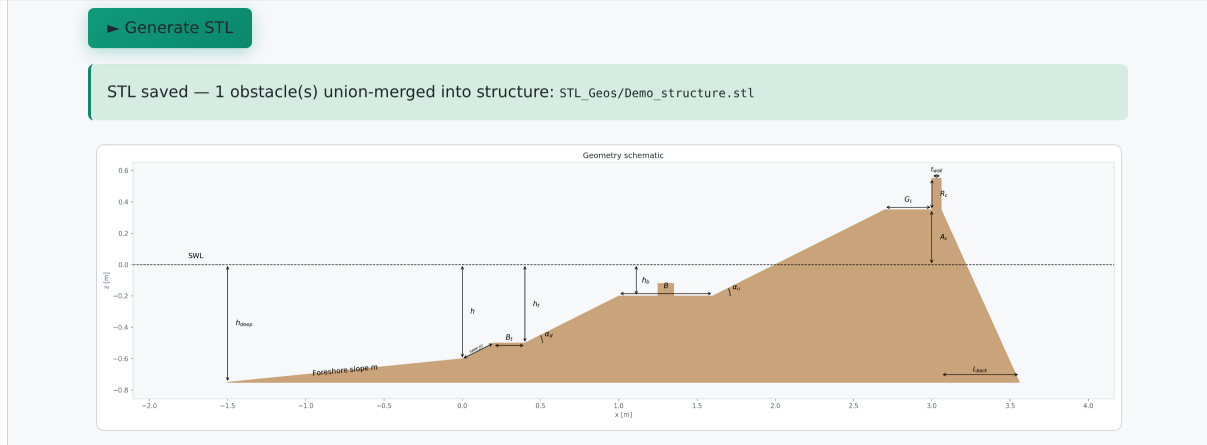


Figure 4.13: After *Generate STL*: the block is merged with the profile (2D union) before the extrusion.

`constant/waveProperties.input` of the case); the waves then propagate freely over 5.06 m before they reach the foreshore. The free-surface gauges of the analysis are placed in this region (Section 4.13). Figure 4.15 shows the resulting flume, with the refinement bands of the next tab.

4.6 Tab *Mesh*

Enter the background cell size, the refinement levels and the bands (Figure 4.16), then press *Preview Refinement*. For the demonstration structure:

- background cells $\Delta x = \Delta z = 0.05$ m and one cell in y; the build log will give $306 \times 1 \times 35$ cells (Section 4.8);
- levels 2 in both bands: cells of $0.05/2^2 = 0.0125$ m, i.e. $H_{m0}/\Delta_{\text{cell}} \approx 9$ cells per significant wave height;
- interface band: $z \in \pm(H_{m0,\text{toe}} + 0.02) = \pm 0.135$ m over the whole flume (blue in the first figure);
- run-up band (red): from 0.5 m landward of the offshore end of the STL ($x = -1.0$ m) to `RUNUP_X_END` $= 3.2$ m, beyond the crown wall, and up to 0.2 m above the top of the STL, $z = 0.75$ m.

The second figure shows the vertical line `x01` at the overtopping section, $x = 3.06$ m, at the landward face of the crown wall, from its top ($z = 0.55$ m) to the top of the run-up band: the discharge, the energy flux, and the flow depth and velocity are all measured there (Section 7.5).

Check before going on. The interface band must contain the waves over the whole flume, and the run-up band must contain the slope, the crest, the overtopping section and the space above it (Section 7.4).

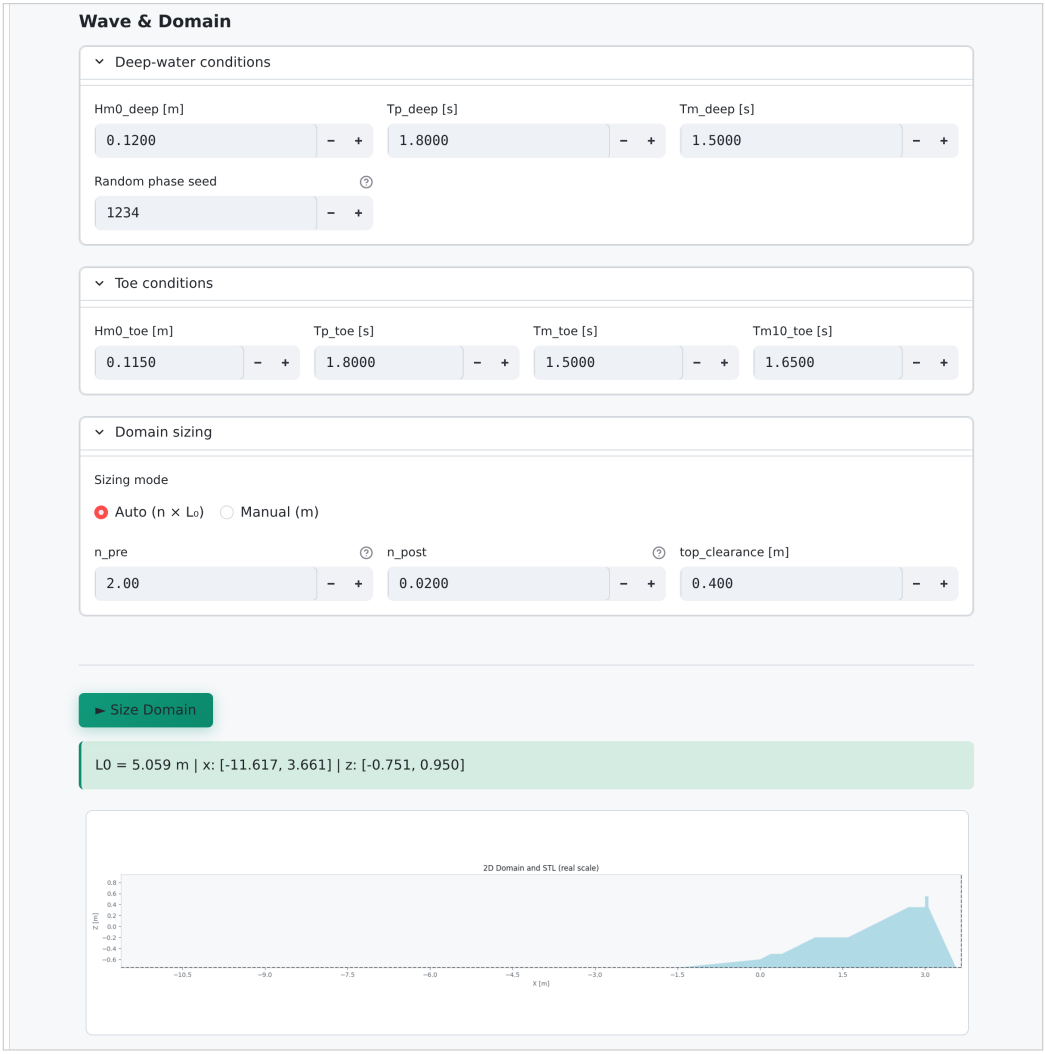


Figure 4.14: Tab *Wave and Domain* with the inputs of the demonstration structure, and the result of *Size Domain*: L_0, the extent of the domain, and the domain drawn at true scale with the structure.

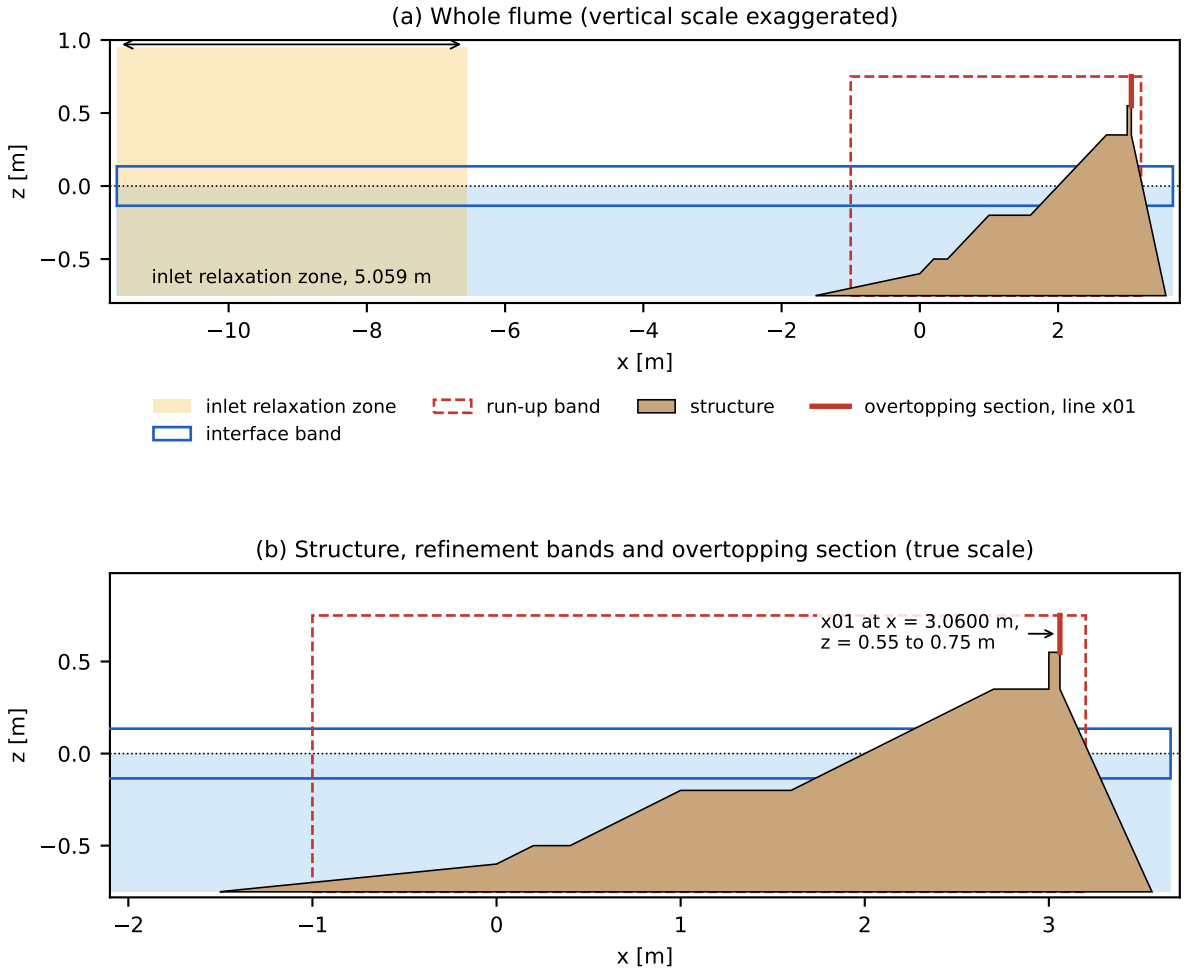


Figure 4.15: The numerical flume of the demonstration structure after *Build Case*: domain, inlet relaxation zone, refinement bands, structure, and overtopping section (line `x01`).

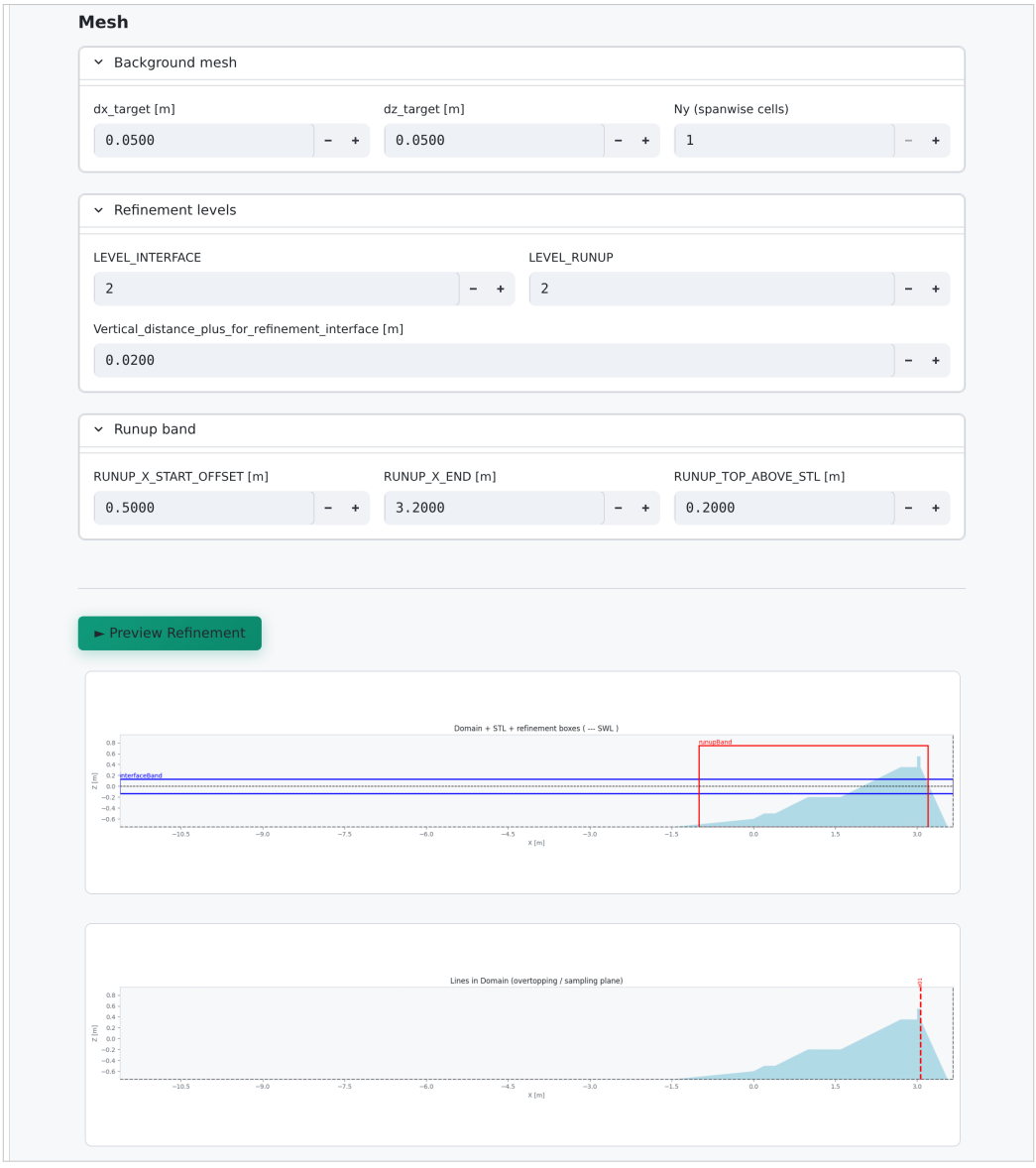


Figure 4.16: Tab *Mesh*: background mesh, refinement levels, run-up band; *Preview Refinement* draws the refinement bands and the position of the overtopping section (`x01`).

4.7 Tab *Simulation and Job*

The settings of Figure 4.17 are those of the demonstration campaign:

- *Simulation control*: `END_TIME` = 1000 s (about 550 peak periods), `DELTA_T` = 0.01 s (first time step only), `WRITE_INTERVAL` = 0.1 s (fields, free surface, and hence the sampling of the gauges and the snapshots of the event figures), `MAX_CO` = `MAX_ALPHA_CO` = 0.8;
- *Overtopping flow sampling*: line `x01` written every 0.02 s;
- *RANS roughness*: $K_s = 10^{-3}$ m and $C_s = 0.5$ on the structure (this panel is shown only for the RANS model);
- *HPC / Decomposition*: 8 processes, method `scotch`.

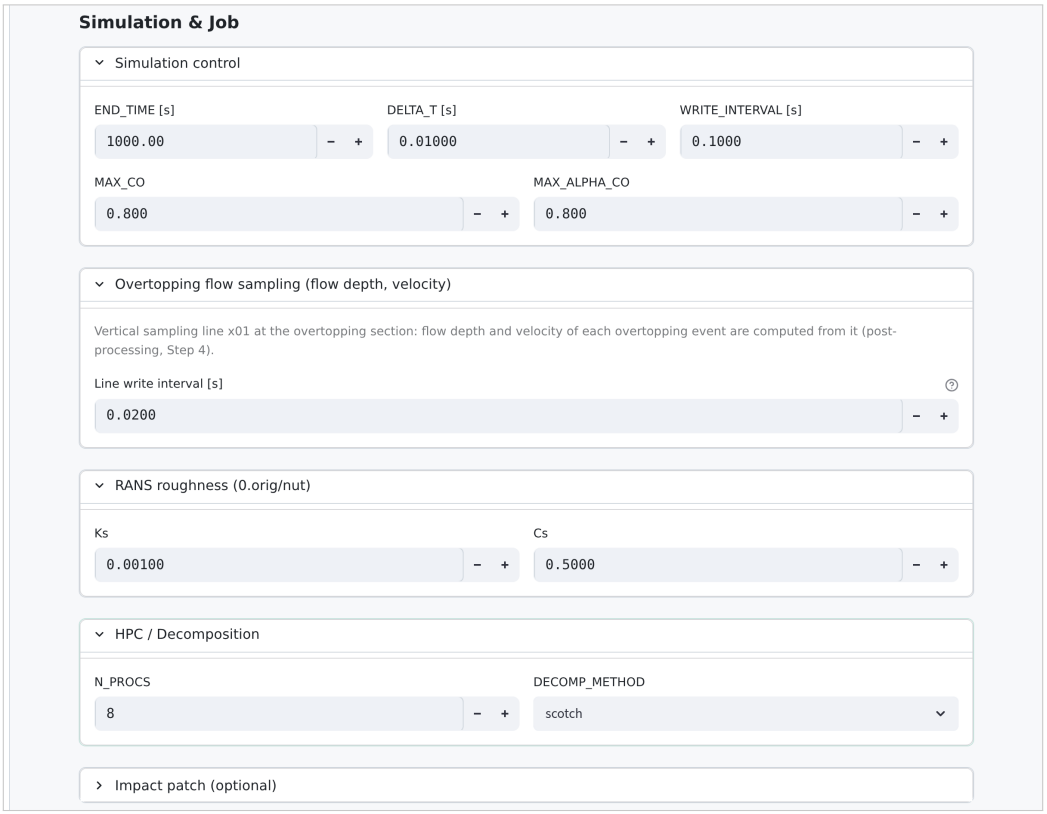


Figure 4.17: Tab *Simulation and Job* with the settings of the demonstration structure.

Force on parts of the structure (optional). Open *Impact patch*, tick *Create new force/impact patch*: the profile appears split into numbered segments (Figure 4.18). For the demonstration structure they are: 1, the short vertical face at the offshore end; 2, the foreshore; 3, the connector ramp; 4, the toe berm; 5, the lower slope; 6, the berm; 7, the upper slope; 8, the crest; 9, the seaward face of the crown wall; 10, its top; 11, its landward face; 12, the landward slope. Select the segments whose force is wanted in *Select segment(s) for force measurement*: here segment 9, the face of the wall hit by the waves. *Build Case* writes the patch `force_seg_09` into `createPatchDict`, and `job_Run.sh` creates it at the start of the run, with the force function `forceImpact_force_seg_09` (Section 5.6). `SOURCE_PATCH` is the patch of the structure from which the faces are taken; keep `coast`. Segments inside a porous zone are drawn dashed and cannot be selected.

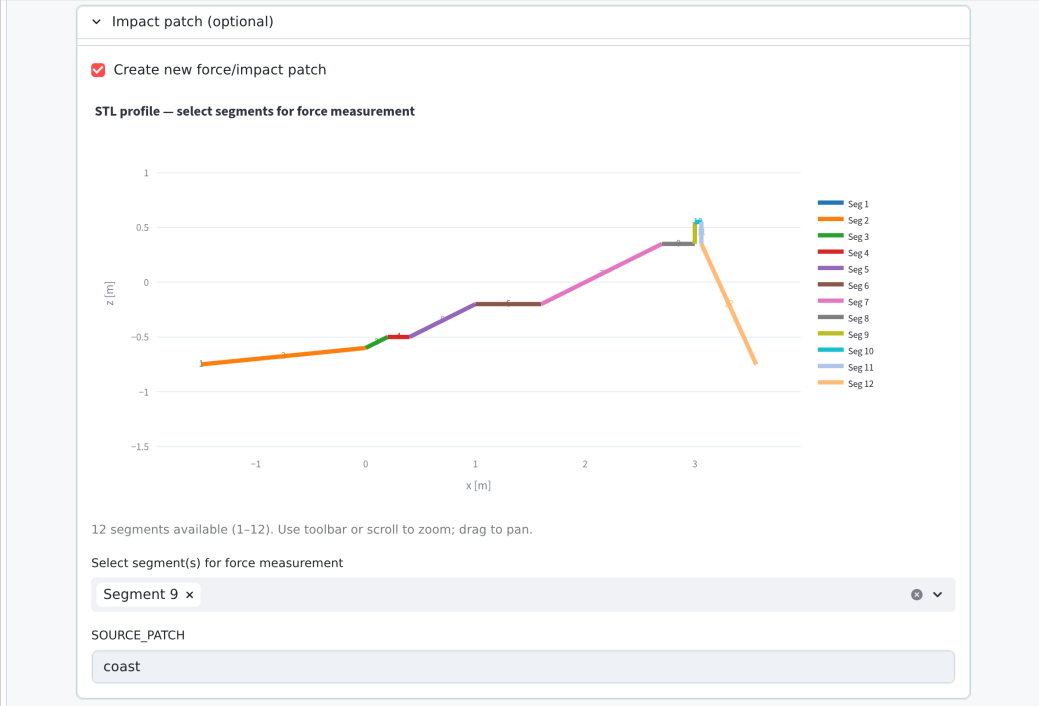


Figure 4.18: *Impact patch*: the twelve segments of the demonstration structure, and segment 9 (seaward face of the crown wall) selected for the force.

4.8 Tab *Run*

Build Case. The button is active once the STL and the domain exist. It copies the template into the folder `Demo_structure` and writes every dictionary; the build log lists each step (Figure 4.19). Here the background mesh has $306 \times 1 \times 35$ cells, the patch `force_seg_09` is prepared in `createPatchDict`, and the solver is `waveFoam` (no porous zone; `porousWaveFoam` with porous zones). The case can be inspected, or copied to a cluster, before it is launched (Chapter 9).

Launch Simulation. It starts `job_Run.sh` in a tmux session named after the case and writes `run_info.json`, the case report `run_report.html` and the set-up figures (`figures/`). The status *Simulation launched* turns green. The run continues if the browser or the application is closed. To watch it in a terminal:

```
cat Demo_structure/TMUX_SOCKET.txt     # the socket of the session
tools/tmux/bin/tmux -S /tmp/tmux_Demo_structure.sock attach
# leave the session without stopping the run: Ctrl-b, then d
```

4.9 Tab *Monitor*

Press *Refresh*: the table lists every case folder under the scan directory (the text field; by default the COAST folder) that contains `TMUX_SOCKET.txt`, i.e. every case that has been launched (Figure 4.20). Columns: status icon, case, status (`RUNNING`, `FINISHED_0`, `FINISHED_<code>`, `KILLED`, `DEAD?`), whether the tmux session is alive, run time, number of solver processes, memory, and the phase (solver, current simulated time over `endTime`). The table is not refreshed automatically: press *Refresh* again to update it.

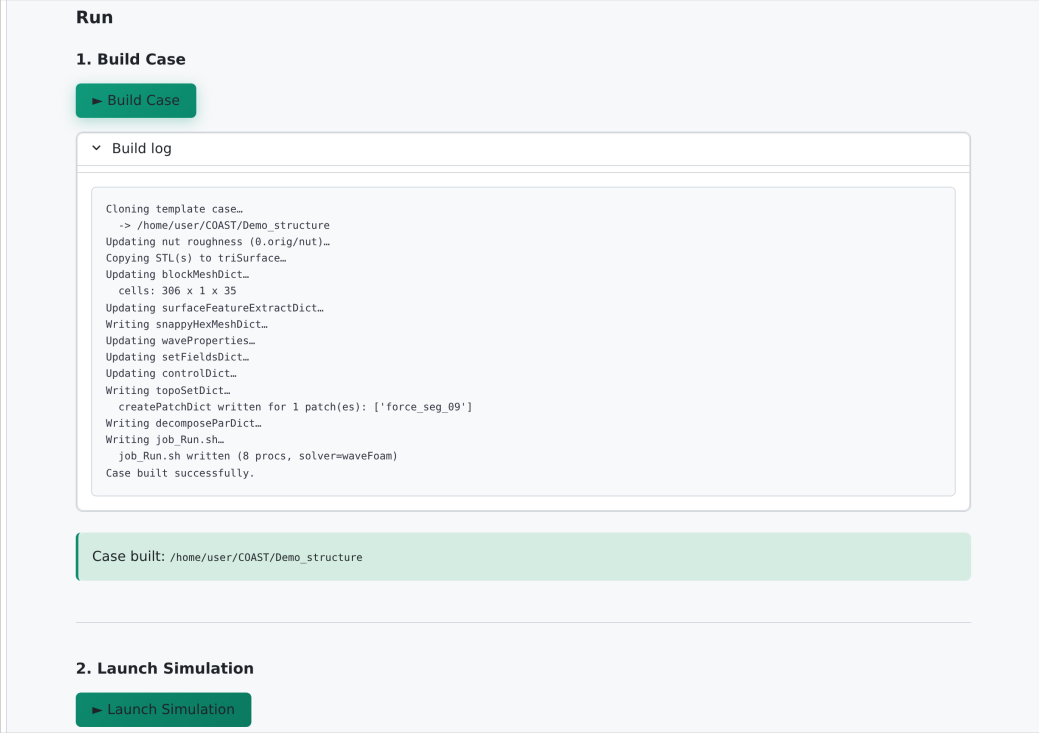


Figure 4.19: Tab *Run* after *Build Case*: the build log and the path of the case. *Launch Simulation* starts the run.

Select a case in *Select case for details / kill* to see its status, run time, processes and memory, a progress bar, and the last 30 lines of the solver log. A run that finished normally has the status `FINISHED_0`, and its log ends with `End` and `Finalising parallel run`, as in Figure 4.20. The buttons:

- *Open Report in New Tab*: the case report `run_report.html`; it is served by a small file server of the application on port 8502;
- *Open Post-processing*: the analysis page, in a new browser tab, with this case selected;
- *Kill*: stops the run (tmux session) and sets the status to `KILLED`; it is active only for a running case.

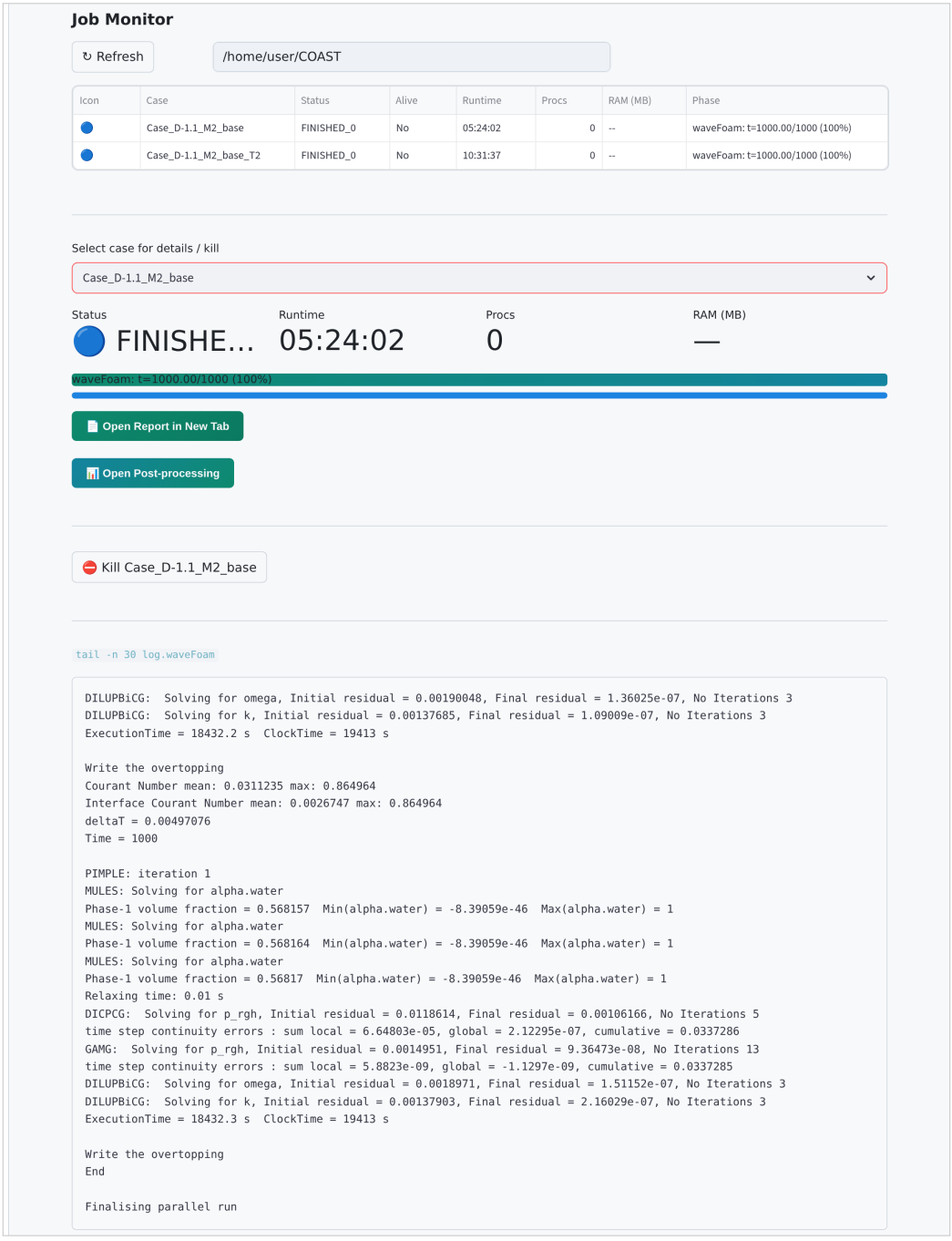


Figure 4.20: Tab *Monitor*: the table of the launched cases, and the details of the selected run.

4.10 An example of an analysed run

The post-processing page is shown with a completed run of a test of the EurOtop database (test D-1.1): a smooth, impermeable dike with a 1:1.5 seaward slope, on the flat bed of a laboratory flume, without berm or crown wall. It was set up with the mesh, time control and processes of Sections 4.6 and 4.7. Table 4.2 gives the record of the database and the inputs of the run; Table 5.2 explains the correspondence of the columns in general. Figure 4.21 shows its flume with the four gauges used below.

Table 4.2: Example run: values of the EurOtop database (test D-1.1) and inputs of the run.

Database column	**Database value**	**COAST input and value**
`Hm0 d`, `Tp d`, `Tm d`	0.1138 m, 1.743 s, 1.404 s	`Hm0_deep` = 0.114 m, `Tp_deep` = 1.74 s, `Tm_deep` = 1.4 s
`hdeep`, `h`, `ht`	0.7565 m (all three)	`h_deep` = `h` = `h_t` = 0.757 m
`mmm`, `Bt`	1000, 0	`m` = 1000 (flat flume), `B_t` = 0
`cotαd`, `cotαu`	1.5, 1.5	`cot_alpha_d` = `cot_alpha_u` = 1.5
`B`, `hb`, `tanαB`	0, 0, 0	`B` = 0, `h_b` = 0, `tan_alpha_B` = 0 (no berm)
`Ac`, `Rc`, `Gc`	0.11 m, 0.11 m, 0	`A_c` = `R_c` = 0.11 m (no crown wall), `G_c` = 0
`gf`	1.0	Smooth: no porous zone (Table 5.3)
`Tm1,0t`	1.603 s	`Tm10_toe` = 1.6 s
`q`, `Kr`	1.591×10^{-3} m^3 s^{-1} m^{-1}, 0.704	Measured results, for comparison after the run

4.11 Post-processing page: case, gauges and settings

Open the page from the sidebar (`postprocessing`) or with *Open Post-processing*. At the top (Figure 4.22):

1. **Select case.** The list contains the folders of the COAST folder that hold a case; *enter path manually* accepts any other folder.
2. **Gauge and extraction settings.** Type the x-positions of the gauges, separated by commas, and press *Extract free-surface gauges*. Each gauge is read from the stored free surface and written to `Wave_Analysis/data/` (Section 10.1); a gauge already extracted is not extracted again unless *Force re-extraction* is ticked. When gauges exist, the field lists them: here $x = -3.30$, -3.69, -4.68 and -8.80 m (Figure 4.21). The gauge at -8.80 m lies in the inlet relaxation zone; the three others are the reflection gauges of Section 4.13, between that zone and the toe.
3. **Analysis settings.** Trimming *AUTO* (or *MANUAL* with a cut time), and the segment length of the Welch spectrum. For the example run the automatic segment length with the divisor $N = 12$ was used.

The trimming and the spectrum settings apply to tabs 2 to 6 and 9.

4.12 Tabs 1 to 5: the waves at each gauge

Each of these tabs has one panel per gauge (click on $x = \ldots$ to open or close it); the page opens with the panel of the first gauge of the list. The screenshots of tabs 1, 2, 3 and 5 show the gauge

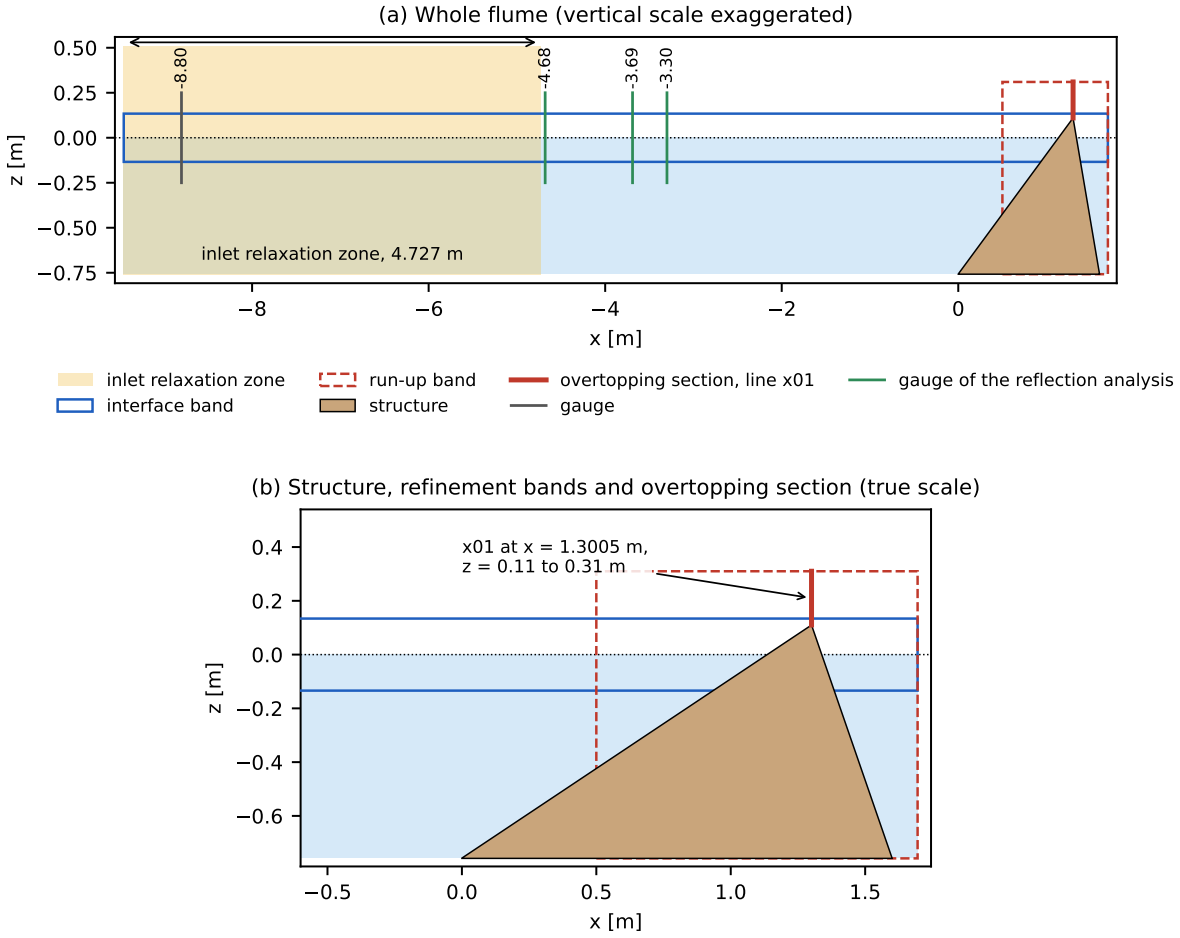


Figure 4.21: The numerical flume of the example run: domain, inlet relaxation zone ($x \in [-9.454, -4.727]$ m, $L_0 = 4.727$ m), refinement bands, structure, overtopping section (line `x01` at the crest, $x = 1.3005$ m), and the four gauges of the analysis, three of which form the reflection array.

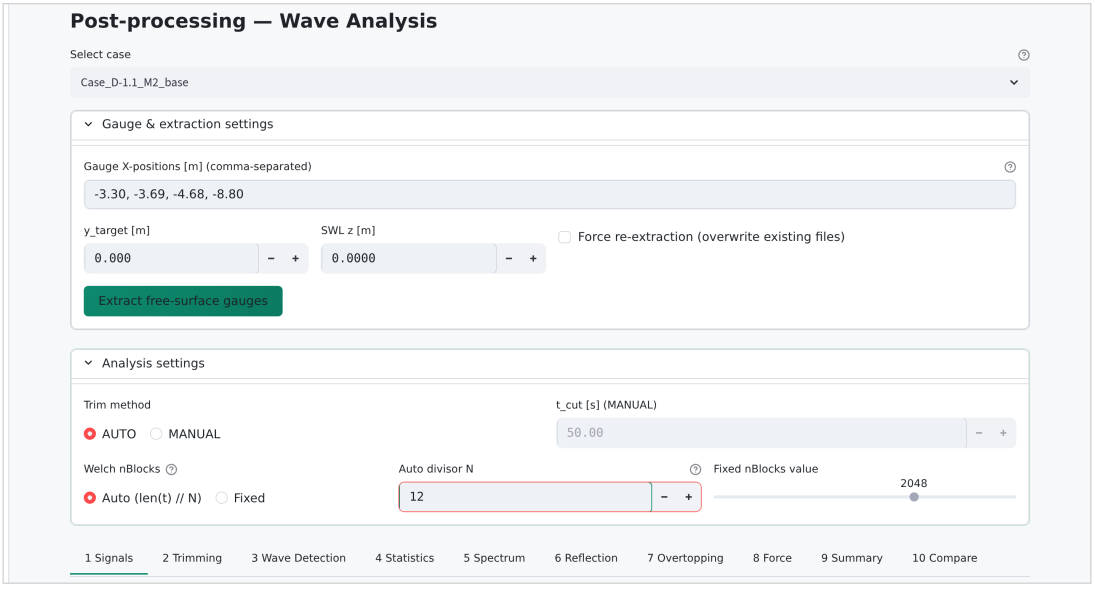


Figure 4.22: Top of the post-processing page: case, gauges, analysis settings, and the ten tabs.

$x = -8.80$ m, opened below the closed panels of the three other gauges. This gauge lies in the inlet relaxation zone, where the solution is forced towards the generated waves: its spectrum checks that the waves generated are those requested. The screenshot of tab 4 shows the first gauge, $x = -3.30$ m, the closest to the structure.

1 Signals. The target of the wave dictionary ($H_s = 0.114$ m, $T_p = 1.74$ s, $\gamma = 3.3$) and the raw elevation $\eta(t)$ (Figure 4.23). Look for the start-up (the first waves arrive after some seconds), for spikes, and for gaps.

2 Trimming. The raw signal (grey), the part kept (blue), and the cut time (Figure 4.24); the message gives the cut time and the percentage of the record removed (Section 10.2). At $x = -8.80$ m the automatic test cuts at $t = 60.1$ s and removes 6.0 % of the record: the windows that start earlier have a variance that differs from the variance of the end of the record by more than the tolerance of the test. At $x = -3.30$ m the same test cuts at $t = 0.1$ s and removes nothing. The cut differs from gauge to gauge in *AUTO* mode (tab 6 reports the latest of its three gauges, 120.1 s). Look at each trimmed signal, and use *MANUAL* with a cut time when the start of the record should be removed in any case.

3 Wave Detection. Number of waves detected by zero down-crossing in the trimmed signal (624 at $x = -8.80$ m), and a window of the signal with the individual waves, their crests and troughs (Figure 4.25); the slider *Periods to show* sets the length of the window, which starts at the cut time.

4 Statistics. $H_{1/3}$, $H_{1/10}$, $H_{\max}$, the mean period and the number of waves, and the PDF, CDF and probability of exceedance of the wave heights with the Rayleigh distribution (Figure 4.26; Section 10.4). At $x = -3.30$ m: $H_{1/3} = 0.120$ m, $H_{\max} = 0.193$ m, 613 waves.

5 Spectrum. The measured spectrum and the target JONSWAP spectrum, with H_{m0}, T_p, $T_{m-1,0}$, T_{01} and T_{02}; the small numbers under the values are the differences from the target (Figure 4.27; Section 10.5). In the relaxation zone, at $x = -8.80$ m, the measured spectrum lies

on the target: $H_{m0} = 0.113$ m (target 0.114 m) and $T_p = 1.740$ s, as imposed. A gauge closer to the structure measures the incident waves *and* the waves reflected by the dike: at $x = -3.30$ m, $H_{m0} = 0.124$ m, 0.010 m more than the target. The incident part alone is given by tab 6.

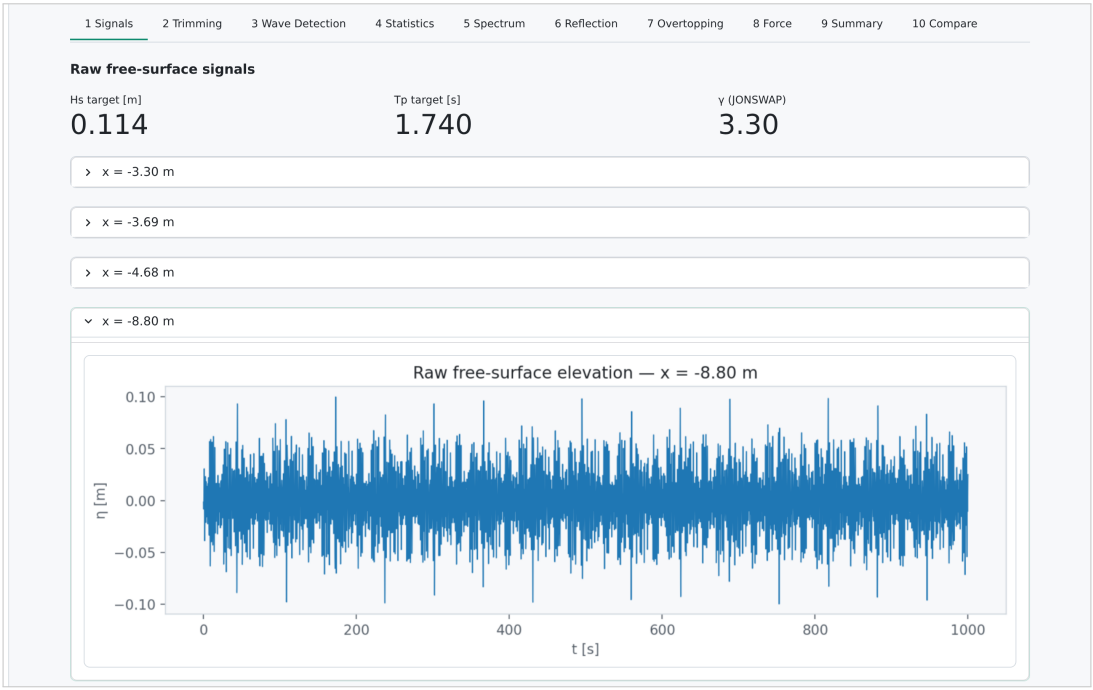


Figure 4.23: Tab *1 Signals*: target values, the list of the gauges, and the raw free-surface elevation at $x = -8.80$ m.

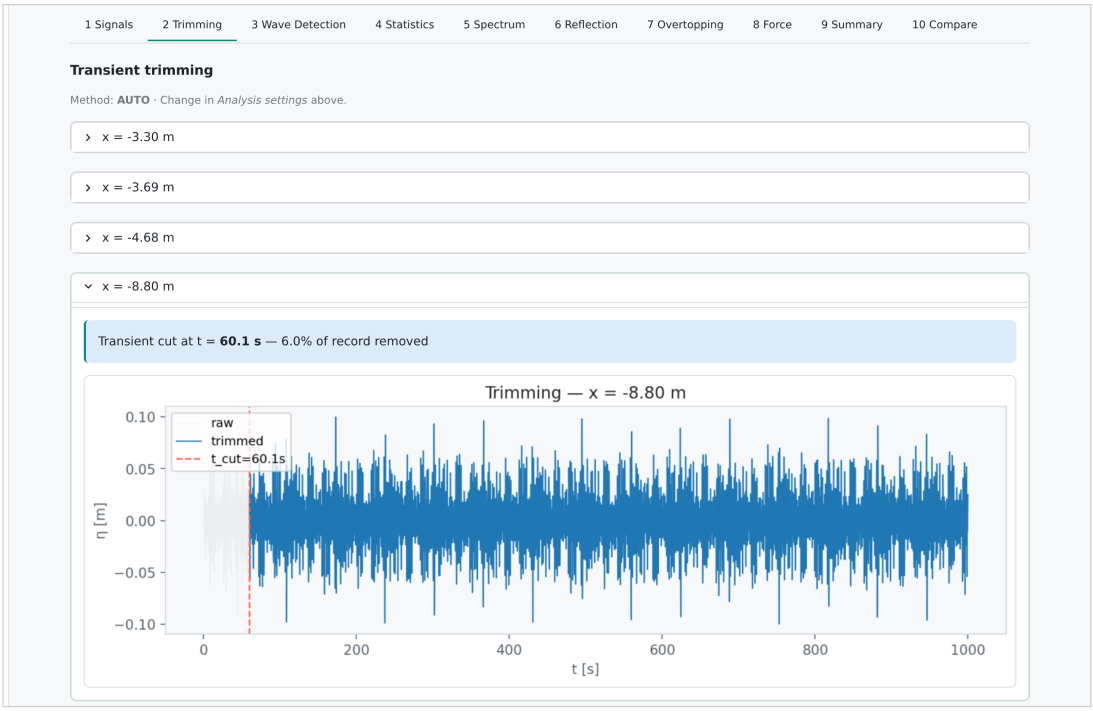


Figure 4.24: Tab *2 Trimming*: cut of the start-up transient at $x = -8.80$ m (AUTO): 60.1 s.

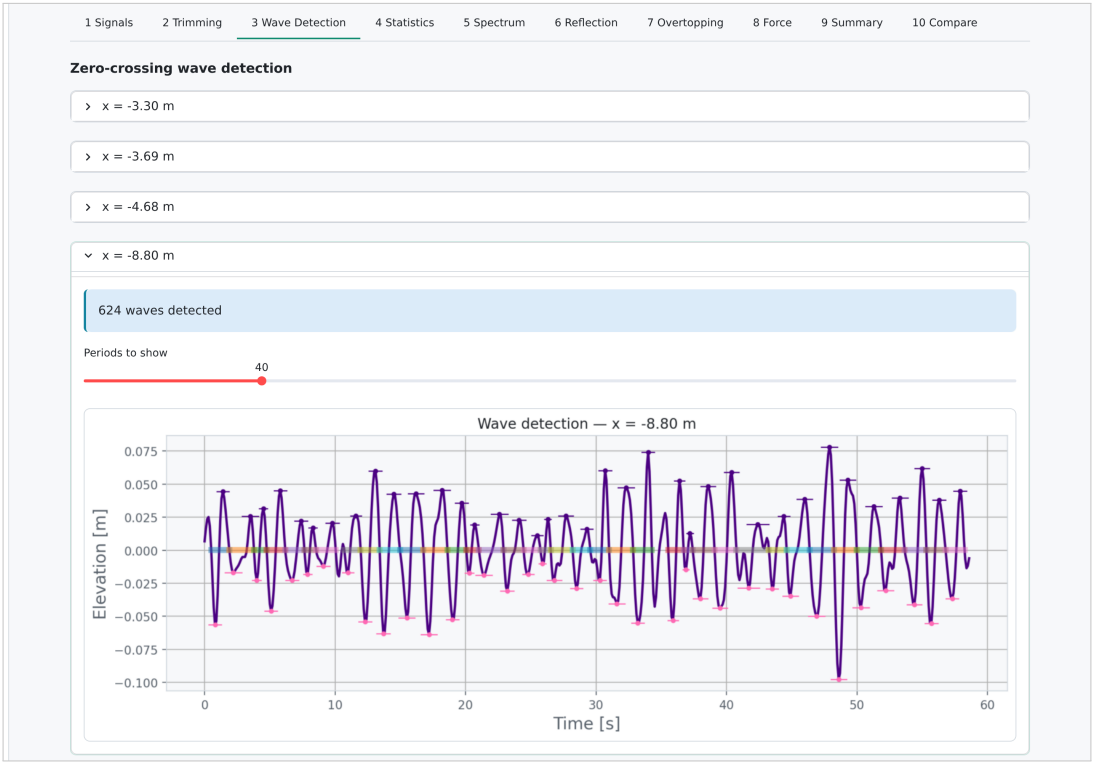


Figure 4.25: Tab *3 Wave Detection*: waves found by zero down-crossing at $x = -8.80$ m.

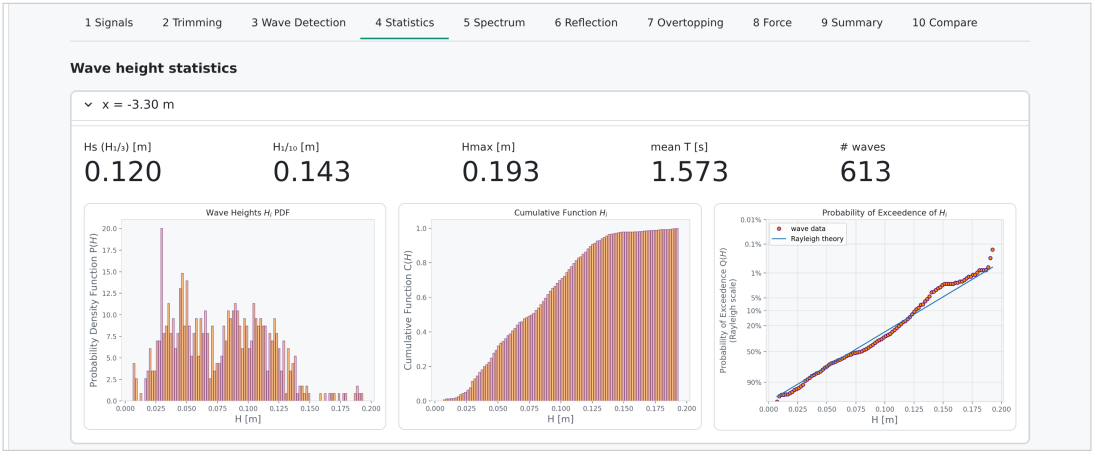


Figure 4.26: Tab *4 Statistics*: characteristic heights and distributions of the wave heights at $x = -3.30$ m.

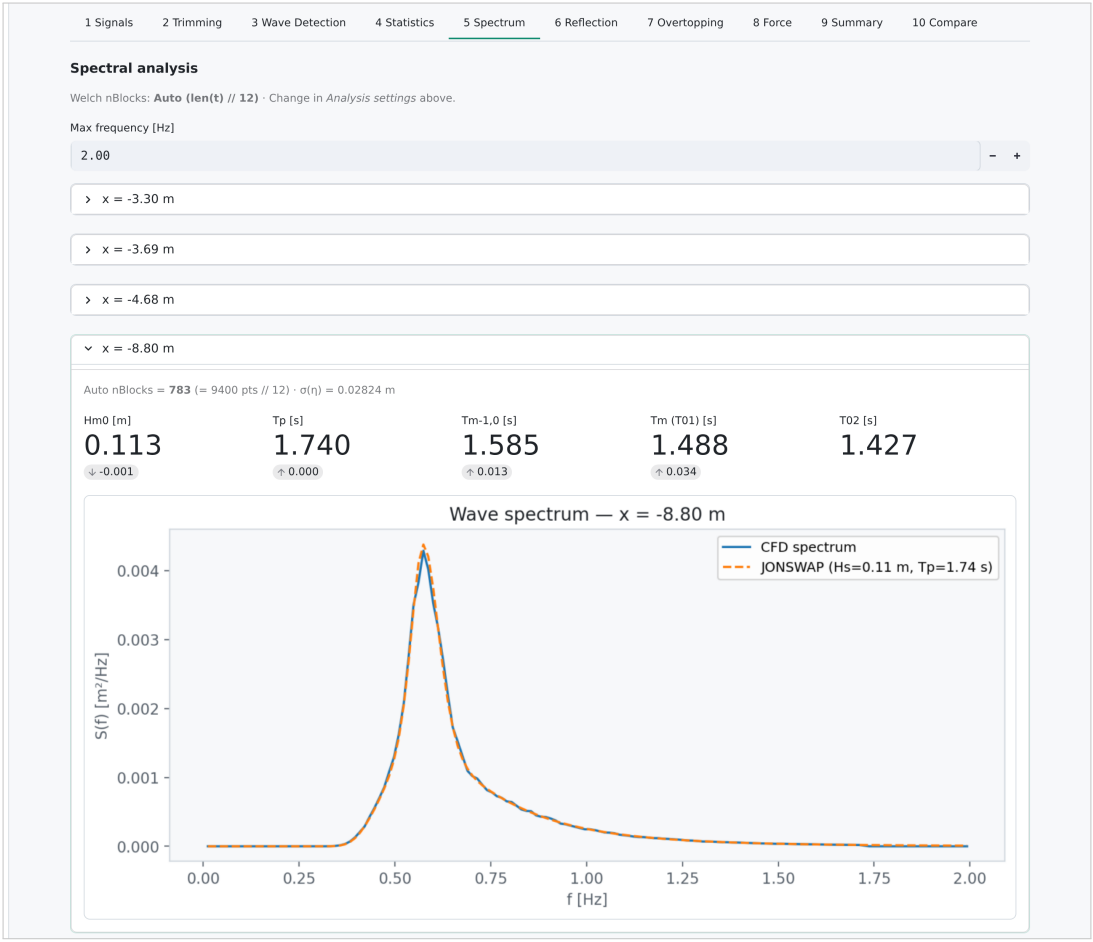


Figure 4.27: Tab *5 Spectrum*: Welch spectrum at $x = -8.80$ m, in the inlet relaxation zone (segment length: record length divided by 12), and target JONSWAP spectrum.

4.13 Tab 6: reflection

The tab proposes three gauges (Figure 4.28; Section 10.6): gauge 1 0.05 m after the end of the inlet relaxation zone ($-4.727 + 0.05 = -4.677$ m), gauge 2 a quarter of the wavelength L_p further, gauge 3 a tenth of L_p after gauge 2. The line under the inputs gives $L_p = 3.948$ m (at $T_p = 1.74$ s and $h = 0.757$ m), so $x_2 = -3.690$ m and $x_3 = -3.295$ m. Missing gauges are extracted with *Extract reflection gauges*. The analysis band is $[0.29, 1.72]$ Hz, i.e. $[0.5\,f_p, 3\,f_p]$ around $f_p = 0.575$ Hz, and the signals are detrended.

Results for the example run: $H_{m0,\mathrm{inc}} = 0.1040$ m, $H_{m0,\mathrm{ref}} = 0.0722$ m, $K_r = 0.694$ (the database gives $K_r = 0.704$ for the test). The incident wave height is 0.010 m below the target, i.e. 91 % of it. The three signals are analysed from $t = 120.1$ s, the latest of their three cut times. The figures show the incident and reflected elevation reconstructed at gauge 1, and the incident and reflected spectra with the target.

> **Read tabs 5 and 6 together.** Tab 5 at a gauge in front of the structure gives incident plus reflected waves; tab 6 separates them. The incident height $H_{m0,\mathrm{inc}}$ is the sea state that actually reaches the structure; it is the value to use in R_c^* and q^* when the result is compared with formulae or with the test (Chapter 11).

4.14 Tab 7: overtopping

The tab runs the four steps of Section 10.7–10.10 in order, each with a button, a status message on the right ("`<file>` ready" once the output exists), the table of results and the figures. Outputs that already exist are shown when the tab is opened.

Step 1: discharge and events. Set the window and the detection thresholds in *Detection parameters*, then press *Run OV_Test.py* (Figure 4.29). For the example run the window starts at the first overtopping (cumulative volume $\geq 10^{-5}$ m^3/m), and events start at $q^+ = 10^{-3}$ and end at 3×10^{-4} m^3 s^{-1} m^{-1}. The log of the script follows (Figure 4.30). For the example run:

- window $[8.9, 1000]$ s and $\overline{q} = 1.366 \times 10^{-3}$ m^3 s^{-1} m^{-1} (database: 1.591×10^{-3});
- 342 events; the events carry 98.0 % of the overtopped volume, the background 1.9 %, and the closure error is 0.06 %;
- the largest event: 1.82×10^{-2} m^3/m between 17.26 and 18.10 s, peak 4.82×10^{-2} m^3 s^{-1} m^{-1}.

Then come the table of events (ranked by volume) and the three figures (Figure 4.31).

Steps 2 and 3: energy. Press *Run kinetic.py*, then *Run merge_OV_energy.py* (Figures 4.32 and 4.33). These two scripts have no settings in the interface. For the largest event of the example run, $E = 32.2$ J/m, of which 12.3 J/m kinetic and 20.0 J/m potential energy.

Step 4: flow depth and velocity. *Parameters*: $H_{\min}$, and the number of incident waves N_w if $h_{2\%}$ and $u_{2\%}$ are wanted (0: not computed). Press *Run flow_depth_velocity.py*; reading the sampling line takes a few minutes for a long run. The summary (Figure 4.35) gives, for the example run:

- 12 of the 342 events have fewer than 3 line samples;

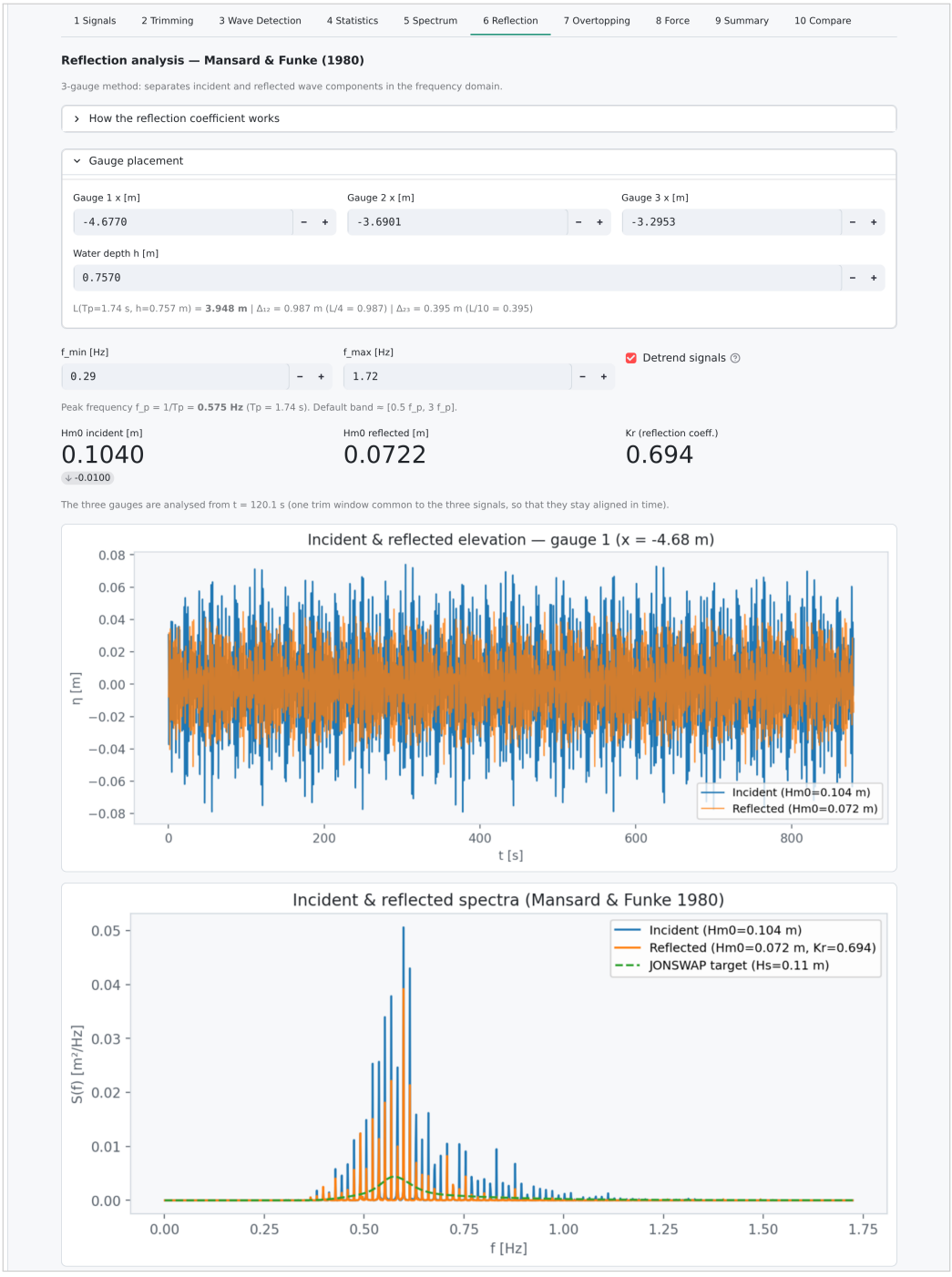


Figure 4.28: Tab *6 Reflection*: gauge array, analysis band, $H_{m0,\mathrm{inc}}$, $H_{m0,\mathrm{ref}}$ and K_r of the example run, incident and reflected time series at gauge 1, and spectra.

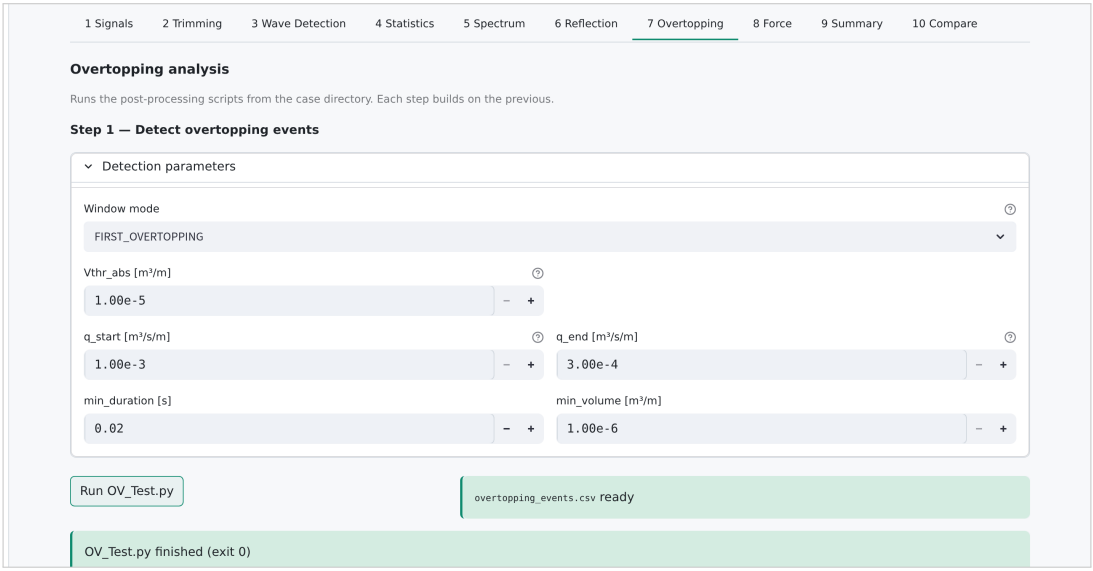


Figure 4.29: Tab *7 Overtopping*, Step 1: detection parameters, *Run OV_Test.py*, and its result.

- largest $h_{f,\max} = 4.9$ cm; 170 of the 342 events are thinner than one cell (1.25 cm);
- largest $u_{f,\max} = 1.01$ m/s; 98 events have no resolved velocity;
- volume check $\sum V_{\text{line}} / \sum V_e = 0.78$ (Section 10.10).

Figures of one event. Under Step 4, choose an event in the list (each entry gives its window, volume, $h_{f,\max}$, $u_{f,\max}$ and the number of field outputs in its window), set the options if needed, and press *Make figures C and D for event N* (Figure 4.36). Event 1 of the example run lasts 0.84 s and contains 10 field outputs, written every 0.1 s; its nine snapshots go from 17.2 to 18.0 s. The text box gives the values of the event; the figures can be downloaded as PDF or PNG (Section 10.10.5).

4.15 Tabs 8 to 10: force, summary and reports, comparison

8 Force. For a case with force segments (Section 5.6), one panel per segment with the peak $|F_x|$, its time, the maximum and minimum, and $F_x(t)$ (Section 10.11). The example run has none, and the tab says so (Figure 4.37).

9 Summary. One row per gauge with the wave heights and periods of tabs 3 to 5 and the target (Figure 4.38); *Download summary CSV* saves it. The wave-analysis report (HTML) collects the figures of the tabs already visited and the results of tab 7; visit every tab of interest first. For the PDF report, type the name to print on its cover in *User Name*, press *Generate PDF report*, then *Download PDF Report*. Both reports are also written into the case folder (Section 3.5).

10 Compare. Select two cases or more and the signal to compare: cumulative overtopping volume, horizontal force, total energy flux, or event energy against volume (Figure 4.39). The cumulative volume is read from the overtopping record of each run (Section 10.12); the force needs force segments, the energy flux Step 2, and the event energies Step 3, in every selected case. The figure compares the example run with a second run of the same test made with a

```
============================================================
Overtopping post-processing summary
============================================================

Averaging window selection:
  Mode  : FIRST_OVERTOPPING
  Rule  : AUTO (FIRST_OVERTOPPING): t1 = first significant overtopping (V_pos >= Vthr_abs).
  Vthr_abs=1.00e-05 m^3/m, t2_end_margin=0.0 s
  Window: [t1, t2] = [8.900 s, 1000.000 s]

Mean overtopping discharge:
  q_mean = 1.365868e-03 m^3 s^-1 m^-1
  (computed over the time window [8.900 s, 1000.000 s])

------------------------------------------------------------
Event definition (hysteresis-based, wave-like events)
------------------------------------------------------------
  Event start threshold : q_start = 1.00e-03 m^3 s^-1 m^-1
  Event end threshold   : q_end   = 3.00e-04 m^3 s^-1 m^-1
  Minimum event duration: 0.02 s
  Minimum event volume  : 1.00e-06 m^3 m^-1

  Total detected events (full signal)  : 342
  Events outside averaging window      : 0  (discarded)
  Events within averaging window [t1,t2]: 342

------------------------------------------------------------
Overtopped volume budget  (within averaging window [t1, t2])
------------------------------------------------------------
  Total overtopped volume within window (from q^+(t)):
    V_window = 1.353711e+00 m^3 m^-1

  Volume carried by detected wave-events (within window):
    V_events = 1.327137e+00 m^3 m^-1  (98.0 % of window total)

  Background / tail volume (within window, outside detected events):
    V_background = 2.581598e-02 m^3 m^-1  (1.9 % of window total)

  Volume closure check (within window):
    V_events + V_background = 1.352953e+00 m^3 m^-1
    Closure error           = 7.59e-04 m^3 m^-1  (0.06 %)

------------------------------------------------------------
Event statistics
------------------------------------------------------------
  Event volume statistics [m^3 m^-1]:
    min  = 1.244e-05
    mean = 3.881e-03
    max  = 1.821e-02

  Event peak discharge statistics [m^3 s^-1 m^-1]:
    min  = 1.006e-03
    mean = 1.297e-02
    max  = 4.821e-02

------------------------------------------------------------
Largest overtopping events (by volume)
------------------------------------------------------------
  Event #1:
    Volume         = 1.820898e-02 m^3 m^-1
    Time interval  = [17.256 s , 18.100 s]
    Duration       = 0.844 s
    Peak discharge = 4.820920e-02 m^3 s^-1 m^-1
  Event #2:
    Volume         = 1.649990e-02 m^3 m^-1
    Time interval  = [726.671 s , 727.466 s]
    Duration       = 0.795 s
    Peak discharge = 4.478540e-02 m^3 s^-1 m^-1
  Event #3:
    Volume         = 1.640767e-02 m^3 m^-1
    Time interval  = [404.522 s , 405.339 s]
    Duration       = 0.816 s
    Peak discharge = 4.384340e-02 m^3 s^-1 m^-1

------------------------------------------------------------
Consistency check (event volume vs cumulative curve)
------------------------------------------------------------
  For the three largest events:
    Event #1: V_event = 1.820898e-02 ; ΔV_curve = 1.820898e-02  (m^3 m^-1)
    Event #2: V_event = 1.649990e-02 ; ΔV_curve = 1.649990e-02  (m^3 m^-1)
    Event #3: V_event = 1.640767e-02 ; ΔV_curve = 1.640767e-02  (m^3 m^-1)
  → Agreement indicates consistent event detection and integration.

============================================================
Saved event table: /home/user/COAST/Case_D-1.1_M2_base/overtopping_events.csv
============================================================
/home/user/COAST/Case_D-1.1_M2_base/OV_Test.py:519: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be shown
  plt.show()
/home/user/COAST/Case_D-1.1_M2_base/OV_Test.py:538: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be shown
  plt.show()
/home/user/COAST/Case_D-1.1_M2_base/OV_Test.py:556: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be shown
  plt.show()
```

Figure 4.30: Step 1: summary printed by `OV_Test.py` for the example run (also saved in `overtopping_summary.txt`).

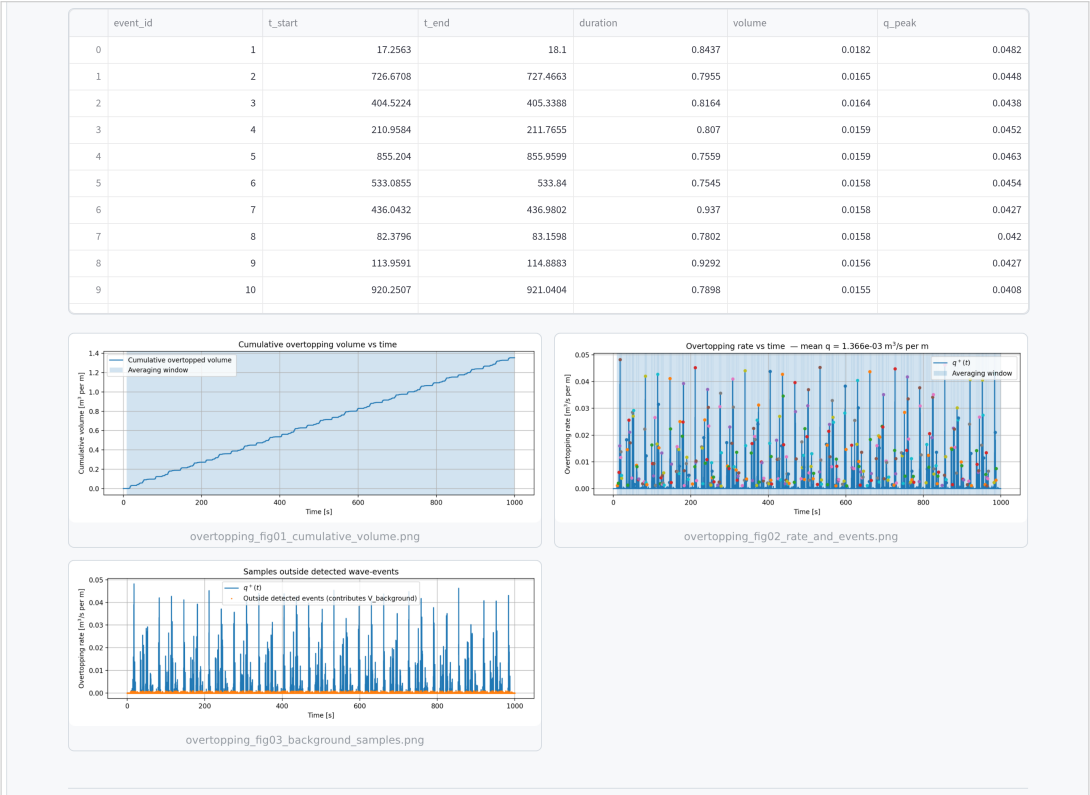


Figure 4.31: Step 1: table of events, cumulative volume with the averaging window, rate $q^{+}(t)$ with the events, and samples outside the events.

smaller time step: after 1000 s, the second run has overtopped about 8 % more water. *Download figure (PNG)* saves the figure.

4.16 The case folder at the end

After the run and the analysis, the case folder contains the set-up (`0.orig`, `constant`, `system`, `job_Run.sh`), the records of the run (`run_info.json`, `run_report.html`, `figures/`, `TMUX_*`, `log.*`), the raw results (`processor*`, `postProcessing/`), and the outputs of the analysis: `Wave_Analysis/` (gauges, figures, `reflection_results.json`, `wave_report.html`), the files of Steps 1 to 4 and their copies of `OV_Test.py` and `flow_depth_velocity.py` with the settings used, `event_figures/`, and `<case>_report.pdf`. Every file is described in Appendix B. The time folders in `processor*` hold the fields; they are needed by the event figures and by any later analysis of the fields, and they are most of the disk space of the case.



Figure 4.32: Step 2: energy of each event, flux components, cumulative energy, and histogram of the event energies.



	event_id	t_start	t_end	duration	V_event	q_peak	E_KE	E_PE	E_total	E_per_volume	F_total_mean	F_total_max	median_energy_threshold	is_strong_event	strength
0	1	17.2563	18.1	0.8437	0.0182	0.0482	12.2706	19.9695	32.2401	1770.559	38.2146	280.9375	3.0573	☑	strong
1	2	726.6708	727.4663	0.7955	0.0165	0.0448	11.9828	17.9649	29.9477	1815.0213	37.6479	304.2918	3.0573	☑	strong
2	3	404.5224	405.3388	0.8164	0.0164	0.0438	8.2926	17.4607	25.7533	1569.5895	31.544	84.4378	3.0573	☑	strong
3	4	210.9584	211.7655	0.807	0.0159	0.0452	9.0347	17.19	26.2246	1644.2147	32.4955	155.4765	3.0573	☑	strong
4	5	855.204	855.9599	0.7559	0.0159	0.0463	7.6422	17.1394	24.7816	1555.6166	32.7849	76.2285	3.0573	☑	strong
5	6	533.0855	533.84	0.7545	0.0158	0.0454	7.8116	17.0126	24.8242	1569.0822	32.9021	77.5263	3.0573	☑	strong
6	7	436.0432	436.9802	0.937	0.0158	0.0427	5.033	16.5216	21.5546	1365.8513	23.0043	60.991	3.0573	☑	strong
7	8	82.3796	83.1598	0.7802	0.0158	0.042	7.4899	16.5594	24.0493	1524.2457	30.8246	70.0948	3.0573	☑	strong
8	9	113.9591	114.8883	0.9292	0.0156	0.0427	4.9894	16.3305	21.3199	1366.2343	22.9434	61.233	3.0573	☑	strong
9	10	920.2507	921.0404	0.7898	0.0155	0.0408	9.8882	16.4876	26.3757	1701.206	33.3976	238.9464	3.0573	☑	strong

Figure 4.33: Step 3: merged table of volume and energy of each event. Its nine figures follow it in the tab (Figure 4.34).

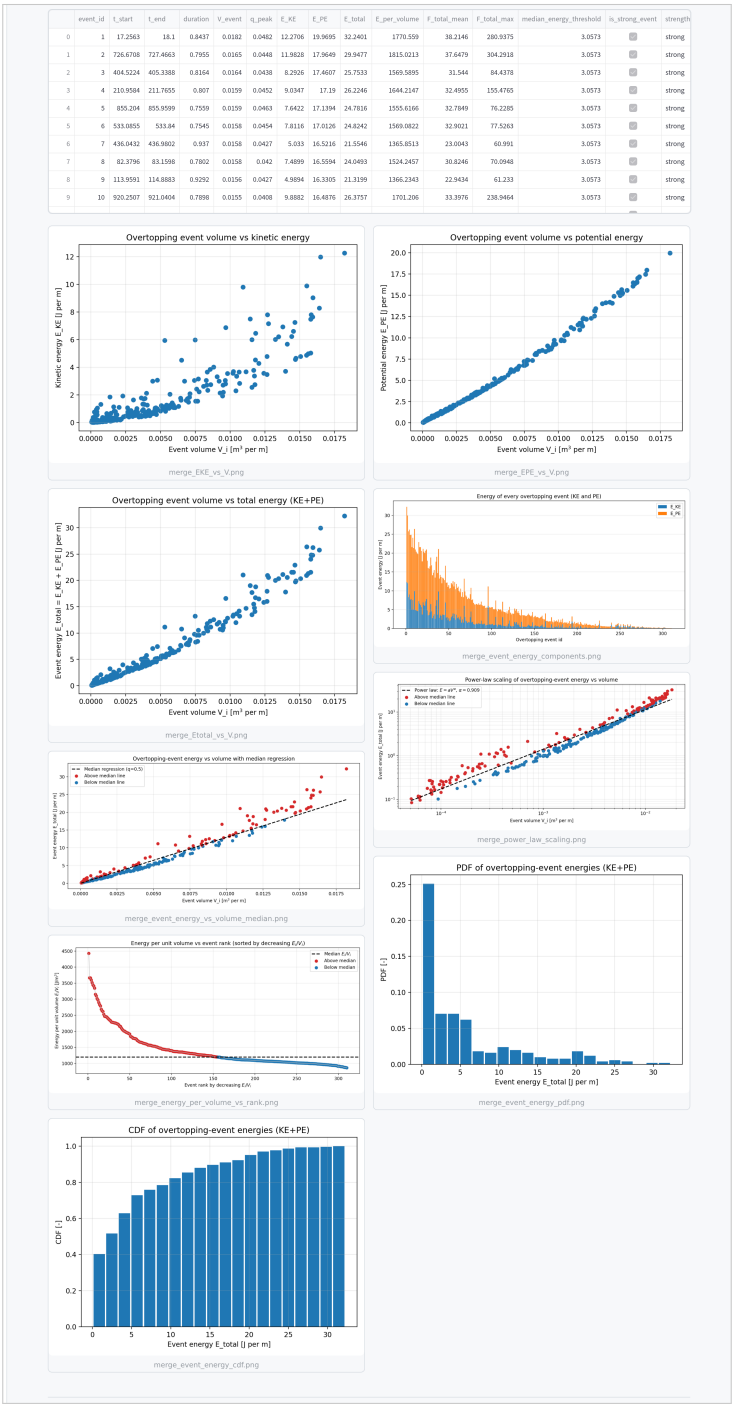


Figure 4.34: Step 3: the figures of the energy–volume analysis, as shown in the tab (Section 10.9).

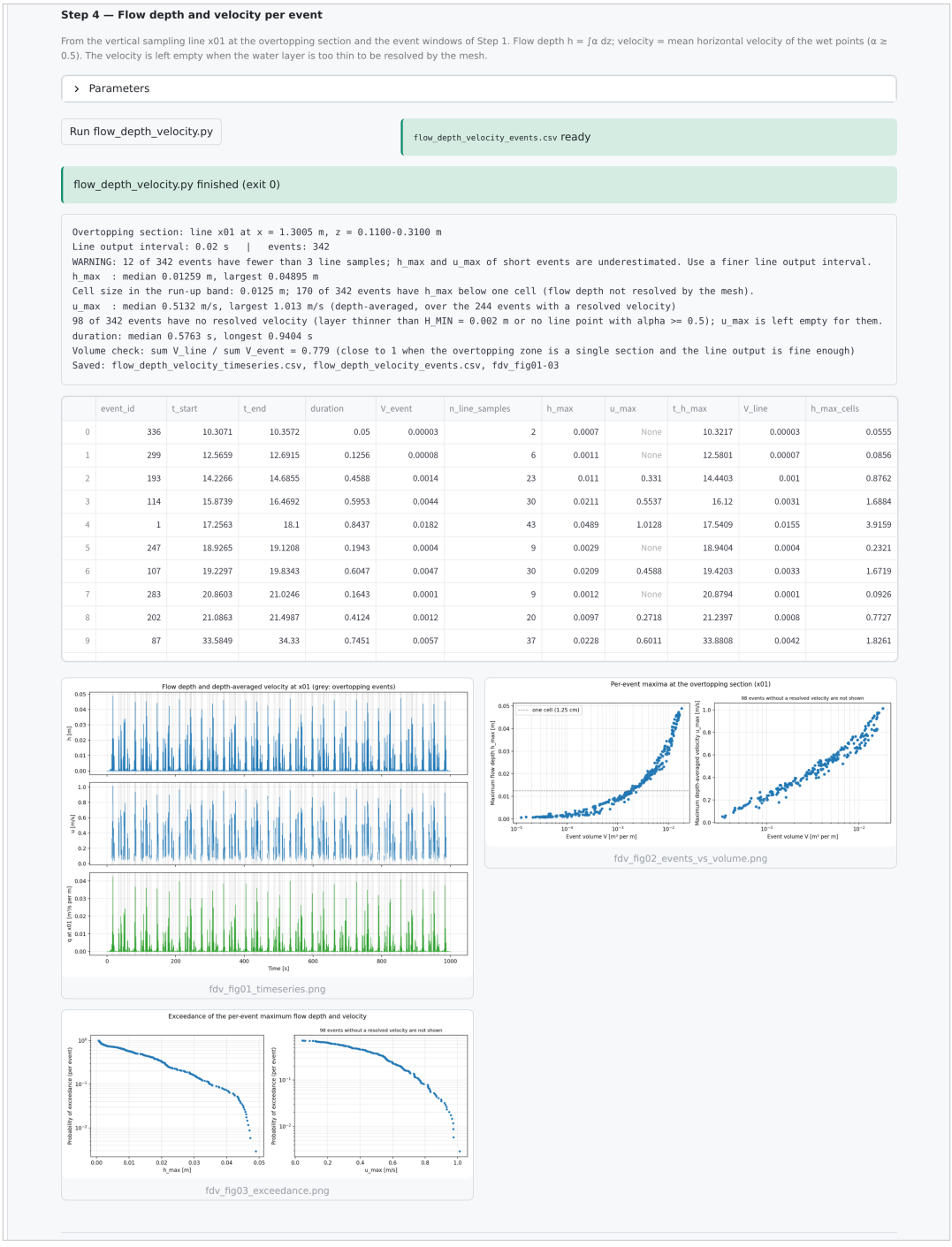


Figure 4.35: Step 4: summary, table of events (ordered in time), time series of h_f, u_f and q_l, maxima per event against the volume, and their exceedance.

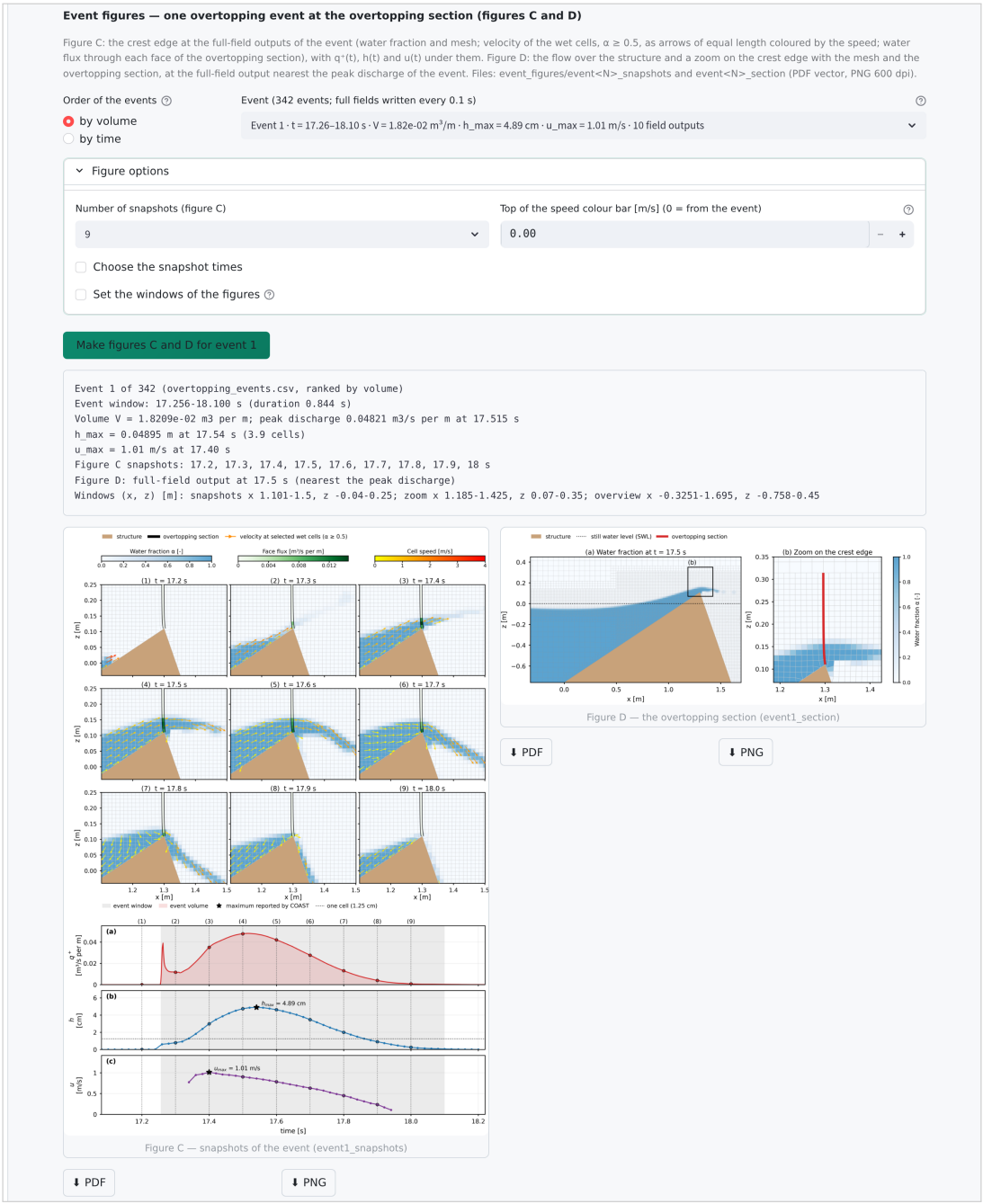


Figure 4.36: Figures of event 1 of the example run: choice of the event and options, values of the event, snapshots (figure C) and overtopping section (figure D).

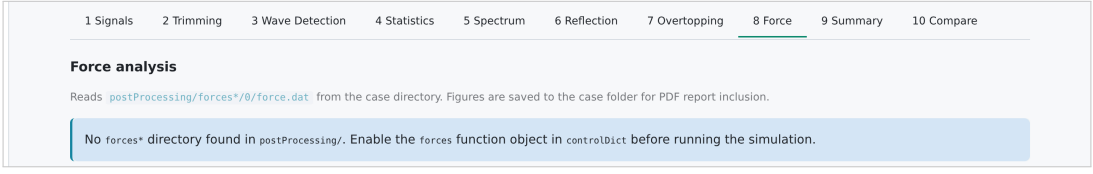


Figure 4.37: Tab *8 Force* for a case without force segments.

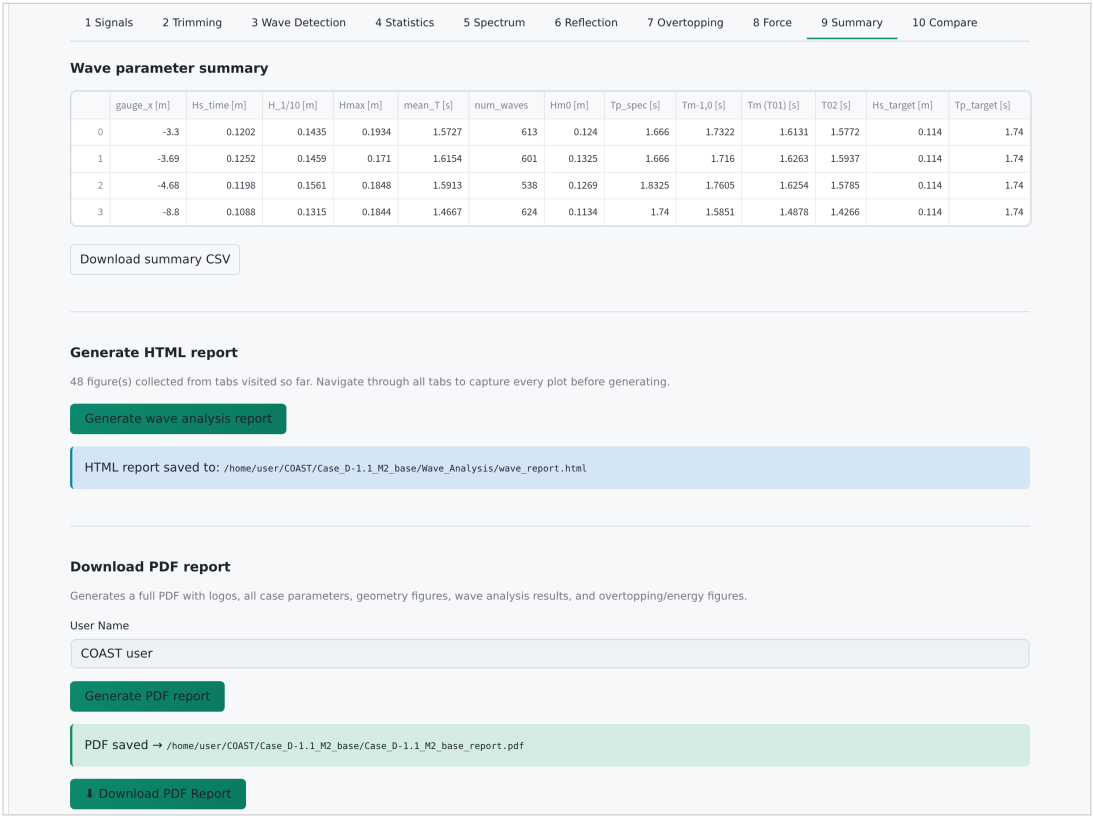


Figure 4.38: Tab *9 Summary*: wave parameters of the four gauges of the example run, and the HTML and PDF reports.

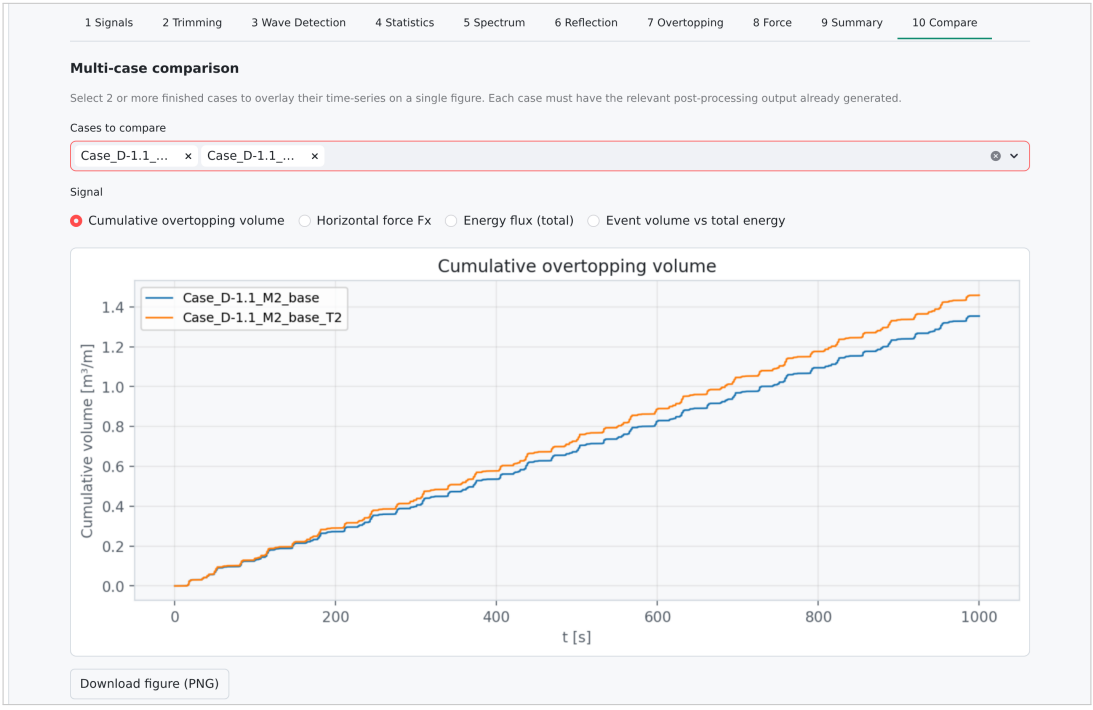


Figure 4.39: Tab *10 Compare*: cumulative overtopping volume of two runs of the same test.

Chapter 5

Structure Geometry

5.1 EurOtop parameterisation

The structure cross-section follows the standard EurOtop parameterisation (EurOtop, 2018): foreshore slope, toe, seaward slopes, berm, crest, and crown wall, with all levels referenced to the SWL. Because the parameterisation coincides with that of the EurOtop database, a database record maps directly onto a COAST geometry, which is how the demonstration cases of Chapter 11 were set up. The parameters are listed in Table 5.1 and shown on a generated section in Figure 5.1; Appendix A gives the name of each input in the interface.

Table 5.1: Structure geometry parameters (EurOtop convention, plus the placement parameters of COAST).

Symbol	Parameter	Unit	Description
SWL	Still-water level	m	Reference level for all elevations
m	Foreshore slope	–	Cotangent of the foreshore slope. The value $m = 1000$ selects the *flume* configuration: flat bed with a short ramp between h_{deep} and h
h_{deep}	Deep-water depth	m	Water depth at the offshore boundary, where waves are generated
h	Toe depth	m	Water depth at the structure toe
h_t	Toe-berm depth	m	Water depth above the toe berm ($h_t = h$ if there is no toe berm)
B_t	Toe-berm width	m	Horizontal width of the toe berm (0 if none)
h_b	Berm depth	m	Water depth above the main berm, taken at mid-berm; negative if the berm lies above the SWL
B	Berm width	m	Horizontal width of the main berm (0 if none)
$\tan\alpha_B$	Berm slope	–	Slope of the main berm, positive when the berm rises landward (0 for a horizontal berm); `tanαB` in the EurOtop database
$\cot\alpha_d$	Lower slope	–	Cotangent of the seaward slope between the toe berm and the berm
$\cot\alpha_u$	Upper slope	–	Cotangent of the seaward slope between the berm and the crest
A_c	Armour crest level	m	Crest level above the SWL
G_c	Crest width	m	Horizontal width of the crest
R_c	Crest freeboard	m	Top of the crown wall above the SWL; when $R_c > A_c$ a crown wall is added at the landward end of the crest
t_{wall}	Wall thickness	m	Horizontal thickness of the crown wall
L_{back}	Landward length	m	Horizontal length of the landward part: back slope, or length of the collector
x_{toe}	Toe position	m	x-coordinate of the structure toe
x_{margin}	Offshore margin	m	Additional flat bed seaward of the foreshore
Δz_{min}	Base thickness	m	Thickness of solid below the deepest bed level
L_y	Domain width	m	Spanwise extent of the STL extrusion

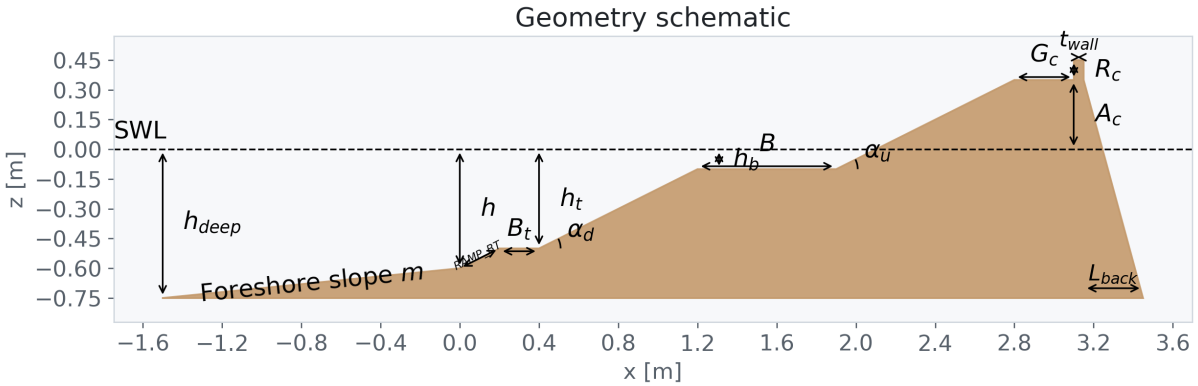


Figure 5.1: Structure geometry generated by the COAST parametric engine: cross-section with the EurOtop parameters of Table 5.1, referenced to the SWL.

5.1.1 From a record of the EurOtop database to the inputs of COAST

The EurOtop database (`Database_Eurotop_v5.xlsx` in the repository, sheet `Database`) gives one test per row. Table 5.2 lists the columns used to set up a case, and the input of the interface that receives each of them. The units of the database are those of the laboratory test (model scale), and COAST uses them as they are.

Table 5.2: Columns of the EurOtop database and the corresponding inputs of COAST.

Database column	**COAST input (tab)**	**Remark**
`Hm0 d`, `Tp d`, `Tm d`	`Hm0_deep`, `Tp_deep`, `Tm_deep` (Wave and Domain)	Sea state at the wave generation boundary (Section 6.1)
`Tm1,0 d`	–	Not an input; it can be compared with $T_{m-1,0}$ of the *Spectrum* tab at a gauge offshore
`hdeep`	`h_deep` (Geometry)	Depth at the wave generation boundary
`mmm`	`m` (Geometry)	Foreshore slope; 1000 is the flat flume of the database, also in COAST
`h`, `ht`, `Bt`	`h`, `h_t`, `B_t` (Geometry)	Toe depth, depth on the toe berm, toe-berm width
`Hm0 toe`, `Tp toe`, `Tm toe`, `Tm1,0t`	`Hm0_toe`, `Tp_toe`, `Tm_toe`, `Tm10_toe` (Wave and Domain)	Expected toe conditions; only `Hm0_toe` acts on the case (interface band)
`cotαd`, `cotαu`	`cot_alpha_d`, `cot_alpha_u` (Geometry)	Lower and upper seaward slopes; 0 gives a vertical face
`cotαexcl`, `cotαincl`	–	Mean slopes derived by the database from the other columns, not inputs
`B`, `hb`, `tanαB`	`B`, `h_b`, `tan_alpha_B` (Geometry)	Berm width, depth on the berm (COAST takes it at mid-berm), berm slope. `Bh` is not used
`Ac`, `Rc`, `Gc`	`A_c`, `R_c`, `G_c` (Geometry)	Armour crest, crest freeboard (crown wall when $R_c > A_c$), crest width
`gf_d`, `gf_u`, `gf`	Porous layers or wall roughness (Geometry, Simulation and Job)	Roughness factor γ_f of the lower slope, upper slope, and mean; it tells the type of armour and the number of layers (Section 5.5.1)
`D50_d`, `D50_u`, `D`	`d50` of a porous layer, thickness d	Armour size [m] of the lower slope, upper slope, and mean (Section 5.5.1)

continued on next page

Table 5.2 continued.

Database column	COAST input (tab)	Remark
β, `Spread s`	–	Obliquity and directional spreading: a pseudo-2D case has normal, long-crested waves (Chapter 12)
`RF`, `CF`	–	Reliability and complexity factors of the test, useful to select records
`q`, `Kr`, `Kt`	–	Measured mean discharge, reflection and transmission coefficients: to be compared with the results of COAST

The values of the toe (h, $H_{m0,\mathrm{toe}}$, ...) are those of the database; the free-surface elevation that COAST computes at the toe is a result, to be compared with them after the run (Section 10.5).

5.2 Profile construction

The closed two-dimensional cross-section polygon is built from left (offshore) to right (landward): vertical levels first, then the horizontal extent of each segment.

5.2.1 Vertical levels

$$z_{\mathrm{deep}} = z_{\mathrm{SWL}} - h_{\mathrm{deep}} \tag{5.1}$$
$$z_{\mathrm{toe,water}} = z_{\mathrm{SWL}} - h \tag{5.2}$$
$$z_{\mathrm{toe,berm}} = z_{\mathrm{SWL}} - h_t \tag{5.3}$$
$$z_{\mathrm{berm}} = z_{\mathrm{SWL}} - h_b \tag{5.4}$$
$$z_{\mathrm{berm},s/e} = z_{\mathrm{berm}} \mp \tfrac{1}{2} B \tan\alpha_B \tag{5.5}$$
$$z_{\mathrm{crest}} = z_{\mathrm{SWL}} + A_c \tag{5.6}$$
$$z_{\mathrm{wall,top}} = z_{\mathrm{SWL}} + R_c \tag{5.7}$$
$$z_{\mathrm{min}} = z_{\mathrm{deep}} - \Delta z_{\mathrm{min}} \tag{5.8}$$

z_{berm} is the berm level at mid-berm; $z_{\mathrm{berm},s}$ and $z_{\mathrm{berm},e}$ are the levels of its seaward and landward edges, which coincide with z_{berm} for a horizontal berm ($\tan\alpha_B = 0$). The berm must stay between the toe-berm and crest levels. z_{min} closes the polygon at the bottom and must lie below every other level.

5.2.2 Horizontal distances

Each sloping segment's horizontal extent follows from its elevation difference and slope cotangent,

$$\Delta x = \Delta z \, \cot\theta. \tag{5.9}$$

From the toe at x_{toe}, the profile is assembled as follows.

- **Foreshore.** Seaward of the toe the bed rises from z_{deep} to $z_{\mathrm{toe,water}}$ over $\Delta x_{\mathrm{fs}} = (z_{\mathrm{toe,water}} - z_{\mathrm{deep}})\, m$. In the flume configuration ($m = 1000$) this length is replaced by the short ramp `dx_foreshore_ramp`. A flat bed of length x_{margin} is added seaward of the foreshore.
- **Toe.** If $h_t \neq h$, a short connector ramp joins the toe level to the toe-berm level (`dx_toe_transition` in the flume configuration, `dx_berm_connector` otherwise); the toe berm then extends over B_t.

- **Seaward slopes, berm, crest.** The lower slope extends $\Delta x_d = (z_{\mathrm{berm},s} - z_{\mathrm{toe,berm}}) \cot \alpha_d$, the berm B (rising from $z_{\mathrm{berm},s}$ to $z_{\mathrm{berm},e}$), the upper slope $\Delta x_u = (z_{\mathrm{crest}} - z_{\mathrm{berm},e}) \cot \alpha_u$, and the crest G_c.
- **Crown wall.** If $R_c > A_c$, a wall of thickness t_{wall} and top at $z_{\mathrm{wall,top}}$ is placed at the landward end of the crest.
- **Landward end.** Without a collector, the profile descends from the crest (or from the landward face of the wall) to z_{min} over a horizontal distance L_{back}, and the overtopped water flows into the landward part of the flume. With the collector option, a box of length L_{back} is placed behind the crest instead (Section 5.3).

The three connector lengths exist to keep the polygon well conditioned for meshing and have no physical meaning; they rarely need changing.

5.2.3 STL extrusion

The two-dimensional polygon is triangulated by ear clipping, then extruded to 3D by replicating the front face at $y = 0$ and the back face at $y = L_y$ and connecting corresponding edges with quadrilateral faces, each split into two triangles. The result is written as an STL file with `numpy-stl` in `STL_Geos/`. The surface is watertight by construction, which `snappyHexMesh` requires.

5.3 Structure type: breakwater mode and collector

Breakwater mode. For low-crested structures and breakwaters with water on both sides, *Breakwater mode* fills the flume with water on both sides of the structure at initialisation and adds an outlet relaxation zone that absorbs the transmitted waves (Section 6.5). The collector is disabled in this mode.

Collector. The *collector* option (`USE_COLLECTOR`) adds a box behind the crest, of length L_{back}, that retains the overtopped water. It reproduces the overtopping tank of laboratory tests and allows an independent estimate of the discharge from the water volume accumulated in the box (script `LOOP_OVERTOPPING.py`, Chapter 14). The standard discharge of Section 10.7 does not need it.

5.4 Obstacles and imported geometry

Non-standard configurations (storm walls, recurves, buildings on the crest, scour protection) are handled within the same automated workflow by merging obstacles with the parametric profile. The obstacle editor (*Add a new obstacle*) shows the profile in an interactive view in which coordinates are read by hovering, and offers three input modes:

- **Rectangle**: left and right x, bottom and top z, in metres;
- **Polygon vertices**: an explicit list of (x, z) coordinates, one vertex per line;
- **Import STL**: the outline of an existing STL on a chosen y-plane is extracted and used as an obstacle polygon.

Each obstacle can afterwards be resized (width, height) and moved (centre) or removed. Obstacles are solid. They are combined with the profile by a **2D boolean union** (Shapely) before

extrusion, so the merged body is a single watertight solid rather than overlapping surfaces. This matters for meshing: overlapping STL sheets produce unpredictable `snappyHexMesh` behaviour, whereas a unioned polygon does not. If Shapely is not available, the obstacle surfaces are only concatenated and the application says so.

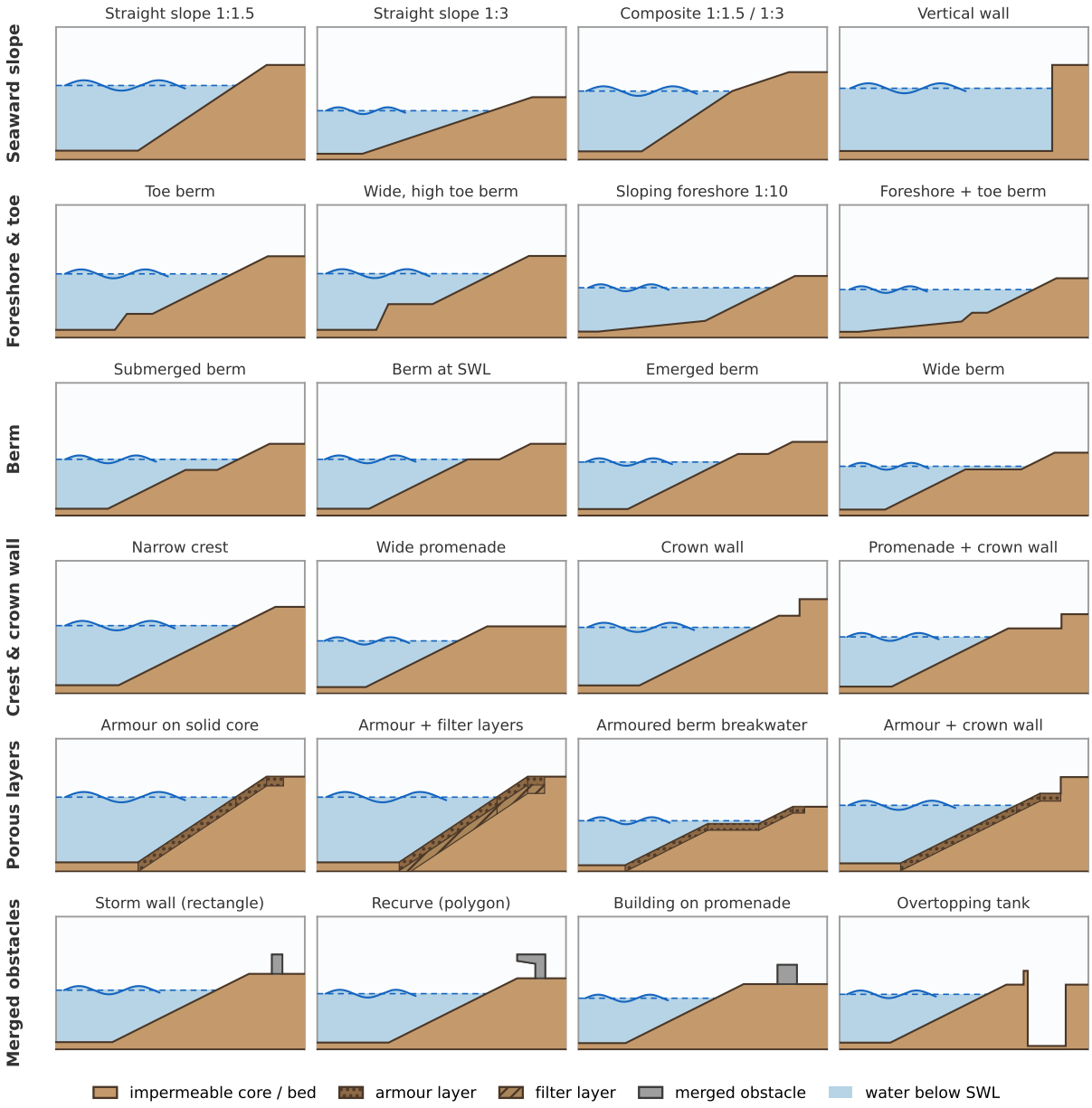


Figure 5.2: Structure configurations reachable from the COAST parametric engine, grouped by the geometric feature each exercises. All panels share one base section with a single feature enabled, and are produced by the same code that writes the STL surfaces: the EurOtop profile builder, the layered porous-zone builder, and the 2D boolean union for merged obstacles. All sections are drawn at true scale.

5.5 Porous layers

For permeable structures, porous layers are attached to selected surface segments of the profile. The available zones are, from offshore to landward,

`foreshore` | `toe_slope` | `toe_berm` | `lower_slope` | `berm` | `upper_slope` | `crest` | `back_slope`

A zone that does not exist in the current geometry (for example a berm of zero width) is skipped. The `back_slope` zone exists only when the structure has neither a crown wall nor a collector.

Layers. Up to five **concentric layers** can be defined, from the outside inward (typically armour, filter, and further sub-layers). Each layer has a thickness, with optional per-zone overrides (for example a thicker armour on the upper slope than on the crest), and its own resistance parameters ($n, d_{50}, \mathrm{KC}, \alpha_{\mathrm{vG}}, \beta_{\mathrm{vG}}, \gamma_{\mathrm{AM}}$) of Section 8.6, with optional per-zone overrides of n and d_{50}. The **core** below the layers is either solid (meshed out) or porous with its own porosity, d_{50}, α_{vG} and β_{vG}.

Construction. The layer bands are obtained by translating each surface segment **vertically downward** by the cumulative layer thickness, and are clipped at $z_{\min}$. The thickness d entered in the interface is therefore a *vertical* thickness. On a slope of angle θ, a layer of thickness t measured normal to the surface corresponds to

$$d = \frac{t}{\cos\theta} = t\,\frac{\sqrt{1+\cot^2\theta}}{\cot\theta}, \tag{5.10}$$

which should be used when the design gives the normal thickness of the armour (for example $t = n_{\mathrm{layers}}\, k_t\, D_{n50}$ in the Rock Manual, CIRIA et al., 2007).

Crest wedge. The `crest` band ends vertically at the landward end of the crest. The material landward of that line, between the crest end and the back slope, belongs to the `back_slope` zone. With the option *back_slope: crest wedge only*, the `back_slope` bands fill only this wedge: their inner boundary is horizontal, at $z_{\mathrm{crest}} - d$, from the crest end to the back slope, and the back slope below that level stays solid. With the same thickness on the crest and on the back slope, the porous crest then extends without a step to the back slope. Without the option, the `back_slope` bands follow the whole back slope down to $z_{\min}$.

OpenFOAM set-up. Each layer–zone band is written as a separate STL, registered as a cell zone `porosity_<layer>_<zone>` by `topoSet`, and given its own entry in `constant/porosityZones` with the coefficients of Section 8.6; a porous core becomes the zone `porosity_core`. The solid part of the structure (the profile minus the porous bands, and minus a porous core) is the surface meshed by `snappyHexMesh`. Enabling any porous zone switches the solver to `porousWaveFoam`.

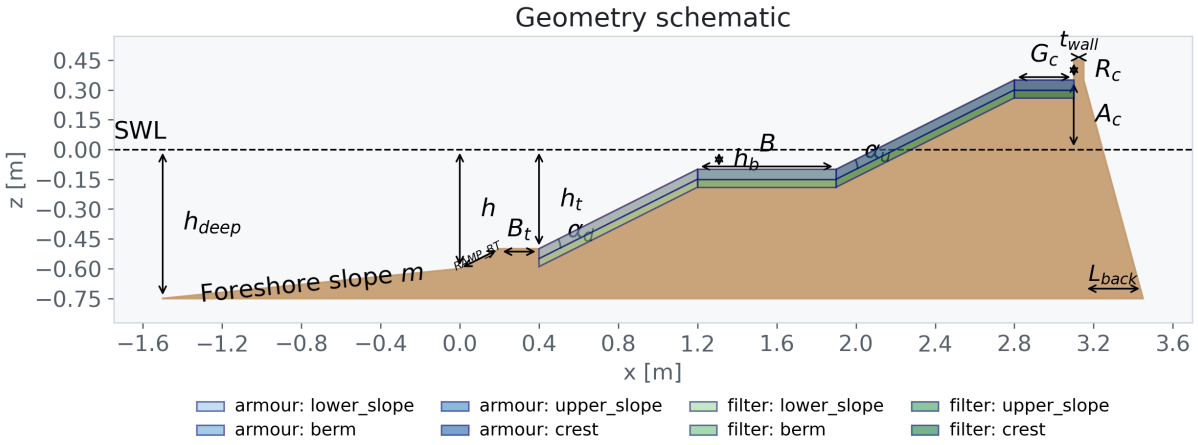


Figure 5.3: Cross-section of a layered permeable structure: armour and filter layers below the selected surface segments, over a core that may be impermeable or porous.

5.5.1 How many armour layers: the roughness factor γ_f

The EurOtop database does not describe the armour of a test in words: it gives its roughness factor γ_f (columns `gf_d`, `gf_u` and `gf`, Table 5.2) and its size (`D50_d`, `D50_u`, `D`). The value of γ_f identifies the type of armour through Table 6.2 of the EurOtop manual (EurOtop, 2018), reproduced in Table 5.3: for rock it gives the **number of armour layers** n_{layers} and whether the **core** is permeable. The table was derived for 1:1.5 slopes and breaker parameters $2.8 \leq \xi_{m-1,0} \leq 4.5$ (EurOtop, 2018).

Table 5.3: Roughness factor γ_f of permeable rubble mound structures with a 1:1.5 slope, from Table 6.2 of EurOtop (2018), and what it means for a COAST case. Values marked * are printed in italics in EurOtop: estimated or extrapolated. "–": not stated in the table.

Type of armour layer (EurOtop)	γ_f	**Armour layers n_{layers}**	**Core**
Smooth impermeable surface	1.00	0	Impermeable: no porous zone
Rocks (1 layer, impermeable core)	0.60	1	Impermeable
Rocks (1 layer, permeable core)	0.45	1	Permeable
Rocks (2 layers, impermeable core)	0.55	2	Impermeable
Rocks (2 layers, permeable core)	0.40	2	Permeable
Cubes (1 layer, flat positioning)	0.49	1	–
Cubes (2 layers, random positioning)	0.47	2	–
Antifers	0.50	–	–
HARO's	0.47	–	–
Tetrapods	0.38	–	–
Dolosse	0.43*	–	–
Accropode™ I	0.46	1	–
Xbloc®; CORE-LOC®; Accropode™ II	0.44	1	–
Cubipods, one layer	0.49*	1	–
Cubipods, two layers	0.47*	2	–

The number of layers of Accropode™, CORE-LOC®, Xbloc® and cubes in one layer follows the text of EurOtop, which calls them one-layer systems. The table does not state the core under the concrete units; its title refers to permeable rubble mound structures. Several units share the

same γ_f (0.47: cubes in two layers, HARO's, Cubipods in two layers), so for concrete units γ_f alone does not identify the unit: the reference of the test (column `Reference` of the database) does.

From γ_f to the inputs of COAST.

1. $\gamma_f = 1$: smooth impermeable slope. Do not enable porous zones; the surface roughness enters only through the wall roughness K_s of RANS cases (Section 8.5). Values above 0.60 and below 1 belong to impermeable revetments with a rough surface, *not* to rubble mounds (for example 0.90 for basalt or placed blocks; Table 5.2 of EurOtop, 2018); they are also modelled as a solid surface with a wall roughness.

2. $\gamma_f \leq 0.60$: rubble mound. Read the number of armour layers n_{layers} in Table 5.3 and compute the thickness of the armour normal to the slope, $t = n_{\text{layers}}\, k_t\, D_{n50}$ (CIRIA et al., 2007, Eq. 3.25), with the layer thickness coefficient k_t of the armour and the armour size of the record.

3. Convert t into the *vertical* thickness d entered in COAST with Eq. (5.10), with the slope of each zone. On a 1:1.5 slope, $d = 1.20\,t$; on 1:2, $d = 1.12\,t$.

4. Model the armour as **one** COAST layer of thickness d with the porosity and size of the armour (n_{layers} rock layers make one porous layer, not n_{layers} COAST layers). Add further COAST layers only for an underlayer or filter described in the reference of the test.

5. Core: impermeable core → *Solid core*. Permeable core → a porous core with its own porosity and size, if they are known; otherwise a solid core is an approximation that removes the flow into the core, and must be stated with the results.

The choices of the demonstration campaign are given in Section 11.3.

Porous results carry a calibration uncertainty. The resistance coefficients $(\alpha_{\text{vG}}, \beta_{\text{vG}})$ have no universal values, and n and d_{50} must represent the real armour (Section 8.6). Transferring one coefficient set across different armour families is a known source of scatter; it is visible in the demonstration campaign of Chapter 11. Treat predictions for permeable structures as conditional on that choice.

5.6 Profile segmentation and force patches

When *Create new force/impact patch* is enabled in the *Simulation and Job* tab, the structure outline on the plane $y = 0$ is split into numbered segments up to the overtopping section, and shown in an interactive view. Segments lying inside porous zones are drawn dashed and cannot be selected: the porous resistance model already accounts for the forces there. The faces of the bottom of an obstacle resting on the structure are removed from the list.

For each selected segment, COAST creates a face set `force_seg_NN` (the structure faces inside a box around the segment), turns it into a wall patch with `createPatch`, copies the boundary conditions of the structure patch `coast` to it, and adds one OpenFOAM `forces` function object `forceImpact_force_seg_NN` that integrates the fluid stresses over the patch during the run. No manual editing of the case is required. The resulting load time series are analysed in Section 10.11.

Chapter 6

Wave Conditions and Computational Domain

6.1 Wave parameters

COAST distinguishes two sets of wave parameters, and the distinction matters.

Deep-water parameters ($H_{m0,\mathrm{deep}}$, $T_{p,\mathrm{deep}}$, $T_{m,\mathrm{deep}}$) describe the sea state at the offshore boundary. $H_{m0,\mathrm{deep}}$ and $T_{p,\mathrm{deep}}$ define the target JONSWAP spectrum imposed at the wave generation boundary, at the depth h_{deep}; $T_{p,\mathrm{deep}}$ also sets L_0 and therefore the horizontal extent of the flume. $T_{m,\mathrm{deep}}$ is recorded in the case metadata.

Toe parameters ($H_{m0,\mathrm{toe}}$, $T_{p,\mathrm{toe}}$, $T_{m,\mathrm{toe}}$, $T_{m-1,0,\mathrm{toe}}$) are the user's best estimate of the sea state at the structure. They do *not* drive wave generation. $H_{m0,\mathrm{toe}}$ sets the half-height of the free-surface refinement band (Eq. 7.3), and all toe values are recorded in the case metadata so that the achieved conditions can be checked against them after the run (Section 10.5).

Random-phase seed. The phases of the wave components are random numbers drawn from an integer seed, entered in the *Wave and Domain* tab (*Random phase seed*). The default seed is **1234**. The same seed with the same spectrum gives exactly the same incident wave train; another seed gives another realisation of the same sea state (same H_{m0} and T_p). The seed is written to the wave generation dictionary `constant/waveProperties.input` (keyword `seedForRandomPhase`) and to `run_info.json`. Keep it fixed to reproduce a run or to compare structures, meshes or time steps under the same incident waves; vary it to estimate the statistical uncertainty of a result (Section 8.7).

The seed keyword. waves2Foam reads the seed only under the keyword `seedForRandomPhase`. If this keyword is missing, the phases are drawn from the system clock and change at every run. Case folders created with older COAST templates use the keyword `seedForRandom`, which waves2Foam does not read: the incident waves of these cases cannot be reproduced exactly. Check the keyword in `constant/waveProperties.input` before reusing an old case for a comparison.

The toe values are an *input estimate*, never a result. The only authoritative toe conditions are those measured from the simulated gauge signals after the run. If the two disagree substantially, the refinement band may have been sized wrongly and the case should be reconsidered.

6.2 Deep-water wavelength

From linear wave theory,

$$\boxed{L_0 = \frac{g\,T_{p,\mathrm{deep}}^2}{2\pi}} \tag{6.1}$$

with $g = 9.81\ \mathrm{m\,s^{-2}}$.

6.3 Domain sizing

The computational domain is a rectangular box sized from the bounding box of the STL and from the wave conditions.

6.3.1 Horizontal extent

In the automatic mode the flume extends a chosen number of deep-water wavelengths on either side of the STL:

$$x_{\min} = x_{\mathrm{STL,min}} - n_{\mathrm{pre}}\,L_0, \tag{6.2}$$

$$x_{\max} = x_{\mathrm{STL,max}} + n_{\mathrm{post}}\,L_0. \tag{6.3}$$

In the manual mode the two lengths are given directly in metres (inlet and outlet lengths, measured from the offshore and landward ends of the STL), for example when a specific laboratory flume is reproduced.

The upstream length must contain the inlet relaxation zone (Section 6.4) *and* leave a region of free propagation between the end of that zone and the structure, in which the waves develop and where the gauges for spectral and reflection analysis are placed. If it is too short, the structure stands in or against the generation zone and the incident waves are corrupted. The downstream length can be small for a structure with an open landward end; in breakwater mode it must contain the outlet relaxation zone, and the application lengthens it automatically when it is too short.

6.3.2 Vertical extent

$$z_{\min} = z_{\mathrm{STL,min}}, \tag{6.4}$$

$$z_{\max} = z_{\mathrm{STL,max}} + \Delta z_{\mathrm{clear}}, \tag{6.5}$$

where the air clearance $\Delta z_{\mathrm{clear}}$ (*top_clearance*) must exceed the expected height of the overtopping jet above the structure: if the jet reaches the top boundary, the flow is disturbed and the discharge is affected.

6.3.3 Lateral extent

The domain spans $y \in [0, L_y]$ with a single cell in y for pseudo-2D simulations ($N_y = 1$, Section 7.3).

6.4 Relaxation zones

Wave generation and absorption use the relaxation-zone technique of Jacobsen et al. (2012). In an inlet zone that starts at $x_{\min}$ and whose length is a fixed multiple of L_0 (one L_0 in the present version, constant `n_relax_inlet` of `coast_app.py`; the zone is written in `constant/waveProperties.input`), the computed solution is blended toward the target wave solution. The zone simultaneously generates the incident waves and absorbs the waves reflected by the structure that travel back offshore. Without breakwater mode, the downstream boundary carries passive outflow conditions (Section 8.3) and no absorption zone.

6.5 Breakwater mode

For low-crested structures and breakwaters with water on both sides, *Breakwater mode* adds an **outlet relaxation zone** at the downstream end of the flume, whose length is also a fixed multiple of L_0 ($1.5\,L_0$ in the present version, constant `n_relax_outlet`). This zone relaxes the solution toward still water and so absorbs the transmitted waves; without it, transmitted waves reflect off the downstream boundary and contaminate the record. The outlet velocity condition is set to allow outflow, and the initial water level is imposed on both sides of the structure.

6.6 Initial conditions

At $t = 0$ the fluid is at rest. In the standard mode, water ($\alpha = 1$) fills the flume below the SWL from $x_{\min}$ to just seaward of the overtopping section, and the region landward of that section is dry ($\alpha = 0$). In breakwater mode, water fills the whole flume below the SWL. Waves enter progressively from the inlet relaxation zone, with a soft-start ramp defined in the wave generation dictionary.

Chapter 7

Mesh Generation and Measurement Devices

7.1 Background mesh: blockMesh

The background hexahedral mesh is generated by `blockMesh` with cell counts obtained by rounding up the domain dimensions divided by the target cell sizes:

$$N_x = \left\lceil \frac{x_{\max} - x_{\min}}{\Delta x_{\text{target}}} \right\rceil, \qquad N_z = \left\lceil \frac{z_{\max} - z_{\min}}{\Delta z_{\text{target}}} \right\rceil, \tag{7.1}$$

so that the background cells are at most as large as the targets. N_y is an input and equals one for pseudo-2D simulations.

7.2 Surface refinement: snappyHexMesh

`snappyHexMesh` removes the cells inside the solid, conforms the mesh to the STL surface (snapping onto the surface and its feature edges, extracted by `surfaceFeatureExtract`), refines the cells along the structure surface, and applies two refinement regions (Figure 7.1). A refinement level ℓ divides the background cell size by 2^ℓ:

$$\Delta x_\ell = \frac{\Delta x}{2^\ell}, \qquad \Delta z_\ell = \frac{\Delta z}{2^\ell}. \tag{7.2}$$

Buffer layers of intermediate size are inserted between regions of different levels.

7.2.1 Interface band

A horizontal slab centred on the SWL that contains the expected free-surface excursion over the whole flume:

$$z \in \left[z_{\text{SWL}} - (H_{m0,\text{toe}} + \Delta z_e),\ z_{\text{SWL}} + (H_{m0,\text{toe}} + \Delta z_e)\right], \qquad x \in [x_{\min}, x_{\max}], \tag{7.3}$$

refined at level $\ell_{\text{interface}}$, where Δz_e is a user-adjustable buffer.

Because the band is computed from the same wave parameters as the rest of the case, the mesh resolution follows the physics by construction: increasing $H_{m0,\text{toe}}$ automatically widens the band.

7.2.2 Run-up band

A box around the structure in which the run-up, the overtopping tongue, and the flow over the crest are resolved:

$$x \in [x_{\text{rs}},\ x_{\text{re}}], \qquad z \in [z_{\text{STL,min}},\ z_{\text{STL,max}} + \Delta z_{\text{up}}], \tag{7.4}$$

refined at level ℓ_{runup}. The start $x_{\text{rs}} = x_{\text{STL,min}} + \Delta x_{\text{offset}}$ (`RUNUP_X_START_OFFSET`) is measured from the offshore end of the STL and kept within the STL; the end x_{re} (`RUNUP_X_END`) is an absolute x-coordinate; Δz_{up} (`RUNUP_TOP_ABOVE_STL`) is the height of the band above the highest point of the structure.

The run-up band must contain the crest, the overtopping section, and the space above it. The overtopping discharge, the energy flux, and the flow depth and velocity are all measured at the overtopping section (Section 7.5), whose vertical extent ends at the top of the run-up band. Step 4 also takes the cell size of this band as the reference resolution of the flow depth.

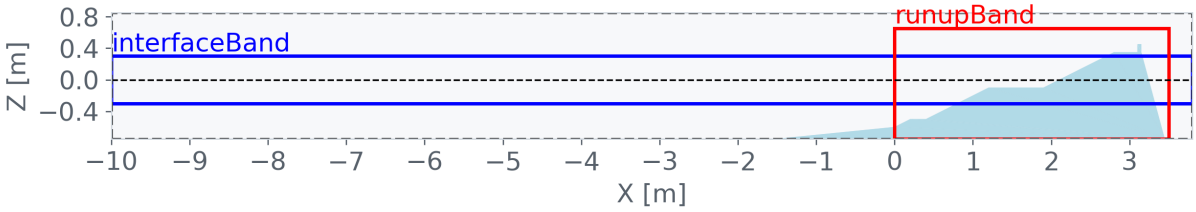


Figure 7.1: Numerical wave flume with the STL geometry and the `snappyHexMesh` refinement regions: the interface band (blue) spans the free-surface excursion over the whole domain; the run-up band (red) adds refinement around the structure. The dashed line marks the SWL.

7.3 Extrusion to pseudo-2D

After snapping, the mesh is reduced to a single-cell-thick layer in y by `extrudeMesh`; the front and back patches are of type `empty`. All simulations are therefore pseudo-2D; the consequences are discussed in Chapter 12.

7.4 Resolution guidance

Mesh resolution is the user's responsibility and the single most common cause of a misleading result. The free surface should be resolved by a sufficient number of cells per wave height in the interface band, and the band should cover the whole propagation path rather than only the vicinity of the structure: a wave that is damped numerically between the inlet and the toe arrives smaller than intended, and the resulting discharge is biased low. VOF interface capturing introduces resolution- and Courant-dependent amplitude and celerity errors (Larsen et al., 2019), which is why the spectral check (Section 10.5) is an essential verification step rather than an optional one.

Near the crest, the cell size of the run-up band,

$$\Delta_{\text{cell}} = \frac{\min(\Delta x_{\text{target}},\, \Delta z_{\text{target}})}{2^{\,\ell_{\text{runup}}}}, \tag{7.5}$$

limits what can be resolved: an overtopping tongue thinner than about one cell is represented only by a smeared interface, and its depth and velocity are not resolved (Section 10.10). The only reliable check is to repeat the case with a finer mesh and a smaller time step and to compare the quantities of interest.

7.5 Measurement devices written at build time

The overtopping section is placed at the landward end of the crest: at the landward face of the crown wall when there is one, at the end of the crest width G_c otherwise (the start of the collector when it is used). Four kinds of measurement are written into the case.

Free-surface iso-surface. The $\alpha = 0.5$ iso-surface is written at every field output time (`WRITE_INTERVAL`). Free-surface gauge signals are reconstructed from it after the run, at any position (Section 10.1).

Overtopping section. A face zone named `overtopping` is built by `topoSet` and `setsToZones`. It is a **single vertical wall of internal mesh faces**: the faces shared by the cells immediately seaward and immediately landward of the section, from the highest point of the structure up to the top of the run-up band, without boundary faces. When porous layers reach the section (porous crest wedge, or porous `back_slope`), the zone starts lower, at the top of the impermeable part of the structure at the section, so that the water flowing through the layers is counted as well. Because a face zone can only use existing faces, the wall lies on the mesh faces closest to the section. During the run, the waves2Foam function object `overtopping` sums the water flux through every face of the zone at every time step (Section 10.7). The zone must be a single wall: each additional wall of faces would count the same water again. The decomposition keeps every face of the zone with both its cells on one processor (Section 8.9). The faces of the zone on the mesh of a case are shown in Figure 10.12.

Sampling line `x01`. A vertical line at the overtopping section (at a fixed y inside the flume), from the highest point of the structure to the top of the run-up band, with uniformly spaced points. At each output time, the velocity $\boldsymbol{U}$, the pressure p_{rgh}, the density ρ, and the water fraction α are interpolated at the points (`cellPoint` interpolation) and written to `postProcessing/line/<time>/`. The line is written at its own interval, `LINE_WRITE_INTERVAL`, independent of the field output interval. It provides the energy flux (Step 2) and the flow depth and velocity (Step 4).

> **Choose the line interval before the run.** An overtopping event can last only a fraction of a second: if the line is written too rarely, the instant of maximum depth or velocity is missed, and the shortest events receive no sample at all. The interval cannot be refined after the run. Disk use grows in proportion to the inverse of the interval.

Forces. When force segments are selected, one `forces` function object per segment patch (Section 5.6).

Chapter 8

Numerical Model

8.1 Solver and governing equations

Cases run `waveFoam`, or `porousWaveFoam` when any porous zone is active, from the waves2Foam library (Jacobsen et al., 2012). Both solve the incompressible Navier–Stokes equations for two immiscible phases, water and air, with VOF interface capturing. COAST selects the solver automatically at build time.

8.1.1 Continuity

$$\nabla \cdot \boldsymbol{U} = 0 \tag{8.1}$$

8.1.2 Momentum

$$\frac{\partial(\rho \boldsymbol{U})}{\partial t} + \nabla \cdot (\rho \boldsymbol{U}\,\boldsymbol{U}) = -\nabla p_{\mathrm{rgh}} - \boldsymbol{g} \cdot \boldsymbol{x}\,\nabla\rho + \nabla \cdot (\mu_{\mathrm{eff}}\,\nabla \boldsymbol{U}) + \boldsymbol{f}_\sigma + \boldsymbol{S}_{\mathrm{porous}} \tag{8.2}$$

where $p_{\mathrm{rgh}} = p - \rho\,\boldsymbol{g} \cdot \boldsymbol{x}$ is the pressure minus the hydrostatic part, μ_{eff} the effective (molecular plus turbulent) viscosity, $\boldsymbol{f}_\sigma$ the surface-tension force (Brackbill et al., 1992), and $\boldsymbol{S}_{\mathrm{porous}}$ the porous resistance term, zero outside porous zones. Inside porous zones, `porousWaveFoam` solves the volume-averaged form of these equations (Jensen et al., 2014), in which $\boldsymbol{U}$ is the filter (Darcy) velocity and the inertia terms are scaled by the porosity and the added-mass coefficient (Section 8.6).

8.1.3 VOF transport

$$\frac{\partial \alpha}{\partial t} + \nabla \cdot (\alpha\,\boldsymbol{U}) + \nabla \cdot \left[\alpha\,(1-\alpha)\,\boldsymbol{U}_r\right] = 0 \tag{8.3}$$

with $\alpha \in [0,1]$ the water volume fraction. The compressive velocity $\boldsymbol{U}_r$, acting only at the interface, keeps the interface sharp; the equation is solved with the bounded MULES scheme (Rusche, 2002).

8.1.4 Mixture properties

$$\rho = \alpha\,\rho_w + (1-\alpha)\,\rho_a, \tag{8.4}$$

$$\mu = \alpha\,\mu_w + (1-\alpha)\,\mu_a. \tag{8.5}$$

The densities, viscosities and surface tension are those of `constant/transportProperties`.

8.2 Discretisation and solution

The equations are discretised with the finite-volume method. The templates use an implicit first-order (Euler) time scheme, a linear-upwind scheme for the convection of momentum, a van Leer scheme with interface compression for the convection of α (sub-cycled within each time step), and

the PIMPLE algorithm for the pressure–velocity coupling. The time step is adapted during the run to respect the two Courant-number limits of Table 8.2, up to a maximum time step set in the template. The schemes and solver settings are in `system/fvSchemes` and `system/fvSolution` of the templates.

8.3 Boundary conditions

Table 8.1: Boundary conditions of the templates. "No-slip" is a fixed zero velocity.

Patch	**Location**	$\boldsymbol{U}$	p_{rgh}	α
`inlet`	Offshore end	zero gradient	fixed flux pressure	zero gradient
`outlet`	Landward end	inlet–outlet (zero inflow)	fixed flux pressure	zero gradient
`ground`	Flume bottom	no-slip	fixed flux pressure	zero gradient
`coast`	Structure surface	no-slip	fixed flux pressure	zero gradient
`top`	Atmosphere	pressure inlet–outlet velocity	total pressure (zero)	inlet–outlet (air enters)
`front`, `back`	Spanwise faces	empty	empty	empty

Waves are imposed through the inlet relaxation zone, not through the inlet patch. In RANS mode, k and ω use wall functions on `coast` and `ground`, and the turbulent viscosity uses the rough-wall function of Section 8.5. Force patches created from structure segments receive the conditions of `coast`.

8.4 Turbulence treatment

Two options are provided. In **laminar** mode no closure is active and $\mu_{\mathrm{eff}} = \mu$. In **RANS** mode the Menter k-ω SST model (Menter, 1994) supplies $\mu_{\mathrm{eff}} = \mu + \rho\nu_t$:

$$\frac{\partial(\rho\,k)}{\partial t} + \nabla\cdot(\rho\,\boldsymbol{U}\,k) = \tilde{P}_k - \beta^{*}\rho\,\omega\,k + \nabla\cdot\left[(\mu + \sigma_k\,\mu_t)\,\nabla k\right] \tag{8.6}$$

$$\frac{\partial(\rho\,\omega)}{\partial t} + \nabla\cdot(\rho\,\boldsymbol{U}\,\omega) = \frac{\gamma}{\nu_t}\,P_k - \beta\,\rho\,\omega^2 + \nabla\cdot\left[(\mu + \sigma_\omega\,\mu_t)\,\nabla\omega\right] + 2(1-F_1)\,\frac{\rho\,\sigma_{\omega 2}}{\omega}\,\nabla k\cdot\nabla\omega \tag{8.7}$$

with turbulent viscosity

$$\nu_t = \frac{a_1\,k}{\max(a_1\,\omega,\; S\,F_2)}, \tag{8.8}$$

where F_1 and F_2 are the blending functions, S the strain-rate magnitude, and a_1, β^{*}, β, γ, σ_k, σ_ω, $\sigma_{\omega 2}$ the model constants of Menter (1994).

A known artefact. Standard two-equation closures over-produce turbulent kinetic energy beneath nearly potential-flow surface waves (Larsen and Fuhrman, 2018), which can damp the waves before they reach the structure. The laminar option provides a turbulence-free reference; both run from otherwise identical case set-ups, so the effect can be isolated by running the pair. If a RANS result looks low, this is the first thing to check.

8.5 Wall roughness

On impermeable structures, wall roughness enters through a rough-wall function for the turbulent viscosity (`nutkRoughWallFunction`) on the structure patch `coast`, with the Nikuradse equivalent sand roughness K_s and the roughness constant C_s entered in the *Simulation and Job* tab. They are written to `0.orig/nut` and apply to RANS only: the laminar solver has no wall function. K_s should represent the actual surface (smooth concrete, asphalt, grass, ...); the values used in the demonstration campaign, following Chen et al. (2021), are given in Section 11.3. The roughness of the flume bottom (`ground`) is set in the template.

8.6 Porous zone modelling

Flow resistance in armour, filter and core follows the extended Darcy–Forchheimer formulation of van Gent (1995), as formalised by Jensen et al. (2014) and implemented in waves2Foam:

$$\boxed{\boldsymbol{S}_{\text{porous}} = -\rho\,(a + b\,|\boldsymbol{U}|)\,\boldsymbol{U}, \qquad a = \alpha_{\text{vG}}\,\frac{(1-n)^2}{n^3}\,\frac{\nu}{d_{50}^2}, \qquad b = \beta_{\text{vG}}\left(1 + \frac{7.5}{\text{KC}}\right)\frac{1-n}{n^3}\,\frac{1}{d_{50}}} \tag{8.9}$$

and the unsteady inertia of the flow in the pores is increased by the added-mass coefficient $C_m = \gamma_{\text{AM}}\,(1-n)/n$. The parameters, entered per layer (Section 5.5), are:

- n: porosity;
- d_{50}: characteristic (median nominal) stone diameter [m];
- KC: Keulegan–Carpenter number of the oscillatory flow in the pores; a very large value suppresses the oscillatory contribution 7.5/KC (Moretto et al., 2020);
- α_{vG}: Darcy (laminar) resistance coefficient;
- β_{vG}: Forchheimer (turbulent) resistance coefficient;
- γ_{AM}: added-mass coefficient.

There is **no universal coefficient set**. Reported values range from the original $(\alpha_{\text{vG}}, \beta_{\text{vG}}) = (1000,\ 1.1)$ of van Gent (1995), through (200, 0.7) for breakwater armour and filter layers (Losada et al., 2008) and (500, 2.0) as a best average across flow regimes (Jensen et al., 2014), to layer-specific sets in large-scale validation studies (Vanneste and Troch, 2015). Without calibration data, take the porosity, the stone size and the layer thicknesses from the design of the structure (for example the Rock Manual, CIRIA et al., 2007), use a literature coefficient set as a starting point, and check the sensitivity of the results by repeating the simulation with other coefficients. The values adopted in the demonstration campaign are given in Section 11.3.

8.7 Wave generation and absorption

Irregular waves are generated by waves2Foam from a JONSWAP spectrum (Hasselmann et al., 1973) with the deep-water H_{m0} and T_p and the peak-enhancement factor γ of the wave dictionary. The spectrum is discretised into N components with equally spaced frequencies; their amplitudes follow from the spectrum, and their phases are random numbers drawn from the seed of Section 6.1. The incident waves are a linear (first-order) superposition of these components: second-order bound long waves are not generated, which can affect the conditions at the toe over a sloping foreshore. The generation starts with a soft ramp, and generation and absorption use the relaxation zones of Chapter 6.

Repeating wave groups. Because the N frequencies are equally spaced with a spacing Δf, the envelope of the generated signal (the sequence of wave groups) repeats with the period

$$T_{\mathrm{rep}} = \frac{1}{\Delta f}, \tag{8.10}$$

while the individual waves within the groups are not repeated identically. A record longer than T_{rep} therefore contains fewer independent wave groups than its duration suggests. Use several seeds, rather than one very long record, to reduce the statistical uncertainty of rare-event quantities.

The achieved incident waves are verified after the run, by spectral comparison at the gauges (Section 10.5) and by reflection analysis (Section 10.6). Neither check is optional if the result is to be used.

8.8 Time control and output

Table 8.2: Time-control and output parameters (*Simulation and Job* tab).

Input	OpenFOAM entry	Role and choice
`END_TIME`	`endTime`	Physical duration of the run. It must contain the start-up transient plus enough waves, and enough overtopping events, for the statistics of interest
`DELTA_T`	`deltaT`	Initial time step; adapted during the run
`MAX_CO`	`maxCo`	Courant-number limit of the flow; it governs the time step and affects the accuracy of wave propagation (Larsen et al., 2019)
`MAX_ALPHA_CO`	`maxAlphaCo`	Courant-number limit at the interface; it controls the transport of α
`WRITE_INTERVAL`	`writeInterval`	Output interval of the fields and of the free-surface isosurface; it is the sampling interval of every gauge signal, so $1/(2\,$`WRITE_INTERVAL`$)$ is the highest frequency of the spectra; it is also the shortest time between the snapshots of the event figures (Section 10.10.5)
Line write interval	`line` function	Output interval of the sampling line `x01` (Section 7.5); it controls the time resolution of Steps 2 and 4

The overtopping flux is written at every time step, independently of these intervals. The force function objects use a fixed output interval set by COAST.

Record length governs statistical convergence. At low overtopping rates a single large event can shift $\overline{q}$ by a factor of several within a record of finite length. It is one of the sources of scatter identified in the campaign of Chapter 11 (Section 11.5.1). Longer runs, or ensembles over phase seeds, are the remedy, not post-processing choices.

8.9 Parallel execution

Cases are decomposed with `decomposePar` over `N_PROCS` processes, with the `scotch` method (graph partitioning) or the `simple` method (slices along x). The decomposition keeps both cells of every face of the overtopping zone on the same processor (`preserveFaceZones` constraint), because the waves2Foam overtopping function object does not handle processor-boundary faces. The mesh is decomposed before `snappyHexMesh`, reconstructed for the extrusion, and decomposed again before the solver; for porous cases, `constant/porosityZones` is copied to every processor directory.

Chapter 9

Building and Running a Case

9.1 Build pipeline

Pressing *Build Case* copies the solver template (`Rans_OF_Main` or `Laminar_OF_Main`) to a new case folder named after the case, and writes every dictionary from the current parameter set. The build log reports each step:

1. Copy the template case (an existing folder of the same name is replaced);
2. Write the wall roughness in `0.orig/nut` (RANS only);
3. Write `constant/porosityZones` and add the porous convection scheme to `fvSchemes` (porous cases only);
4. Copy the structure and porous STLs to `constant/triSurface`;
5. Update `blockMeshDict` (domain extent and $N_x \times N_y \times N_z$);
6. Update `surfaceFeatureExtractDict`;
7. Write `snappyHexMeshDict` (surface, refinement regions);
8. Update the wave dictionary `waveProperties.input` (target spectrum, depth, SWL, seed, relaxation zones) and, in breakwater mode, the outlet velocity condition;
9. Write `setFieldsDict` (initial water);
10. Update `controlDict` (time control, iso-surface interval, sampling line);
11. Write the overtopping zone, the porous cell zones and the force face sets to `topoSetDict`, and the force patches to `createPatchDict`;
12. Write `decomposeParDict`;
13. Write `job_Run.sh`.

`run_info.json` and the case report are written when the simulation is launched (Section 3.5).

9.2 The generated job script

`job_Run.sh` is self-contained and can also be submitted directly to a scheduler. Its pipeline is:

Listing 9.1: Execution pipeline written into `job_Run.sh` (simplified).

```
source <OpenFOAM>/etc/bashrc         # detected automatically (Section 2.3)
source <waves2Foam>/bin/bashrc       # detected automatically (Section 2.3)
set -euo pipefail
command -v <solver> || exit 1        # stop early if the solver is missing

foamCleanTutorials
blockMesh
surfaceFeatureExtract
decomposePar -force
mpirun -n $N_PROCS snappyHexMesh -parallel -overwrite
reconstructParMesh -constant
extrudeMesh                           # reduce to a single cell in y
```

```
13
14 rm -rf 0 && cp -r 0.orig 0
15 # with force patches: copy the 'coast' boundary conditions to each new
16 # patch and write one forces function per patch; otherwise remove the
17 # forces function from controlDict
18 topoSet                                  # overtopping zone, porous zones,
       face sets
19 setsToZones -noFlipMap
20 createPatch -overwrite                   # only when force patches are
       requested
21
22 setWaveParameters
23 setWaveField
24 setFields
25 decomposePar -force
26 # porous cases: copy constant/porosityZones to every processor directory
27 mpirun -n $N_PROCS porousWaveFoam -parallel   # or waveFoam
28
29 mpirun -n $N_PROCS postProcess -func writeCellCentres -parallel -time 0
30 mpirun -n $N_PROCS postProcess -func writeCellVolumes -parallel -time 0
```

The complete output of the job is in `TMUX_OUT.log`. The longest steps also write their own logs: `log.snappy` (meshing), `log.topoSet`, `log.setsToZones`, `log.createPatch`, and `log.waveFoam` or `log.porousWaveFoam` (solver). When a run fails, these logs are the first place to look.

9.3 Execution and monitoring

Launch Simulation starts `job_Run.sh` in a dedicated tmux session named after the case, so the run survives interface restarts and remote disconnections. The session state is kept in the case folder: `TMUX_STATUS.txt` reads `RUNNING` during the run, then `FINISHED_<code>` (`FINISHED_0` for a successful run) or `KILLED`; `TMUX_SOCKET.txt` gives the socket with which the session can be attached from a terminal (`tmux -S <socket> attach`).

The *Monitor* tab searches a scan directory for case folders and shows, for each, its status, whether the session is alive (a status `DEAD?` means that the session disappeared while the status was still `RUNNING`), the run time, the number of solver processes, the memory used, and the solver progress (current time over `endTime`). For the selected case it shows the last lines of the solver log and offers three buttons: open the case report, open the post-processing page, and kill the run. The command-line tools of Section 2.6 give the same information without a browser.

Chapter 10

Post-Processing Pipeline

This chapter documents every method implemented in the COAST analysis pipeline. The analysis is organised in three groups:

- **Wave-field analysis** (Sections 10.1–10.6): gauge signals, trimming, wave detection, statistics, spectra, and reflection;
- **Overtopping chain** (Sections 10.7–10.10), four steps run in order from the *Overtopping* tab (Table 10.1);
- **Forces** (Section 10.11).

Table 10.1: The four steps of the overtopping chain.

Step	Script	Input data	Main results
1	`OV_Test.py`	`overtopping.dat` (overtopping section)	$q^+(t)$, $V^+(t)$, $\overline{q}$, events $\{V_e, q_{\mathrm{peak},e}\}$
2	`kinetic.py`	sampling line `x01`, events of Step 1	$F_{\mathrm{KE}}(t)$, $F_{\mathrm{PE}}(t)$, event energies
3	`merge_OV_energy.py`	results of Steps 1 and 2	E_e, e_e, energy–volume fits
4	`flow_depth_velocity.py`	sampling line `x01`, events of Step 1	$h_f(t)$, $u_f(t)$, $h_{f,\max}$, $u_{f,\max}$ per event, $h_{2\%}$, $u_{2\%}$

Steps 2 to 4 use the event windows of Step 1, so each event has one volume, one energy, one maximum flow depth and one maximum velocity, all at the same section. After Step 4, the same tab draws figures of one chosen event (Section 10.10.5). The overtopping chain uses its own averaging window (Section 10.7.2); it does not use the trimming of the wave analysis.

10.1 Free-surface signal extraction

During the run, the $\alpha = 0.5$ iso-surface (the air–water interface) is written at every field output time. Free-surface elevation signals $\eta(t)$ at any gauge position x_g are reconstructed *after* the simulation from the stored iso-surfaces: at each output time, the elevation is the mean height of the interface points inside a narrow window around (x_g, y_g), minus the SWL. Output times at which no interface point lies in the window are skipped.

Gauge positions therefore never need to be declared in advance. This is a practical advantage over both laboratory practice and conventional CFD probe set-ups, and it is what allows the reflection module to place its gauges freely during the analysis. The sampling interval of the signals is the field output interval.

Place gauges where the free surface is single-valued. Where the interface folds over (breaking waves, overturning jets, splashes), several interface points lie above the same x, and their mean is not a free-surface elevation. Gauges for wave analysis belong in the region of non-breaking waves seaward of the structure.

Extracted signals are stored as two-column text files (time [s], elevation [m]):

`Wave_Analysis/data/eta_from_freeSurface_x_<pos>m.txt`

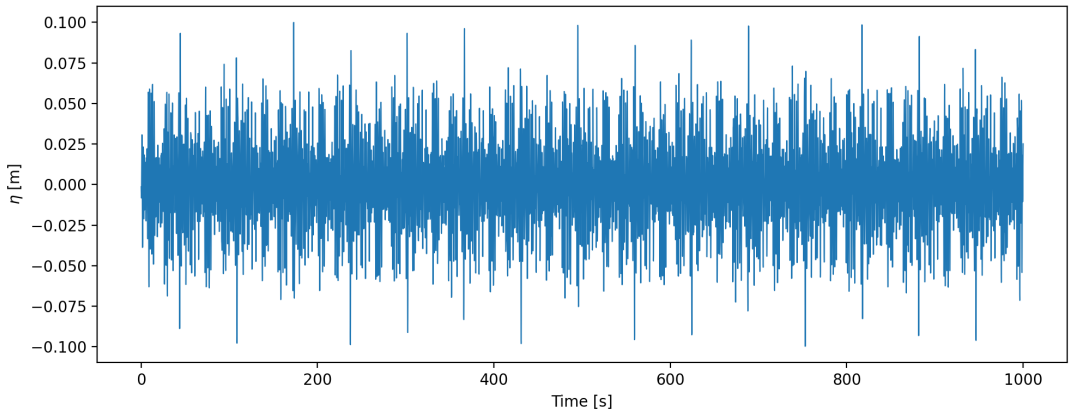


Figure 10.1: Raw free-surface elevation $\eta(t)$ at a gauge.

10.2 Transient trimming

The start-up transient must be removed before any wave statistic is computed. Two modes are available: **AUTO**, automatic stationarity detection (below), and **MANUAL**, a user-set cut time t_{cut}. The cut applies to every wave-analysis tab, and the trimmed signals start again at $t = 0$.

10.2.1 Stationarity detection

The automatic algorithm uses a sliding-window variance test:

1. Compute a **reference variance** from the tail of the record (the last fraction f_{tail} of the samples):
$$\sigma^2_{\text{ref}} = \text{Var}(\eta[i_{\text{tail}} :]); \tag{10.1}$$

2. Slide a window of duration T_w with a step T_s; for the window starting at index k,
$$\sigma^2_k = \text{Var}(\eta[k : k + N_w]); \tag{10.2}$$

3. Form the relative variance difference
$$\delta_k = \frac{|\sigma^2_k - \sigma^2_{\text{ref}}|}{\sigma^2_{\text{ref}}}; \tag{10.3}$$

4. Declare the signal stationary from the start of the first of N_{consec} consecutive windows that satisfy
$$\delta_k \leq \varepsilon. \tag{10.4}$$

If no such sequence is found, the signal is not trimmed. The parameters f_{tail}, T_w, T_s, ε and N_{consec} are set in `stationarity.py` and are not exposed in the interface.

The test compares against the *end* of the record, so it assumes that the run does reach a stationary state. On a record that is too short, or that drifts, it can return an early cut time that looks plausible and is not. Inspect the trimmed signal in the *Trimming* tab, and use the MANUAL mode when in doubt.

10.3 Wave detection by zero down-crossing

Individual waves are identified by **zero down-crossing** analysis of the signal (McCormick, 2010): a wave starts where the mean-subtracted elevation crosses zero downward and ends at the next downward crossing, so that each wave consists of a trough followed by a crest.

10.3.1 Mean removal

$$\eta'(t) = \eta(t) - \bar{\eta}, \qquad \bar{\eta} = \frac{1}{N}\sum_{i=1}^{N} \eta(t_i). \tag{10.5}$$

10.3.2 Zero crossings by linear interpolation

For each pair of consecutive samples with $\eta'_i > 0 > \eta'_{i+1}$, the crossing time is

$$\boxed{t_{\text{cross}} = t_i - \eta'_i \cdot \frac{t_{i+1} - t_i}{\eta'_{i+1} - \eta'_i}}. \tag{10.6}$$

10.3.3 Individual wave properties

For each wave bounded by successive down-crossings at t_a and t_b:

$$H_j = \max_{t\in[t_a,t_b]} \eta'(t) - \min_{t\in[t_a,t_b]} \eta'(t), \tag{10.7}$$

$$T_j = t_b - t_a. \tag{10.8}$$

An interval is kept only if its trough precedes its crest, as the down-crossing definition requires.

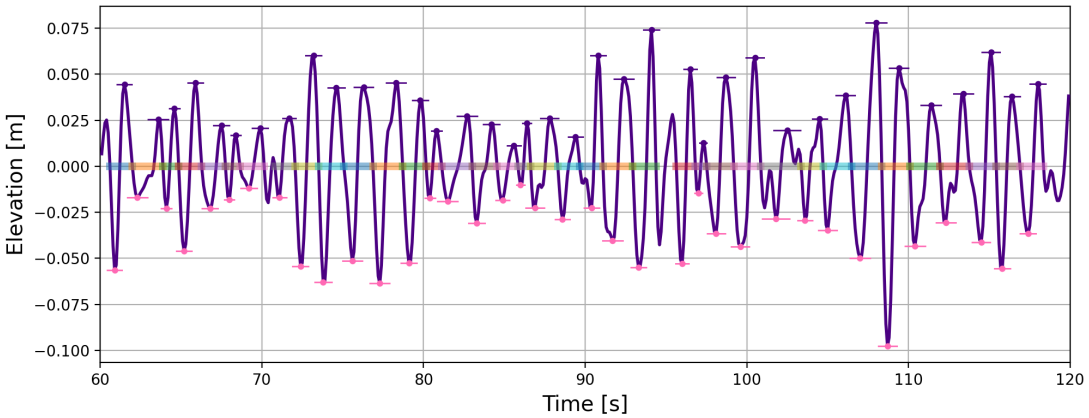


Figure 10.2: Zero-crossing wave detection on part of a trimmed signal: crest and trough of each wave are marked, and coloured segments show the individual wave periods.

10.4 Wave statistics

Given N_w detected waves with heights $\{H_1, \ldots, H_{N_w}\}$, and $H_{(j)}$ the j-th largest:

10.4.1 Characteristic heights

$$H_{\mathrm{mean}} = \frac{1}{N_w} \sum_{j=1}^{N_w} H_j, \tag{10.9}$$

$$H_{\mathrm{rms}} = \sqrt{\frac{1}{N_w} \sum_{j=1}^{N_w} H_j^2}, \tag{10.10}$$

$$H_{\mathrm{max}} = \max_j H_j, \tag{10.11}$$

$$H_{1/3} = \frac{1}{\lfloor N_w/3 \rfloor} \sum_{j=1}^{\lfloor N_w/3 \rfloor} H_{(j)}, \tag{10.12}$$

$$H_{1/10} = \frac{1}{\lfloor N_w/10 \rfloor} \sum_{j=1}^{\lfloor N_w/10 \rfloor} H_{(j)}. \tag{10.13}$$

The mean wave period $\bar{T} = N_w^{-1} \sum_j T_j$ and the number of waves N_w are reported with them.

10.4.2 Empirical distributions

With n_k the number of waves in height bin k and ΔH the bin width,

$$\mathrm{PDF}(H_k) = \frac{n_k}{N_w \, \Delta H}, \tag{10.14}$$

$$\mathrm{CDF}(H_k) = \sum_{i=1}^{k} \mathrm{PDF}(H_i) \, \Delta H, \tag{10.15}$$

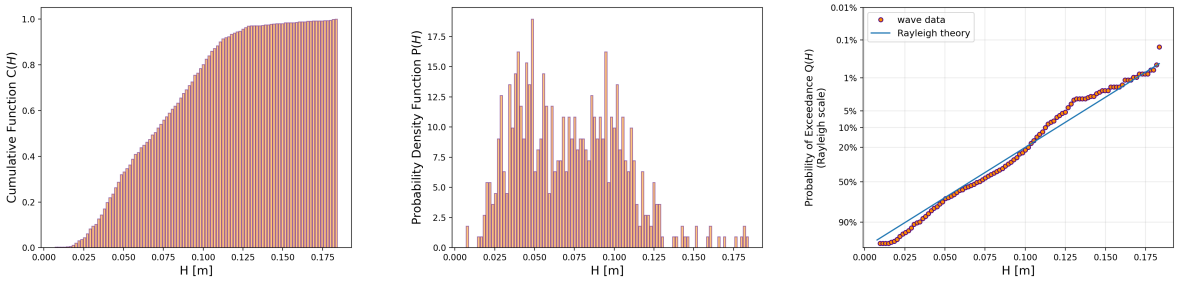
$$\mathrm{PoE}(H_k) = \sum_{i=k}^{N_{\mathrm{bins}}} \mathrm{PDF}(H_i) \, \Delta H. \tag{10.16}$$

10.4.3 Rayleigh reference

The Rayleigh probability of exceedance,

$$\boxed{\mathrm{PoE}_{\mathrm{Rayleigh}}(H) = \exp\left(-\frac{H^2}{H_{\mathrm{rms}}^2}\right)}, \tag{10.17}$$

is overlaid as a reference. It describes narrow-band waves in deep water. Departure from Rayleigh in the tail is expected in depth-limited conditions; departure across the whole distribution usually indicates a problem with the wave field rather than a physical effect.



(a) Cumulative distribution. (b) Probability density. (c) Exceedance, Rayleigh scale.

Figure 10.3: Wave-height statistics from the zero-crossing decomposition, with the Rayleigh reference. The vertical axis of (c) is a Rayleigh scale, on which the Rayleigh distribution (Eq. 10.17) is a straight line.

10.5 Spectral analysis

10.5.1 Welch power spectral density

The power spectral density $S(f)$ of the trimmed signal is estimated by the Welch method (Welch, 1967): the signal is divided into segments of N_{seg} samples overlapping by half, each segment is multiplied by a Hann window and Fourier transformed, and the periodograms are averaged. The sampling frequency is $f_s = 1/\Delta t$, with Δt the field output interval. The segment length is either automatic, $N_{\text{seg}} = \lfloor N/D \rfloor$ for a record of N samples and a divisor D, or fixed by the user (and then limited so that at least two segments remain). The frequency resolution is $\Delta f = f_s/N_{\text{seg}}$: longer segments resolve the peak better, shorter segments give a smoother, less noisy estimate.

10.5.2 Spectral moments and parameters

$$\boxed{m_n = \int_0^\infty f^n \, S(f)\,\mathrm{d}f \approx \sum_k f_k^n \, S(f_k)\,\Delta f} \tag{10.18}$$

(for $n = -1$ the sum excludes $f = 0$), from which

$$H_{m0} = 4\sqrt{m_0} \qquad \text{spectral significant wave height,} \tag{10.19}$$

$$T_{01} = \frac{m_0}{m_1} \qquad \text{mean spectral period,} \tag{10.20}$$

$$T_{02} = \sqrt{\frac{m_0}{m_2}} \qquad \text{mean zero-crossing period,} \tag{10.21}$$

$$T_{m-1,0} = \frac{m_{-1}}{m_0} \qquad \text{spectral period } T_{m-1,0}, \tag{10.22}$$

$$f_p = \arg\max_f S(f), \quad T_p = 1/f_p \qquad \text{peak frequency and period.} \tag{10.23}$$

$T_{m-1,0}$ is the period used by the EurOtop overtopping formulae, which is why it is reported alongside the others. The tab shows H_{m0}, T_p, $T_{m-1,0}$ and T_{01} with their difference from the values of the target spectrum.

10.5.3 Target JONSWAP spectrum

The measured spectrum is overlaid on the target JONSWAP spectrum, built with the H_{m0}, T_p and γ read from the wave dictionary of the case (the deep-water inputs):

$$\boxed{S_J(f) = C\, f^{-5} \exp\left[-\frac{5}{4}\left(\frac{f_p}{f}\right)^4\right] \gamma^{\,b(f)}, \qquad b(f) = \exp\left[-\frac{(f-f_p)^2}{2\,\sigma^2 f_p^2}\right],} \tag{10.24}$$

with $f_p = 1/T_p$ and the spectral-width parameter

$$\sigma = \begin{cases} 0.07 & f \le f_p \\ 0.09 & f > f_p \end{cases} \tag{10.25}$$

(Hasselmann et al., 1973; DNV, 2021). The constant C is computed numerically so that $4\sqrt{\int S_J \,\mathrm{d}f} = H_{m0}$ on the frequency axis of the measured spectrum.

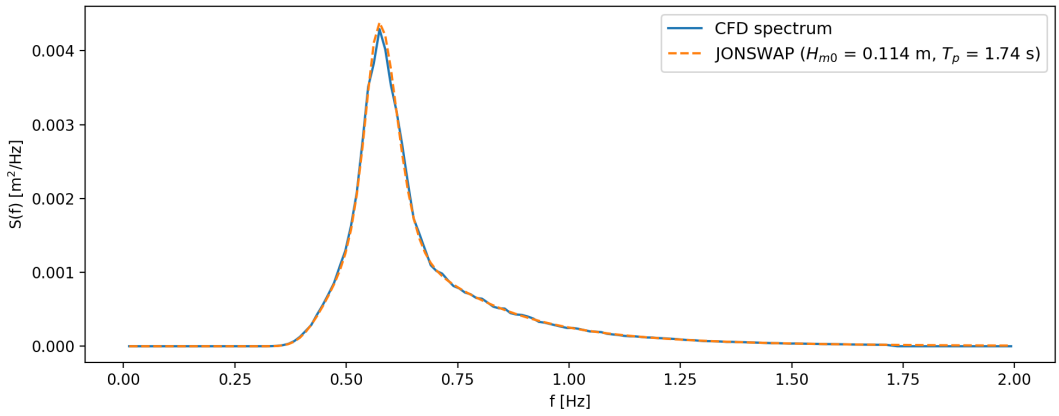


Figure 10.4: Measured spectrum compared with the target JONSWAP spectrum at a gauge.

This is the primary wave-field check, and it must be read in the right place. The target is the spectrum imposed at the generation boundary, at the depth h_{deep}. It is the right reference at gauges in the region of constant depth between the generation zone and the foreshore, where the two curves should agree. A deficit at the peak, a shifted peak, or spurious low-frequency energy there means that the incident waves are not those requested, and every discharge, force, and energy derived from that run is affected: do not proceed. Keep in mind that a gauge measures the incident *and* the reflected waves; the incident spectrum alone is given by the reflection module. Over a sloping foreshore and near the structure, shoaling and breaking change the spectrum: compare there with the expected toe conditions recorded in `run_info.json`.

10.6 Reflection analysis

The signal at any gauge is the superposition of the incident waves and of the waves reflected by the structure. Quantifying the reflection coefficient serves two purposes: it characterises the hydraulic response of the structure (rubble mounds dissipate most of the incident energy while

smooth or vertical faces reflect a large fraction of it, Zanuttigh and van der Meer, 2008), and it gives the incident wave height actually delivered by the flume.

COAST separates the two components with the three-gauge least-squares method of Mansard and Funke (1980), an extension of Goda and Suzuki (1976); see Zelt and Skjelbreia (1992) for arbitrary arrays.

10.6.1 Formulation

For gauge positions $x_1 < x_2 < x_3$ seaward of the structure, linear wave theory gives, at each frequency,

$$\hat{\eta}_j(f) = A_I(f)\, e^{-\mathrm{i}k(x_j - x_1)} + A_R(f)\, e^{+\mathrm{i}k(x_j - x_1)}, \qquad j = 1, 2, 3, \tag{10.26}$$

where $\hat{\eta}_j$ is the Fourier coefficient of the signal at gauge j and the wavenumber $k(f)$ follows from the linear dispersion relation

$$(2\pi f)^2 = g\, k\, \tanh(kh) \tag{10.27}$$

at the water depth h of the array. Three gauges give three equations for the two unknowns A_I and A_R, which are obtained by least squares at each frequency. The one-sided incident and reflected spectra are

$$S_{I,R}(f) = \frac{2\,|A_{I,R}(f)|^2}{N^2\,\Delta f}, \tag{10.28}$$

for a record of N samples and a frequency step Δf. Integrating them over the analysis band gives $H_{m0,\mathrm{inc}} = 4\sqrt{m_{0,I}}$ and $H_{m0,\mathrm{ref}} = 4\sqrt{m_{0,R}}$, and

$$\boxed{K_r = \frac{H_{m0,\mathrm{ref}}}{H_{m0,\mathrm{inc}}}}, \qquad 0 \le K_r \le 1. \tag{10.29}$$

$K_r \approx 0$ means that the structure absorbs the incident waves; $K_r \approx 1$ means near-total reflection and a standing wave in front of the structure.

10.6.2 Procedure

Because gauges are sampled from the stored iso-surfaces, the array is positioned freely after the run and the analysis requires no additional simulation. It proceeds in four steps.

(i) Gauge placement. COAST proposes an array that starts just shoreward of the inlet relaxation zone, with two **unequal** spacings expressed as fractions of L_p, the linear wavelength at the target peak period and the depth of the array. The module displays L_p and both spacings, and the positions and the depth can be changed. The depth is pre-filled with the generation depth of the wave dictionary; it must be the depth at the array. The spacings must differ: Equation (10.26) cannot be solved when a gauge separation is close to a multiple of half a wavelength ($k\,\Delta x = n\pi$), and unequal spacings ensure that no frequency of the analysis band is singular for both gauge pairs at once. Missing gauge signals are extracted from the module.

(ii) Free-surface extraction. At each output time the $\alpha = 0.5$ iso-surface is read, and $\eta(t)$ at each gauge is the mean interface height in a narrow window around x_j. All three gauges are sampled from the *same* output times and so share one time base. This matters: Equation (10.26) separates the components from the phase differences between the signals, so a timing offset between gauges would appear as reflected energy that is not there.

(iii) Transient removal and detrending. The trimming of the analysis settings, common to the three gauges, discards the start-up transient. The mean and a linear trend are then removed from each signal. Detrending is not cosmetic: a slowly drifting mean water level (wave set-up, or mass filling the flume) puts a large spike in the lowest Fourier coefficient, and because a very long wave has almost no phase difference across closely spaced gauges, the least-squares fit cannot distinguish incident from reflected there and splits that energy roughly in half, inflating K_r toward 1. Detrending is enabled by default and can be switched off.

(iv) Separation and integration. The three signals are Fourier-transformed over the whole trimmed record and Equation (10.26) is solved by least squares at each frequency of the analysis band $[f_{\min}, f_{\max}]$; frequencies at which the least-squares system is nearly singular are skipped. The application proposes a band around the peak frequency f_p, which can be changed. The module returns $H_{m0,\mathrm{inc}}$ (with its difference from the target H_{m0}), $H_{m0,\mathrm{ref}}$ and K_r, the incident and reflected time series reconstructed at gauge 1, and the incident and reflected spectra with the target spectrum. The results are written to `Wave_Analysis/reflection_results.json` for inclusion in the PDF report.

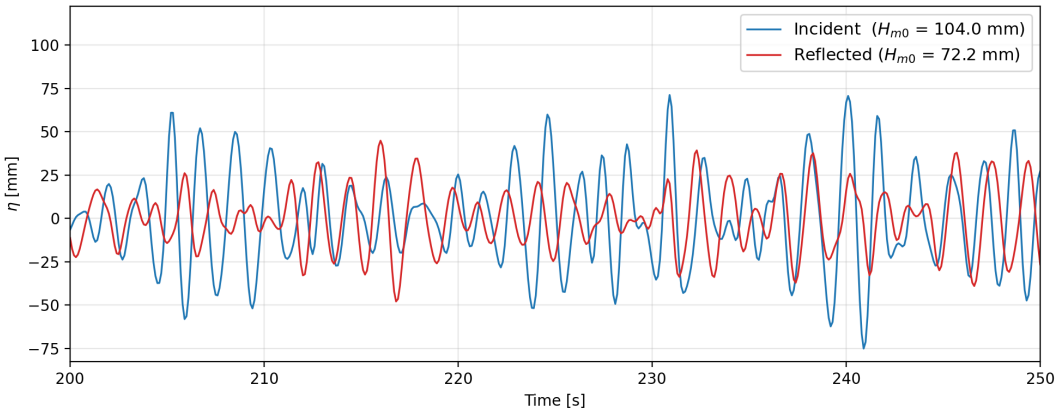


Figure 10.5: Output of the incident/reflected separation: incident and reflected free-surface elevation reconstructed at gauge 1 by the three-gauge Mansard–Funke method, with the spectral heights $H_{m0,\mathrm{inc}}$ and $H_{m0,\mathrm{ref}}$ integrated over the analysis band.

When K_r is not to be trusted. The method assumes linear, non-breaking waves at the array and is reliable only around the peak frequency. Keep the analysis band centred on the wave energy, keep detrending enabled, and remember that the result depends on where the array is placed: too far seaward the reflected waves are damped and partly re-absorbed by the inlet relaxation zone; too close to the structure the gauges enter the shoaling and partial-standing-wave region and K_r saturates. COAST provides no automatic criterion for verifying the placement; see Chapter 12.

10.7 Overtopping discharge and events (Step 1)

Step 1 is performed by `OV_Test.py`.

10.7.1 Discharge

The overtopping flux is recorded during the run by the waves2Foam `overtopping` function object on the overtopping zone (Section 7.5) (Jacobsen et al., 2012). At every time step, for each face f of the zone, it recovers the water volume flux from the solver's volume flux ϕ_f and mass flux $(\rho\phi)_f$,

$$\boldsymbol{q}_f = \frac{(\rho\phi)_f - \rho_a\,\phi_f}{\rho_w - \rho_a}\,\boldsymbol{n}_f = (\alpha\phi)_f\,\boldsymbol{n}_f, \qquad \boldsymbol{Q}(t) = \sum_{f\in S_{\mathrm{ov}}} \boldsymbol{q}_f, \tag{10.30}$$

where $\boldsymbol{n}_f$ is the unit normal of face f and $(\alpha\phi)_f$ the volume of water crossing the face per second. Because ϕ_f changes sign with $\boldsymbol{n}_f$, every contribution to the x-component Q_x is positive for water moving landward, whatever the orientation of the face. This is the flux the solver itself uses to transport the water, so it conserves mass and involves no interpolation. Only the landward component is retained, per unit width:

$$\boxed{q^+(t) = \frac{\max(Q_x(t),\,0)}{L_y}} \qquad [\mathrm{m^3\,s^{-1}\,m^{-1}}]. \tag{10.31}$$

Flume width in Step 1. The width L_y of Eq. (10.31) is a setting at the top of `OV_Test.py`; it is not read from the case. If the case was built with another width, edit the script before running Step 1.

The cumulative overtopped volume follows by trapezoidal integration over the solver time steps,

$$\boxed{V^+(t) = \int_0^t q^+(\tau)\,\mathrm{d}\tau}, \tag{10.32}$$

and the mean discharge over the averaging window $[t_1, t_2]$ is

$$\boxed{\overline{q} = \frac{1}{t_2 - t_1}\int_{t_1}^{t_2} q^+(t)\,\mathrm{d}t}. \tag{10.33}$$

10.7.2 Averaging window

FIRST_OVERTOPPING. t_1 is the first time the cumulative volume reaches a threshold, $V^+(t_1) \geq V_{\mathrm{thr}}$, and t_2 is the end of the record. This avoids biasing $\overline{q}$ low by averaging over the initial period without overtopping. If the threshold is never reached, the whole record is used and the summary says so.

MANUAL. The user sets t_1 and t_2 directly.

`OV_Test.py` also contains a `STATIONARY` mode, based on the running mean and standard deviation of q^+, which is not offered in the interface.

10.7.3 Event detection by hysteresis

Individual events are detected on $q^+(t)$ with a **two-threshold hysteresis** state machine (Table 10.2): an event opens when $q^+ \geq q_{\text{start}}$ and closes when $q^+ \leq q_{\text{end}} < q_{\text{start}}$. The gap between the two thresholds prevents a single overtopping wave from being split into several events by small fluctuations around one detection level.

Table 10.2: Hysteresis event-detection state machine.

State	Condition	Action
IDLE	$q^+(t_i) \geq q_{\text{start}}$	Enter EVENT; set $t_{\text{start}} = t_i$
IN_EVENT	$q^+(t_i) \leq q_{\text{end}}$	Exit EVENT; set $t_{\text{end}} = t_i$
IN_EVENT	End of signal	Close event at the last sample

Each event e carries

$$V_e = \int_{t_{\text{start}}}^{t_{\text{end}}} q^+(t)\,\mathrm{d}t, \tag{10.34}$$

$$q_{\text{peak},e} = \max_{t\in[t_{\text{start}},t_{\text{end}}]} q^+(t), \tag{10.35}$$

$$\Delta t_e = t_{\text{end}} - t_{\text{start}}. \tag{10.36}$$

Events shorter than a minimum duration or smaller than a minimum volume are discarded. Events are detected over the whole record, and only those that lie entirely inside $[t_1, t_2]$ are kept. They are numbered by decreasing volume (event 1 is the largest).

10.7.4 Detection parameters

All settings are exposed in the *Detection parameters* panel of the *Overtopping* tab. When Step 1 is run, the application writes a copy of `OV_Test.py` with these settings into the case folder and runs it there; no script editing is needed.

Table 10.3: Step 1 parameters.

Parameter	Unit	Role and effect
Averaging window		
Window mode	–	`FIRST_OVERTOPPING` (automatic t_1) or `MANUAL`
`Vthr_abs`	m^3/m	Cumulative-volume threshold locating t_1. A smaller value starts the window earlier; a larger value skips weak initial overtopping
t_1, t_2	s	Manual mode: start and end of the window; t_1 should lie after the start-up transient
Event detection		
`q_start`	$\mathrm{m}^3\,\mathrm{s}^{-1}\,\mathrm{m}^{-1}$	Upper hysteresis threshold: an event begins when q^+ reaches it. Raising it ignores weaker overtopping; lowering it captures more, and noisier, events
`q_end`	$\mathrm{m}^3\,\mathrm{s}^{-1}\,\mathrm{m}^{-1}$	Lower hysteresis threshold, strictly below `q_start`: an ongoing event ends when q^+ falls to it
`min_duration`	s	Shorter events are discarded
`min_volume`	m^3/m	Smaller events are discarded

The application refuses to run Step 1 unless $q_{\text{end}} < q_{\text{start}}$. It writes the copy of `OV_Test.py` into the case, runs it, shows its printed summary, and saves the summary as `overtopping_summary.txt`. The script ends with `plt.show()`: in a desktop session, its figures open in windows and Step 1 waits until they are closed, unless COAST was started with `MPLBACKEND=Agg` (Section 2.4).

10.7.5 Volume budget as a quality indicator

Within the averaging window, the overtopped volume is split into

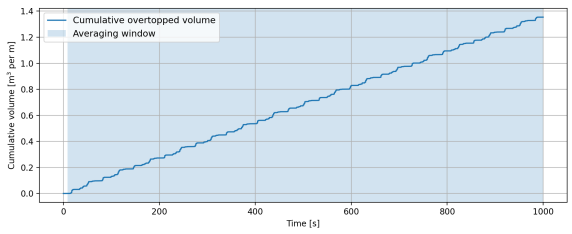
$$V_{\text{window}} = \int_{t_1}^{t_2} q^{+}\,\mathrm{d}t = \underbrace{\sum_{e=1}^{N_e} V_e}_{V_{\text{events}}} + V_{\text{background}}, \tag{10.37}$$

where $V_{\text{background}}$ collects the flux outside the detected events, including the weak discharges that numerical interface diffusion can produce in VOF simulations. The summary reports both parts, their percentages, and the closure error of Eq. (10.37).

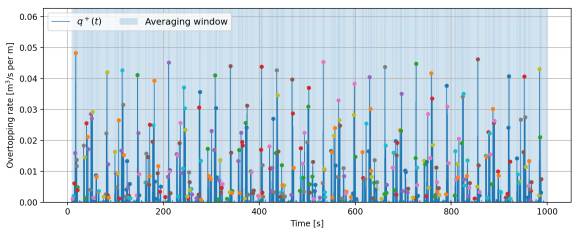
> A small ratio $V_{\text{background}}/V_{\text{window}}$ is a useful **quality indicator**: it means that the event catalogue accounts for essentially all the overtopped water. A large background fraction means either that the thresholds are set too high, or that the case produces spurious diffusive flux, and in that second situation $\overline{q}$ itself is suspect.

10.7.6 Outputs

`overtopping_events.csv` lists the events (`event_id`, `t_start`, `t_end`, `duration`, `volume`, `q_peak`); `overtopping_summary.txt` contains the printed summary (window, $\overline{q}$, number of events, volume budget, largest events). The figures show the cumulative volume with the averaging window, the rate $q^{+}(t)$ with the detected events, and the samples outside the events that make up $V_{\text{background}}$ (Appendix B).



(a) Cumulative volume $V^{+}(t)$; the shaded region is the averaging window of $\overline{q}$.



(b) Rate $q^{+}(t)$; markers indicate the peaks of the detected events.

Figure 10.6: Step 1: discharge, cumulative volume, and hysteresis-based event detection.

10.8 Kinetic and potential energy flux (Step 2)

Step 2 is performed by `kinetic.py`. The energy flux is evaluated on the sampling line `x01`, at the same section as the discharge, so that each event has a volume and an energy defined at one cross-section.

At every output time of the line, the energy carried by the water through the line is integrated over its height, with the water mass per unit volume $\rho_w \alpha$ and the landward velocity $u_n^+ = \max(u_x, 0)$:

$$F_{\mathrm{KE}}(t) = \int_{z_b}^{z_t} \tfrac{1}{2}\, \rho_w\, \alpha\, |\boldsymbol{u}|^2\, u_n^+ \, \mathrm{d}z, \tag{10.38}$$

$$F_{\mathrm{PE}}(t) = \int_{z_b}^{z_t} \rho_w\, \alpha\, g\, (z - z_{\mathrm{SWL}})\, u_n^+ \, \mathrm{d}z, \tag{10.39}$$

with

$$F_{\mathrm{tot}}(t) = F_{\mathrm{KE}}(t) + F_{\mathrm{PE}}(t) \quad [\mathrm{W\,m^{-1}}], \qquad E_{\mathrm{cum}}(t) = \int_0^t F_{\mathrm{tot}}(\tau)\, \mathrm{d}\tau \quad [\mathrm{J\,m^{-1}}]. \tag{10.40}$$

$|\boldsymbol{u}|^2$ includes the vertical velocity. The heights z_b and z_t of the line are read from `system/controlDict`, z_{SWL} from `run_info.json`, and ρ_w from `constant/transportProperties`. Weighting by $\rho_w \alpha$ is the same weighting as that of the overtopped water volume ($\int \alpha u_x \, \mathrm{d}z$), so that the energy and the volume of an event refer to the same water.

Integrating the fluxes over the event windows of Step 1 gives the event energies

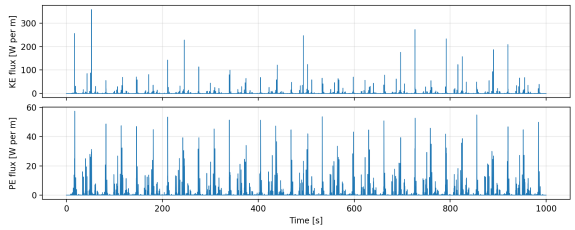
$$E_{\mathrm{KE},e} = \int_{t_{\mathrm{start}}}^{t_{\mathrm{end}}} F_{\mathrm{KE}}\, \mathrm{d}t, \quad E_{\mathrm{PE},e} = \int_{t_{\mathrm{start}}}^{t_{\mathrm{end}}} F_{\mathrm{PE}}\, \mathrm{d}t, \quad E_e = E_{\mathrm{KE},e} + E_{\mathrm{PE},e}. \tag{10.41}$$

All integrals use the trapezoidal rule, with linear interpolation at the window boundaries. If Step 1 has not been run, `kinetic.py` detects its own events on F_{tot} with a hysteresis whose thresholds are fractions of a high percentile of F_{tot}; this fallback is not used in the standard chain.

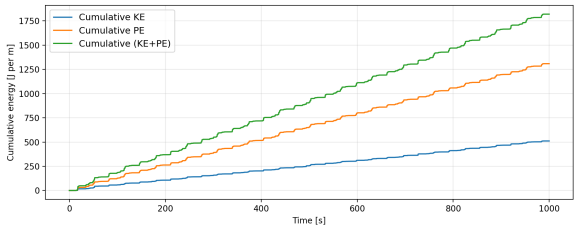
What F_{KE} and F_{PE} measure.

- They are the advective fluxes of kinetic and potential energy of the water through the section; the work of the pressure, $\int p\, u_x \, \mathrm{d}z$, is not included.
- The potential energy is measured from the SWL: water crossing the section carries at least $\rho_w g\, (z_b - z_{\mathrm{SWL}})$ per unit volume, even when it moves slowly.
- Weighting by α includes partially filled cells. A fast, aerated layer at the front of an overtopping tongue (small α) can carry a noticeable part of F_{KE} while holding little water.
- The line values are interpolated from the cells. The first point of the line lies on the structure, where the velocity is zero (no-slip), so the velocity in the lowest cell is underestimated; for thin tongues the line therefore carries somewhat less water than the solver flux of Step 1 (see the volume check of Step 4).

Outputs. `energyFlux_components.dat` (time, F_{KE}, F_{PE}, F_{tot}), `energyFlux_events.dat` (event energies), and figures of the flux components, the cumulative energy, and the histogram of event energies.



(a) Instantaneous kinetic and potential energy flux [$\mathrm{W\,m^{-1}}$].



(b) Cumulative energy per unit width [$\mathrm{J\,m^{-1}}$].

Figure 10.7: Step 2: energy flux through the overtopping section.

10.9 Energy–volume analysis (Step 3)

Step 3, performed by `merge_OV_energy.py`, merges the events of Step 1 with the energy fluxes of Step 2, discards the events whose volume is below a minimum set in the script, and characterises each event by its **specific energy**

$$\boxed{e_e = \frac{E_e}{V_e}} \qquad [\mathrm{J\,m^{-3}}], \tag{10.42}$$

the energy carried per unit volume of overtopped water. For each event it also reports the mean and the peak of F_{tot} during the event.

Two fits describe the ensemble of events.

10.9.1 Median regression

A straight line through the origin, $E_e \approx \hat{\beta}\, V_e$, gives the typical energy per unit volume. It is fitted by median (least absolute deviation) regression,

$$\hat{\beta} = \arg\min_{\beta} \sum_{e=1}^{N_e} \left| E_e - \beta\, V_e \right|, \tag{10.43}$$

solved as a linear programme, so that rare extreme events do not dominate the slope. Events above and below the line are distinguished in the figures.

10.9.2 Power law

Fitted by least squares in log–log space,

$$\boxed{E_e = a\, V_e^{\,\kappa}}, \qquad \ln E_e = \ln a + \kappa \ln V_e. \tag{10.44}$$

A super-linear exponent ($\kappa > 1$) indicates that large events carry more energy per unit volume than small ones. Whether this is relevant for the safety of people or for the erosion of the crest has not been established.

10.9.3 Medians

The median event energy and the median specific energy are also reported; events above the median energy are flagged as “strong”, and the events are ranked by decreasing specific energy.

Outputs. `overtopping_energy_merged.csv` (one row per event with V_e, $q_{\mathrm{peak},e}$, $E_{\mathrm{KE},e}$, $E_{\mathrm{PE},e}$, E_e, e_e, mean and peak flux, and the fit results) and figures of the event energies against volume, the median regression, the power law, the specific energy by rank, and the PDF and CDF of event energies (Appendix B).

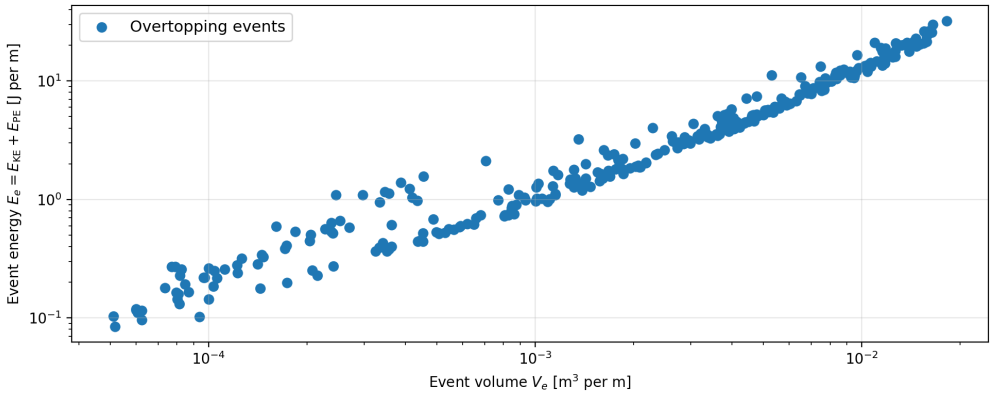


Figure 10.8: Event energy E_e versus event volume V_e for the detected overtopping events, from which the fits of Eqs. (10.43) and (10.44) are made.

10.10 Flow depth and flow velocity (Step 4)

Step 4, performed by `flow_depth_velocity.py`, gives the flow depth and the flow velocity of the overtopping water at the overtopping section, event by event. The design guidance for the flow on the crest of a dike (erosion of the crest and landward slope, safety of people) is expressed in these quantities (EurOtop, 2018).

10.10.1 Quantities at each output time

The script reads the profiles of α and u_x on the sampling line `x01` (Section 7.5), which runs vertically from the top of the structure, at z_b, to z_t, with N_{pts} equally spaced points. At each output time of the line:

$$h_f(t) = \int_{z_b}^{z_t} \alpha \, \mathrm{d}z \qquad \text{flow depth [m]}, \tag{10.45}$$

$$q_l(t) = \int_{z_b}^{z_t} \alpha \, u_x \, \mathrm{d}z \qquad \text{water flux through the line [m}^3\,\text{s}^{-1}\,\text{m}^{-1}\text{]}, \tag{10.46}$$

$$u_f(t) = \frac{1}{|W|} \sum_{k \in W} u_{x,k} \qquad \text{depth-averaged flow velocity [m/s]}, \tag{10.47}$$

where $W = \{k : \alpha_k \geq 0.5\}$ is the set of *wet* points of the line, excluding the first point, which lies on the structure surface where the interpolated velocity is the no-slip value zero.

- h_f is an *equivalent water depth*: the thickness of the water layer with the air removed. An aerated tongue looks thicker than h_f.
- Because the points are equally spaced, u_f is the depth average of u_x over the wet layer. It is not computed as q_l/h_f: when the layer is thin, a few fast-moving, mostly-air points would make that ratio explode.

- u_f is **not defined** (left empty, not set to zero) when $h_f < H_{\min}$ or when no point is wet. Such a layer is spread over about one cell and its velocity is not resolved by the mesh; the water does move, so zero would be wrong.

10.10.2 Values per event

For each event e of Step 1 (window $[t_{\text{start}}, t_{\text{end}}]$):

$$h_{f,\max,e} = \max_{t\in[t_{\text{start}},t_{\text{end}}]} h_f(t), \qquad u_{f,\max,e} = \max u_f(t), \tag{10.48}$$

together with the time of $h_{f,\max,e}$, the duration and volume of Step 1, and the number of line samples inside the window. An event shorter than the line output interval takes the sample nearest to its centre. $h_{f,\max}$ and $u_{f,\max}$ of an event generally occur at different instants: the velocity peaks at the front of the tongue, the depth later.

Two checks are provided:

$$V_{\text{line},e} = \int_{t_{\text{start}}-\Delta t_l}^{t_{\text{end}}+\Delta t_l} \max(q_l, 0)\,\mathrm{d}t, \qquad n_{\text{cells},e} = \frac{h_{f,\max,e}}{\Delta_{\text{cell}}}, \tag{10.49}$$

where Δt_l is the line output interval and Δ_{cell} the cell size of the run-up band (Eq. 7.5). $V_{\text{line},e}$ compares the water seen by the line with the volume V_e of Step 1; $n_{\text{cells},e}$ tells whether the flow depth of the event is resolved by the mesh (values below about one cell are not).

10.10.3 Values exceeded by 2 % of the incident waves

The EurOtop formulae for the flow on the crest use the flow depth and velocity exceeded by 2 % of the *incident waves*, not of the overtopping events (EurOtop, 2018). When the number of incident waves N_w in the analysis window is given (`N_WAVES`), with $k = \lfloor 0.02\, N_w \rfloor$,

$$h_{2\%} = k\text{-th largest } h_{f,\max,e}, \qquad u_{2\%} = k\text{-th largest } u_{f,\max,e}. \tag{10.50}$$

The result is zero when fewer than k events overtop, it is not computed when $k < 1$, and it is reported as not resolved when the event of rank k has no resolved velocity. N_w can be taken from the wave detection of the *Wave Detection* tab over the same time window.

10.10.4 Parameters, outputs and figures

The interface exposes $H_{\min}$ (`H_MIN`) and N_w (`N_WAVES`); the application writes them into a copy of the script in the case folder, as for Step 1. The line output interval is fixed at build time (Section 7.5). The outputs are `flow_depth_velocity_timeseries.csv` (t, h_f, q_l, u_f), `flow_depth_velocity_events.csv` (one row per event), `flow_depth_velocity_summary.txt`, and three figures: the time series of h_f, u_f and q_l with the event windows, the per-event maxima against the event volume (with the one-cell depth), and their probability of exceedance per event, $P_i = i/(N_e + 1)$ for the i-th largest value among N_e events. Events without a resolved velocity are not plotted in the velocity panels, whose title gives their number; they are still counted in N_e. The summary warns when events contain very few line samples, gives the number of events thinner than one cell, and gives the volume check $\sum_e V_{\text{line},e} / \sum_e V_e$.

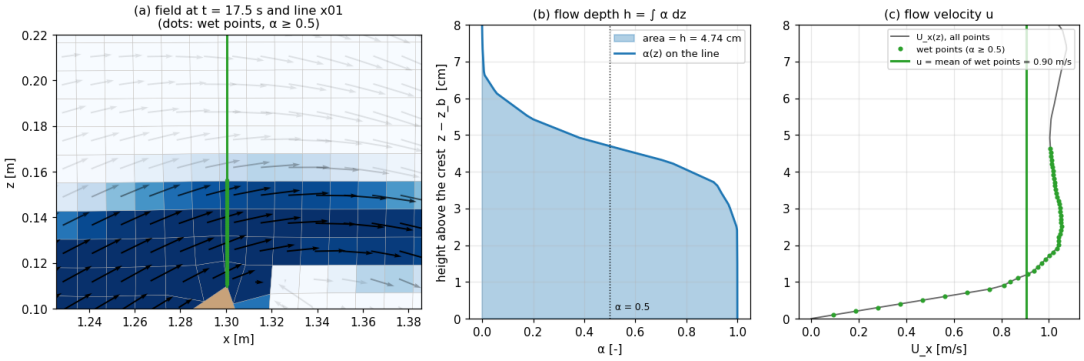


Figure 10.9: Flow depth h_f and depth-averaged velocity u_f at the overtopping section, from the profiles of water fraction and horizontal velocity on the sampling line (example run of Chapter 4, $t = 17.5$ s, the field output of Figure 10.12; h and u in the figure). (a) Water fraction and velocity around the crest edge, and the line `x01`; dots: wet points ($\alpha \geq 0.5$). (b) $\alpha(z)$ on the line: its integral is h_f (Eq. 10.45). (c) $u_x(z)$ on the line, and the mean over the wet points, u_f (Eq. 10.47); the first point, on the crest, is not used.

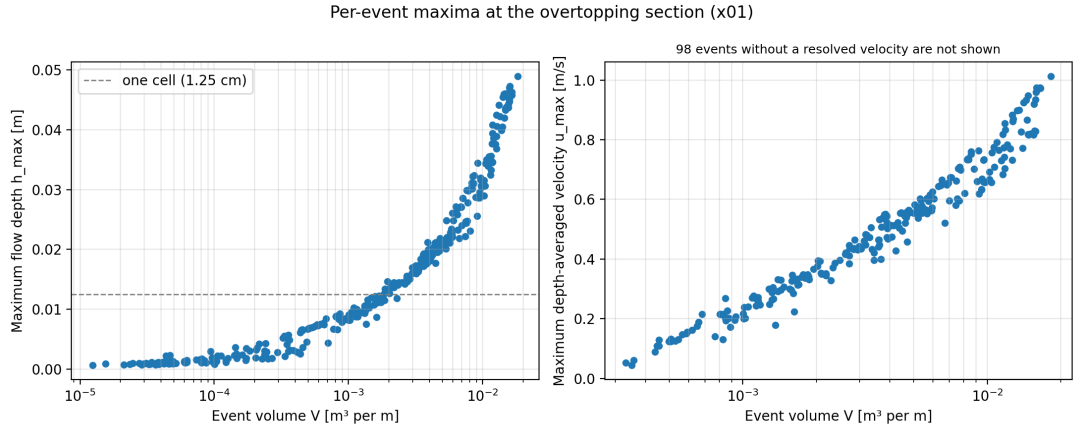


Figure 10.10: Maximum flow depth and maximum velocity of every overtopping event versus its volume: Step 4 figure `fdv_fig02_events_vs_volume.png` of the example run of Chapter 4. Dashed line: depth of one cell of the run-up band; the events without a resolved velocity are not shown in the right panel, and their number is given above it.

Reading flow depths and velocities.

- They are measured at one section, the landward end of the crest. What happens landward of it (the falling jet, the flow on a landward slope) is not measured. The front velocity of the overtopping tongue is not computed.
- Check n_{cells}: events thinner than about one cell are not resolved in depth, and their velocity is often undefined.
- Check the number of line samples per event: with a coarse line interval, the maxima of short events are underestimated.
- The velocity near the structure surface is underestimated by the interpolation (Section 10.8); the volume check $\sum V_{\text{line}} / \sum V_e$ below one is a symptom of it.

10.10.5 Figures of one overtopping event

Under the results of Step 4, the *Overtopping* tab draws two figures of one overtopping event chosen in a list, with the script `event_figures.py`. They show what the values of Steps 1 and 4 summarise: how the water crosses the overtopping section during the event, and how many cells of the mesh the overtopping layer spans. They require Steps 1 and 4 (`overtopping_events.csv` and `flow_depth_velocity_timeseries.csv`) and the field outputs of the run. The script can also be run from the case folder (Section 2.7).

Snapshots of the event (Figure 10.11). The upper part shows the crest at successive field outputs of the event, three per row, in a window around the overtopping section. Each panel shows:

- the water fraction α in the mesh cells, drawn with the cell edges, so that the thickness of the overtopping layer can be counted in cells;
- the velocity of wet cells ($\alpha \geq 0.5$) as arrows of equal length: the arrow gives the direction of the flow, its colour the speed $|\boldsymbol{U}| = (u_x^2 + u_z^2)^{1/2}$. One cell is kept in each square of a regular grid laid over the panel (the wet cell nearest to the centre of the square), so that the arrows stay readable on fine meshes;
- the water flux through each face f of the overtopping section, per metre of width,

$$\frac{q_{x,f}}{L_y} = \frac{(\alpha\phi)_f \, n_{x,f}}{L_y} \quad [\mathrm{m^3\,s^{-1}\,m^{-1}}], \tag{10.51}$$

  the x-component of the face flux $\boldsymbol{q}_f$ of Eq. (10.30) divided by the flume width L_y, with $(\alpha\phi)_f$ read from the field `alphaPhi0.water` written by the solver and $n_{x,f}$ the x-component of the unit normal of the face. Summed over the faces of the section, it gives Q_x/L_y, whose positive part is the discharge q^+ of Step 1 (Eq. 10.31).

The lower part gives, on a common time axis, (a) the discharge $q^+(t)$ of Step 1 with the event window and the event volume, (b) the flow depth $h_f(t)$ with the depth of one cell, and (c) the depth-averaged velocity $u_f(t)$ of Step 4, with the maxima $h_{f,\max}$ and $u_{f,\max}$ reported for the event. Vertical dotted lines mark the snapshot times, so that each panel of the upper part can be read against the time series.

Overtopping section (Figure 10.12). At the field output nearest the peak discharge of the event: (a) the flow over the structure with the mesh, the still water level, and the window of the zoom; (b) a zoom on the crest edge with the mesh cells and the faces of the overtopping section (Section 7.5).

Choices in the interface.

- *Event.* For each event the list gives its window, volume, $h_{f,\max}$, $u_{f,\max}$, and the number of field outputs inside the window. The events can be ordered by volume (event 1 is the largest, as in `overtopping_events.csv`) or by time.
- *Number of snapshots.* The snapshots are taken among the field outputs from just before the start of the event to just after its end: every output, or every k-th output for a long event. An event with fewer outputs gets fewer snapshots.
- *Snapshot times.* The time of the first snapshot and the time between snapshots can be set by hand; each snapshot is then the field output nearest to the requested time.

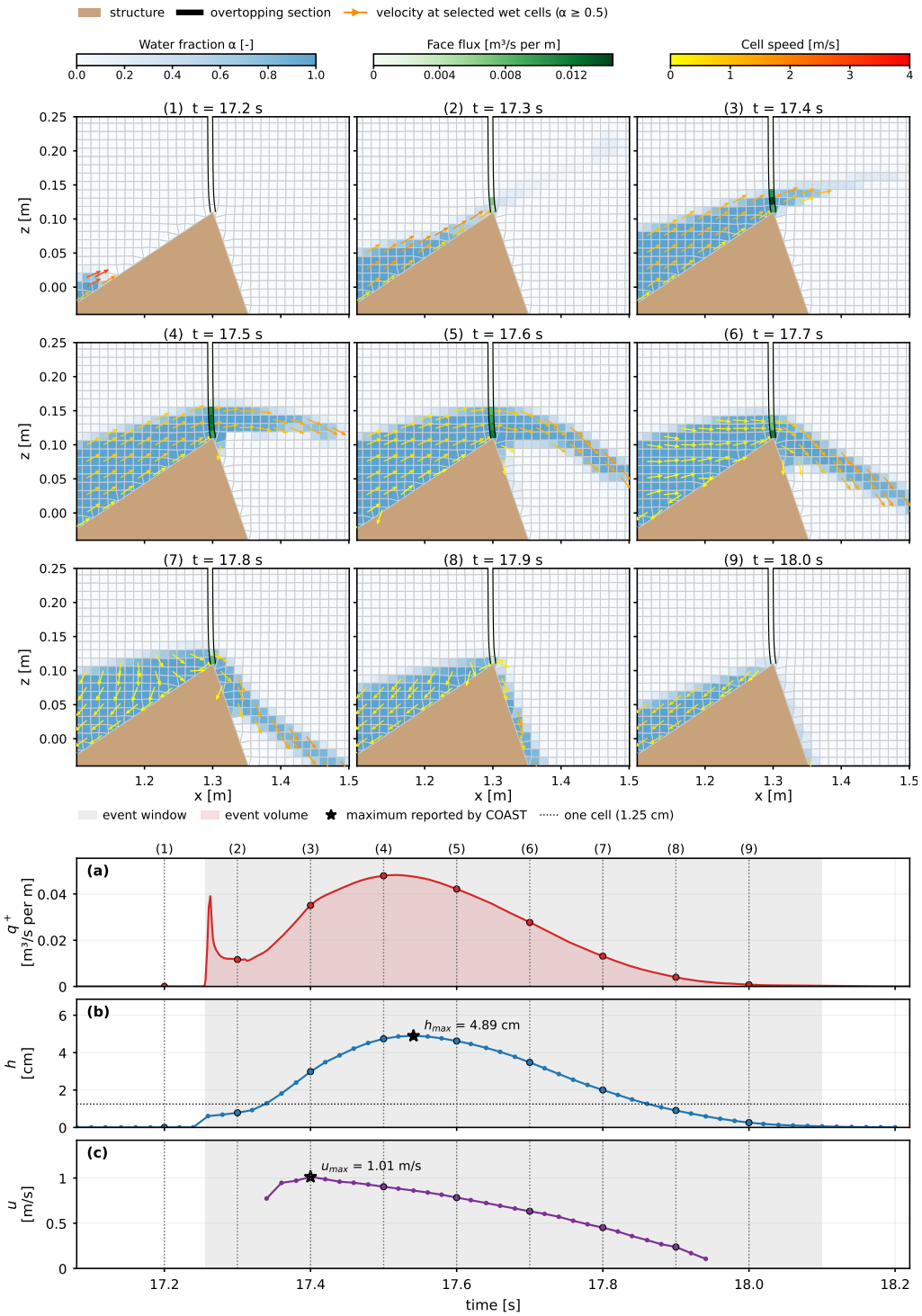


Figure 10.11: Snapshots of one overtopping event, as drawn by the *Overtopping* tab (`event1_snapshots.pdf`): the largest event of the example run of Chapter 4 ($t = 17.26$–18.10 s). (1)–(9): the crest at successive field outputs, with the water fraction and the mesh, the velocity of selected wet cells (arrows of equal length, coloured by the speed), and the water flux through each face of the overtopping section (Eq. 10.51). (a) Discharge q^+ (Step 1); (b) flow depth h_f (dotted line: one cell) and (c) depth-averaged velocity u_f (Step 4) at the section; grey: event window of Step 1; stars: $h_{f,\max}$ and $u_{f,\max}$ reported for the event.

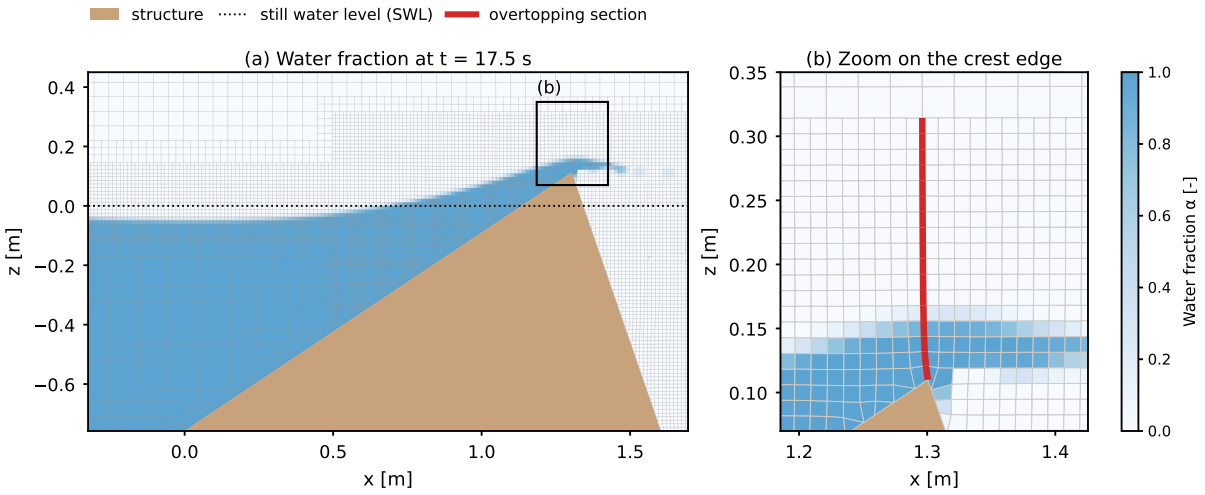


Figure 10.12: Overtopping section during the event of Figure 10.11, at the field output nearest its peak discharge (`event1_section.pdf`). (a) Water fraction and mesh over the structure; dotted line: still water level; box: window of (b). (b) Crest edge: mesh cells and, in red, the faces of the overtopping section through which the discharge is computed.

- *Top of the speed colour bar.* Left at zero, it is rounded up from the largest speed shown. Giving the same value to several events or cases makes their colours comparable.
- *Windows of the figures.* They are set from the position and the length of the sampling line `x01`, so that they follow the size of the structure, and they can be changed by hand.

The interface warns when some cells are faster than the top of the colour bar, and when `alphaPhi0.water` is not written (the faces of the section are then drawn without their flux).

Outputs. The figures are written to the folder `event_figures/` of the case, as PDF (vector) and PNG files named `event<N>_snapshots` and `event<N>_section`. The text file `event<N>_info.txt` lists the event window, the volume, the peak discharge, $h_{f,\max}$ and $u_{f,\max}$ with their times, the snapshot times, and the windows of the figures. The colours, the arrow length, the figure width and the resolution of the PNG images are set at the top of `event_figures.py`.

The snapshots are the field outputs. The figures read the fields α, $\boldsymbol{U}$ and `alphaPhi0.water` written every `WRITE_INTERVAL` (in the `processor*` folders, or in the time folders of a reconstructed case). The time between two snapshots cannot be shorter than `WRITE_INTERVAL`, and an event shorter than `WRITE_INTERVAL` contains one output or none. Choose `WRITE_INTERVAL` before the run if event figures are needed, and keep the time folders of the fields: the figures cannot be drawn once they are deleted.

10.11 Force extraction

For each segment selected at build time (Section 5.6), the OpenFOAM `forces` function object integrates the fluid stresses over the corresponding patch during the run and writes the pressure and viscous contributions to `postProcessing/forceImpact_<patch>/0/force.dat`. The

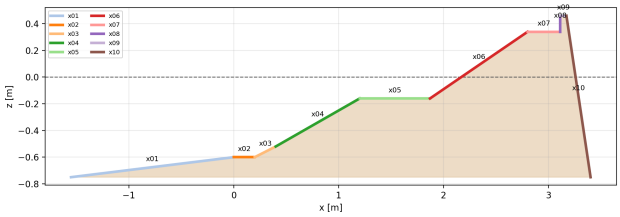
horizontal load on the patch is

$$\boxed{F_x(t) = \int_{\mathcal{S}} (p\,\boldsymbol{n} - \boldsymbol{\tau}\cdot\boldsymbol{n})\cdot\boldsymbol{e}_x \,\mathrm{d}S} \tag{10.52}$$

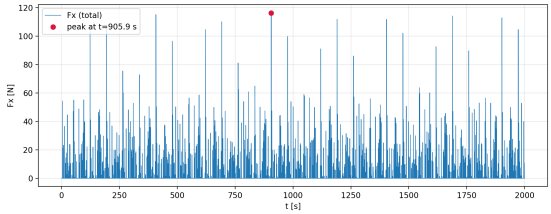
where $\mathcal{S}$ is the patch, $\boldsymbol{n}$ its unit normal pointing out of the fluid, and $\boldsymbol{\tau}$ the viscous stress tensor. The first term is the pressure force, the second the skin friction. COAST reports their sum,

$$F_x = F_{x,\text{pressure}} + F_{x,\text{viscous}}, \tag{10.53}$$

for the full flume width L_y; the load per metre of structure is F_x/L_y. Wave impact loads are pressure-dominated, so the viscous term is small, but it is retained rather than discarded. The *Force* tab shows $F_x(t)$ for each force directory with the peak $|F_x|$, its time, and the maximum and minimum, and saves the figure in the case folder for the PDF report.



(a) Automatic segmentation of the profile; force integration is restricted to the selected segment(s).



(b) Horizontal force $F_x(t)$; the marker indicates the peak.

Figure 10.13: Force analysis: segment selection and the resulting load time series.

> Peak impact pressures on vertical faces are strongly resolution-dependent and sensitive to entrapped air, which a two-phase VOF model at this resolution does not resolve. Treat the peak of F_x as indicative of timing and relative magnitude between comparable cases, not as an absolute design load.

10.12 Summary, comparison and reports

The *Summary* tab tabulates, for every gauge, the zero-crossing heights, the spectral parameters of Section 10.5, and the target values, which is the quickest way to see whether the wave field is consistent along the flume; the table can be downloaded as CSV. The same tab generates the wave-analysis HTML report and the PDF report (Section 3.5). Figures are collected for the reports as the tabs are displayed, so visit every tab of interest before generating a report.

The *Compare* tab overlays, for several cases, the cumulative overtopped volume, the horizontal force, the total energy flux, or the event energy against the event volume, and the figure can be downloaded. The data come from the files of each case: the cumulative volume is the time integral of the x-component Q_x of `postProcessing/overtopping/0/overtopping.dat`, negative values included and without division by L_y (it equals $V^+(t)$ of Step 1 only for $L_y = 1$ m and a flux that is never negative); the force is the last force directory of the case; the energy flux is `energyFlux_components.dat` (Step 2); and the event energies are `overtopping_energy_merged.csv` (Step 3). A case without the needed file is skipped with a warning. This is intended for parameter studies in which all cases were built and analysed by the same code, so that differences reflect the physics rather than the set-up.

Chapter 11

Demonstration Against the EurOtop Database

11.1 Purpose and status of this comparison

This chapter reports a demonstration campaign of 50 cases from the EurOtop database. It shows how COAST behaves over a range of structures and sea states, and it is the evidence base a user should weigh when judging whether the toolkit is appropriate for a given problem.

> **This is a demonstration, not a certification.** The campaign shows that the automated pipeline reproduces the overall trend of the measured discharges, with every case within one order of magnitude of the measurement, for the structure families and parameter ranges covered. It establishes nothing about configurations outside that range, and it does not transfer to your case. Agreement on 50 reference cases is not a warranty; see Chapter 15.

11.2 Campaign overview

The toolkit was demonstrated on **50 reference cases** chosen at random from the EurOtop database, spanning six series (A, B, C, D, F, G), seaward slopes $\cot\alpha \in \{1.5, 2, 4, 5\}$ plus a vertical wall, freeboard ratios $R_c^* \in [0.5, 3.7]$, and two structure families: smooth impermeable structures (series D and F: dikes and a vertical wall) and armoured breakwaters with permeable layers (series A, B, C, G). Measured discharges span four orders of magnitude. The upstream flume length was $n_{\mathrm{pre}} = 2.5$ deep-water wavelengths.

All 50 cases were run with the RANS k-ω SST closure, a background cell size of 0.05 m with two refinement levels (cells of 0.0125 m around the free surface and the structure), and a maximum Courant number of 0.8. The simulated duration was between 500 and 1000 s, limited by the computational time; the number of waves therefore differs between cases, as it also depends on the wave period. These records are much shorter than the roughly 1000 waves of physical model tests (Chapter 12).

The target spectrum was prescribed with the deep-water parameters of each record. For 46 of the 50 cases they are equal to the toe values of the database; for the other four (B-9.5, B-9.16, B-14.12 and F-36.48) the conditions at the toe result from the propagation over the foreshore in the numerical flume.

All cases were generated, executed, and analysed with the same automated pipeline, so differences between cases reflect the structure and the sea state rather than case-by-case choices made by the analyst.

Table 11.1: EurOtop database structure-type labels.

Label	Structure type
A	Rock armour, permeable core
B	Rock armour, impermeable core
C	Concrete armour units
D	Smooth impermeable slopes
F	Smooth dike and vertical wall
G	Rock armour, permeable core

11.3 Surface treatment and porous parameterisation

The structure family is identified through the EurOtop roughness factor γ_f recorded for each case (EurOtop, 2018), which encodes the armour system: $\gamma_f = 1.0$ for smooth impermeable slopes; $\gamma_f = 0.55$ for two rock layers on an impermeable core (series B); γ_f between 0.45 and 0.50 for concrete armour units (series C); $\gamma_f = 0.40$ for two rock layers on a permeable core (series A and G), and $\gamma_f = 0.45$ for one rock layer on a permeable core.

Smooth impermeable cases ($\gamma_f = 1.0$) are meshed as solid no-slip boundaries without porous zones, with wall roughness imposed through the rough-wall function using $K_s = 10^{-3}$ m and $C_s = 0.5$, following Chen et al. (2021).

Armoured cases ($\gamma_f < 1$) carry a Darcy–Forchheimer armour layer. One parameter set, not calibrated against the data of the campaign, is used for all armoured cases: $\alpha_{\mathrm{vG}} = 200$ and $\beta_{\mathrm{vG}} = 0.7$, proposed by Losada et al. (2008) for breakwater armour and filter layers; KC = 128 and an added-mass coefficient of 0.34, the values of the porous-media tutorial case distributed with waves2Foam, which follows Jensen et al. (2014); and a porosity $n = 0.416$, a value adopted for this study within the range of 30–55 % given for rock and concrete armour by EurOtop (2018). The size d_{50} of Eq. (8.9) is the armour size D_{50} of the EurOtop record of each case, also for the concrete units of series C.

The thickness of the armour normal to the slope is $t = n_{\mathrm{layers}}\, k_t\, D_{n50}$ (CIRIA et al., 2007, Eq. 3.25), with $k_t = 1.15$, a value adopted for this study, and D_{n50} equal to the size D_{50} of the record. The number of layers n_{layers} is taken from the roughness factor γ_f of the record with Table 5.3: two layers for $\gamma_f = 0.40$ and 0.55 (rock on a permeable and on an impermeable core), and one layer for $\gamma_f = 0.45$ (rock on a permeable core). For the concrete units of series C, for which γ_f does not fix the number of layers, two layers were used. The thickness of two layers, $t = 2.30\, D_{n50}$, is larger than the Rock Manual guidance for double layers of rock ($k_t = 0.87$–0.96; CIRIA et al., 2007, Table 3.9) and than the $1.83\, D_{n50}$ derived by Moretto et al. (2020). In all armoured cases, including series A and G whose core is permeable, the core is modelled as impermeable (*Solid core*) and covered by the porous armour layer; the flow into the core is not represented.

Using one coefficient set across different armour families is a deliberate simplification, and it is a known contributor to the scatter in the porous series.

11.4 Dimensionless parameterisation

Results are expressed in the dimensionless form of EurOtop (2018):

$$R_c^* = \frac{R_c}{H_{m0}}, \qquad q^* = \frac{\overline{q}}{\sqrt{g\,H_{m0}^3}}, \tag{11.1}$$

with $\overline{q}$ from the Step 1 analysis (Eq. 10.33), where H_{m0} is the spectral significant wave height at the toe of the structure given in the EurOtop database. The same H_{m0} is used for the measured and the computed q^*, so that $\log_{10}(q^*_{\mathrm{C}}/q^*_{\mathrm{E}}) = \log_{10}(q_{\mathrm{C}}/q_{\mathrm{E}})$.

11.5 Results

Agreement is measured primarily by the deviation from the 1:1 line in logarithmic space,

$$d = \log_{10}\frac{q^*_{\mathrm{C}}}{q^*_{\mathrm{E}}}, \tag{11.2}$$

with subscripts E (EurOtop) and C (COAST). The 95 % confidence intervals given in brackets are obtained from 10 000 bootstrap resamples of the cases. Across the 50 cases:

- the root-mean-square deviation is $\mathrm{RMSE}_{\log} = 0.47$ decades (0.40–0.54);
- the mean bias is +0.09 decades (−0.05 to +0.22);
- the mean absolute deviation is 0.39 decades (0.32–0.47);
- 23 cases are within a factor of 2 of the measurement, 34 within half a decade and **all 50 within one order of magnitude** (Figure 11.1, Table 11.2).

The squared Pearson correlation of the logarithmic discharges, $r^2 = 0.88$ (0.81–0.92), is given only as a complementary measure of how well the variation over four orders of magnitude is reproduced; it does not by itself measure agreement. The workflow therefore reproduces the overall discharge trends of different structure types, while individual cases can deviate by up to one order of magnitude.

The residuals are not random. For $\log_{10} q^*_{\mathrm{E}} < -3.5$, 10 of the 14 cases lie below the 1:1 line (mean residual −0.22 decades), whereas for larger discharges the mean residual is +0.21 decades; the root-mean-square deviation is similar in both ranges (0.51 and 0.46 decades). The largest over-predictions (+0.72 to +0.91 decades: D-1.10, D-1.13, D-19.4, C-26.8 and B-14.12) occur at moderate discharges.

The reliability factor RF and the complexity factor CF of the EurOtop database (from 1, the most reliable test or the simplest structure, to 4) are given in Table 11.2. All 50 cases have CF = 1, and 43 have RF = 1 (the other seven have RF = 2 or 3). These seven cases are too few to tell whether the cases with a larger RF deviate more from the measurements.

11.5.1 Sources of residual scatter

Three sources of the residuals have been identified in the campaign, to which the uncertainty of the experimental reference is added; each points to a different remedy.

1. **Incident wave height delivered in the flume.** For the 22 cases for which the database also gives a reflection coefficient, the incident wave height measured by the three-gauge

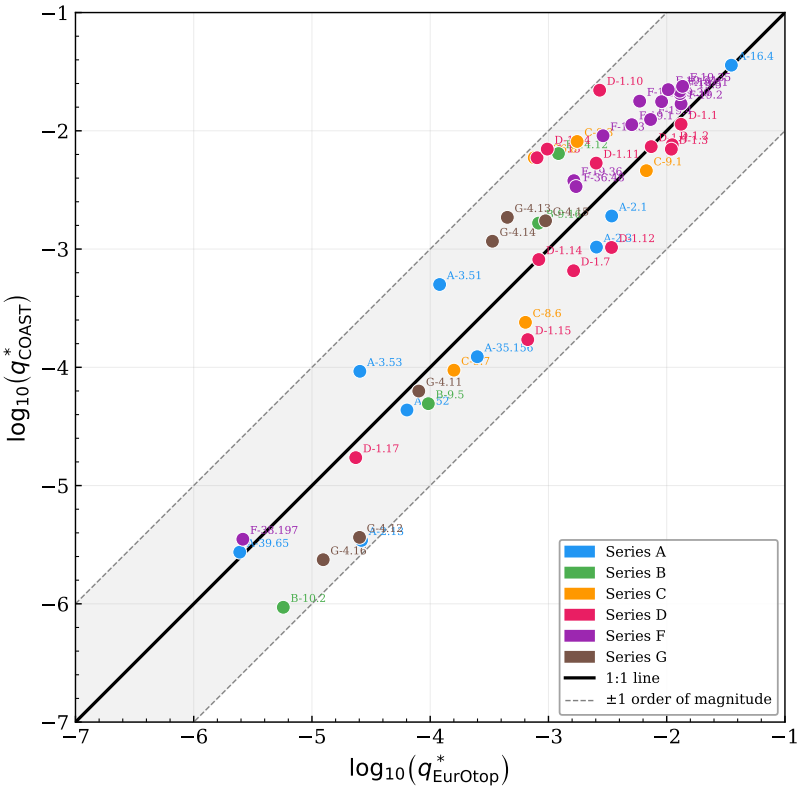


Figure 11.1: Parity of COAST versus EurOtop dimensionless discharge q^* for the 50 reference cases. The solid diagonal is the 1:1 line; dashed lines bound the ±1 order-of-magnitude band. Colours denote EurOtop series. Root-mean-square deviation 0.47 decades, mean bias +0.09 decades ($r^2 = 0.88$).

array of the reflection module (Section 10.6), placed just shoreward of the inlet relaxation zone, is lower than the height prescribed at the generator for 21 cases (ratio 0.57–1.14, median 0.87). The deficit is largest for the small, short-period waves of A-2.13, G-4.16, D-1.12 and G-4.12 (ratio 0.57–0.75), which are also the four most under-predicted of these 22 cases, and the ratio is strongly correlated with the discharge residual (Pearson $r = 0.75$). The under-prediction of small discharges is therefore mainly caused by incident waves that are smaller than intended. Compare the incident wave height of the reflection analysis with the target before interpreting $\overline{q}$.

2. **Statistical uncertainty.** The simulated records contain far fewer waves than physical model tests, and their wave groups repeat (Eq. 8.10); the overtopping of a record is dominated by a few large waves. This affects large as well as small discharges and is a plausible cause of the over-predictions listed above, for which, where it was measured, the delivered wave height is close to the target. Longer simulations, or repetitions with other phase seeds, reduce this uncertainty.

3. **Porous calibration.** A single $(\alpha_{\mathrm{vG}}, \beta_{\mathrm{vG}})$ set cannot be optimal simultaneously for the different armour gradings, core permeabilities, and unit shapes of series A, B, C, and G. Family-specific calibration is expected to reduce the scatter of the porous series.

4. **Uncertainty of the experimental reference.** The EurOtop database combines tests from many laboratories and scales; the measured discharges have their own uncertainty,

especially at very low rates, and some parameters of the database were estimated rather than measured (EurOtop, 2018). The deviations from individual tests therefore include experimental as well as numerical errors.

11.6 Case-by-case results

Table 11.2: COAST versus EurOtop database for the 50 reference cases. R_c^* and q^* as in Eq. (11.1); subscripts E and C denote EurOtop and COAST; RF and CF are the reliability and complexity factors of the EurOtop database. q in $\mathrm{m^3\,s^{-1}\,m^{-1}}$.

Case	$\cot\alpha$	H_{m0} [m]	R_c [m]	R_c^*	q_E	q_C	q_E^*	q_C^*	$\log_{10}\frac{q_C^*}{q_E^*}$	RF	CF
Series A: rock armour, permeable core											
A-2.1	1.5	0.0961	0.0620	0.645	3.20×10^{-4}	1.78×10^{-4}	3.43×10^{-3}	1.90×10^{-3}	−0.255	1	1
A-16.4	2.0	0.0940	0.0480	0.511	3.18×10^{-3}	3.24×10^{-3}	3.52×10^{-2}	3.59×10^{-2}	+0.009	3	1
A-2.13	1.5	0.0507	0.0620	1.223	9.38×10^{-7}	1.23×10^{-7}	2.62×10^{-5}	3.44×10^{-6}	−0.883	1	1
A-2.3	1.5	0.0790	0.0620	0.785	1.77×10^{-4}	7.22×10^{-5}	2.55×10^{-3}	1.04×10^{-3}	−0.390	1	1
A-35.156	2.0	0.1800	0.3500	1.944	5.98×10^{-5}	2.94×10^{-5}	2.50×10^{-4}	1.23×10^{-4}	−0.308	2	1
A-39.65	1.5	0.1020	0.2400	2.353	2.50×10^{-7}	2.79×10^{-7}	2.45×10^{-6}	2.74×10^{-6}	+0.048	2	1
A-3.51	2.0	0.1144	0.1300	1.136	1.46×10^{-5}	6.08×10^{-5}	1.20×10^{-4}	5.02×10^{-4}	+0.621	1	1
A-3.52	2.0	0.1054	0.1300	1.233	6.82×10^{-6}	4.67×10^{-6}	6.36×10^{-5}	4.35×10^{-5}	−0.164	1	1
A-3.53	2.0	0.0925	0.1300	1.406	2.24×10^{-6}	8.15×10^{-6}	2.54×10^{-5}	9.25×10^{-5}	+0.561	1	1
Series B: rock armour, impermeable core											
B-10.2	2.0	0.1200	0.2000	1.667	7.45×10^{-7}	1.22×10^{-7}	5.72×10^{-6}	9.35×10^{-7}	−0.787	1	1
B-14.12	2.0	0.0960	0.1360	1.417	1.13×10^{-4}	5.98×10^{-4}	1.22×10^{-3}	6.42×10^{-3}	+0.723	3	1
B-9.5	2.0	0.1411	0.2019	1.431	1.60×10^{-5}	8.18×10^{-6}	9.64×10^{-5}	4.93×10^{-5}	−0.291	2	1
B-9.16	2.0	0.1256	0.1752	1.395	1.15×10^{-4}	2.31×10^{-4}	8.24×10^{-4}	1.65×10^{-3}	+0.303	2	1
Series C: concrete armour units											
C-26.8	2.0	0.1290	0.1215	0.942	1.10×10^{-4}	8.59×10^{-4}	7.58×10^{-4}	5.92×10^{-3}	+0.893	2	1
C-5.7	1.5	0.1041	0.1420	1.364	1.67×10^{-5}	9.95×10^{-6}	1.59×10^{-4}	9.46×10^{-5}	−0.225	1	1
C-8.3	1.5	0.0805	0.0710	0.882	1.25×10^{-4}	5.81×10^{-4}	1.75×10^{-3}	8.12×10^{-3}	+0.668	1	1
C-8.6	1.5	0.0938	0.1180	1.258	5.75×10^{-5}	2.16×10^{-5}	6.39×10^{-4}	2.39×10^{-4}	−0.426	1	1
C-9.1	1.5	0.0959	0.0740	0.772	6.26×10^{-4}	4.28×10^{-4}	6.74×10^{-3}	4.60×10^{-3}	−0.165	1	1
Series D: smooth impermeable											
D-1.1	1.5	0.1138	0.1100	0.967	1.59×10^{-3}	1.37×10^{-3}	1.32×10^{-2}	1.14×10^{-2}	−0.066	1	1
D-1.2	1.5	0.1057	0.1100	1.041	1.20×10^{-3}	8.20×10^{-4}	1.11×10^{-2}	7.62×10^{-3}	−0.165	1	1
D-1.3	1.5	0.1095	0.1100	1.005	1.24×10^{-3}	7.94×10^{-4}	1.09×10^{-2}	6.99×10^{-3}	−0.195	1	1
D-1.4	1.5	0.0986	0.1100	1.115	7.18×10^{-4}	7.13×10^{-4}	7.40×10^{-3}	7.35×10^{-3}	−0.003	1	1
D-1.7	1.5	0.0595	0.1100	1.848	7.42×10^{-5}	2.98×10^{-5}	1.63×10^{-3}	6.56×10^{-4}	−0.396	1	1
D-1.10	1.5	0.1102	0.1870	1.697	3.11×10^{-4}	2.53×10^{-3}	2.71×10^{-3}	2.21×10^{-2}	+0.910	1	1
D-1.11	1.5	0.1026	0.1870	1.823	2.61×10^{-4}	5.49×10^{-4}	2.53×10^{-3}	5.33×10^{-3}	+0.323	1	1
D-1.12	1.5	0.1033	0.1870	1.810	3.55×10^{-4}	1.07×10^{-4}	3.41×10^{-3}	1.03×10^{-3}	−0.520	1	1
D-1.13	1.5	0.0913	0.1870	2.048	6.92×10^{-5}	5.11×10^{-4}	8.01×10^{-4}	5.92×10^{-3}	+0.869	1	1
D-1.14	1.5	0.0760	0.1870	2.462	5.43×10^{-5}	5.35×10^{-5}	8.29×10^{-4}	8.15×10^{-4}	−0.007	1	1
D-1.15	1.5	0.0764	0.1870	2.448	4.42×10^{-5}	1.13×10^{-5}	6.68×10^{-4}	1.72×10^{-4}	−0.591	1	1
D-1.17	1.5	0.0505	0.1870	3.701	8.35×10^{-7}	6.13×10^{-7}	2.35×10^{-5}	1.72×10^{-5}	−0.134	1	1
D-19.4	4.0	0.1180	0.1500	1.271	1.24×10^{-4}	8.89×10^{-4}	9.77×10^{-4}	7.01×10^{-3}	+0.856	1	1
Series F: smooth dike and vertical wall											
F-19.1	2.0	0.0847	0.1000	1.181	3.91×10^{-4}	8.70×10^{-4}	5.07×10^{-3}	1.13×10^{-2}	+0.347	1	1
F-19.2	2.0	0.1180	0.1000	0.847	1.68×10^{-3}	2.15×10^{-3}	1.32×10^{-2}	1.69×10^{-2}	+0.106	1	1
F-19.3	2.0	0.0853	0.1333	1.563	2.26×10^{-4}	7.09×10^{-4}	2.89×10^{-3}	9.09×10^{-3}	+0.497	1	1
F-19.4	2.0	0.1187	0.1333	1.124	9.35×10^{-4}	1.60×10^{-3}	7.30×10^{-3}	1.25×10^{-2}	+0.232	1	1
F-19.5	2.0	0.1580	0.1333	0.844	2.55×10^{-3}	4.06×10^{-3}	1.30×10^{-2}	2.06×10^{-2}	+0.202	1	1
F-19.30	2.0	0.1573	0.1333	0.847	1.77×10^{-3}	3.45×10^{-3}	9.06×10^{-3}	1.76×10^{-2}	+0.290	1	1
F-19.31	2.0	0.1793	0.1333	0.743	3.07×10^{-3}	5.15×10^{-3}	1.29×10^{-2}	2.17×10^{-2}	+0.225	1	1
F-19.34	2.0	0.1540	0.1000	0.649	1.95×10^{-3}	4.22×10^{-3}	1.03×10^{-2}	2.23×10^{-2}	+0.335	1	1
F-19.35	2.0	0.1700	0.1000	0.588	2.99×10^{-3}	5.22×10^{-3}	1.36×10^{-2}	2.38×10^{-2}	+0.242	1	1
F-19.36	2.0	0.1187	0.1333	1.124	2.10×10^{-4}	4.85×10^{-4}	1.64×10^{-3}	3.79×10^{-3}	+0.364	1	1
F-19.37	2.0	0.1560	0.1333	0.855	1.14×10^{-3}	3.44×10^{-3}	5.91×10^{-3}	1.78×10^{-2}	+0.479	1	1
F-36.48	5.0	0.0781	0.0800	1.024	1.17×10^{-4}	2.31×10^{-4}	1.71×10^{-3}	3.38×10^{-3}	+0.295	1	1

continued on next page

Table 11.2 continued from previous page.

Case	$\cot\alpha$	H_{m0}	R_c	R_c^*	q_{E}	q_{C}	q_{E}^*	q_{C}^*	$\log_{10}\frac{q_C^*}{q_E^*}$	RF	CF
F-38.197	0.0	0.1200	0.2000	1.667	3.39×10^{-7}	4.58×10^{-7}	2.60×10^{-6}	3.52×10^{-6}	+0.131	1	1
Series G: rock armour, permeable core											
G-4.11	2.0	0.0766	0.0700	0.914	5.31×10^{-6}	4.18×10^{-6}	8.00×10^{-5}	6.30×10^{-5}	−0.104	1	1
G-4.12	2.0	0.0593	0.0700	1.181	1.14×10^{-6}	1.65×10^{-7}	2.52×10^{-5}	3.65×10^{-6}	−0.840	1	1
G-4.13	2.0	0.0851	0.0700	0.823	3.49×10^{-5}	1.44×10^{-4}	4.49×10^{-4}	1.85×10^{-3}	+0.616	1	1
G-4.14	2.0	0.0814	0.0700	0.860	2.44×10^{-5}	8.46×10^{-5}	3.36×10^{-4}	1.16×10^{-3}	+0.540	1	1
G-4.15	2.0	0.0933	0.0700	0.750	8.43×10^{-5}	1.55×10^{-4}	9.45×10^{-4}	1.73×10^{-3}	+0.264	1	1
G-4.16	2.0	0.0534	0.0700	1.312	4.81×10^{-7}	9.12×10^{-8}	1.24×10^{-5}	2.36×10^{-6}	−0.722	1	1
Summary (log space): bias = +0.086, $\mathrm{RMSE}_{\log} = 0.472$, $r^2 = 0.878$											

Chapter 12

Limitations and Good Practice

12.1 Limitations

The following limitations define the scope of the present version and should be weighed before relying on any result.

1. **Two-dimensionality.** Simulations are pseudo-2D (single-cell extrusion). Oblique wave attack, directional spreading, and three-dimensional flow features are outside the current scope and enter design practice only through empirical correction factors such as γ_β (EurOtop, 2018). The architecture does not preclude 3D cases, but mesh cost and wave-basin boundary conditions have not been addressed.

2. **Interface-capturing accuracy.** VOF interface capturing introduces wave-amplitude and celerity errors that depend on resolution and Courant number (Larsen et al., 2019). This makes the spectral verification (Section 10.5) an essential check, not an optional one.

3. **Numerical resolution and time step.** Every result depends on the mesh and on the Courant limits. Thin overtopping tongues, of the order of a cell, are not resolved in depth or velocity. Convergence must be checked case by case by refining the mesh and the time step.

4. **Wave generation and record length.** The incident waves are a linear superposition of components; second-order bound long waves are not generated, which can affect the conditions at the toe over a sloping foreshore. With equally spaced frequencies the sequence of wave groups repeats (Eq. 8.10), so a record contains fewer independent wave groups than its duration suggests. Simulated records are usually much shorter than physical model tests, and the overtopping of a record is dominated by a few large waves: the mean discharge of a single record carries a statistical uncertainty, which can be estimated by repeating the run with other seeds.

5. **Porous-media calibration.** Armour and core resistance is volume-averaged (Eq. 8.9); individual armour units and their placement are not resolved. Predictions for permeable structures inherit the uncertainty of $(\alpha_{\mathrm{vG}}, \beta_{\mathrm{vG}}, n, d_{50})$, for which no universal values exist. Using one literature-based set for all armour families contributed visibly to the scatter of series A–C and G in Chapter 11.

6. **Parameter sensitivity.** The results depend on parameters chosen by the user: porous coefficients, wall roughness, mesh resolution, simulated duration, output intervals, and the event-detection thresholds. Their influence should be checked for the quantity of interest.

7. **Measurement at a single section.** Energy flux, flow depth, and flow velocity are measured on one vertical line at the landward end of the crest. The line values are interpolated, which underestimates the velocity next to the structure surface; the energy flux includes the contribution of aerated, partially filled cells and excludes the work of the pressure; and nothing landward of the section (falling jet, flow on the landward slope, front velocity) is measured.

8. **Gauge placement for reflection analysis.** The three-gauge separation assumes linear, non-breaking waves at the array, and the result depends on where the array is placed: too far seaward the reflected waves are damped and partly re-absorbed by the inlet relaxation zone; too close to the structure the gauges enter the shoaling and partial-standing-wave region and K_r saturates. COAST provides no objective criterion for verifying the placement case by case.

9. **Turbulence over-production.** Standard two-equation closures over-produce turbulent kinetic energy beneath nearly potential-flow surface waves (Larsen and Fuhrman, 2018), which can damp the incident waves. The laminar option provides a turbulence-free reference.

10. **Fixed geometry.** Structures are rigid and static: armour displacement, damage progression, and morphological change of the foreshore are not modelled.

11. **Uncertainty of the experimental reference.** When COAST is compared with laboratory data, the deviations include the uncertainty of the measurements as well as that of the simulation.

12. **Computational cost.** A VOF case remains orders of magnitude more expensive than empirical or data-driven estimates, with wall-clock times of hours to days per case depending on the mesh, the record length, and the number of cores. COAST is positioned as a *complement* to such tools, for configurations outside their validity range, for flow detail they cannot provide, and for generating physically consistent data to extend them, rather than as a replacement.

12.2 Recommended verification sequence

Before any result is used, and in this order:

1. Inspect the generated cross-section: is it the structure intended? For porous layers, are the vertical thicknesses consistent with the design (Eq. 5.10)?
2. Check the meshing output (`log.snappy`, `TMUX_OUT.log`) and confirm that the interface band covers the free surface over the whole propagation path and that the run-up band contains the crest and the overtopping section.
3. Inspect the raw gauge signals and the trimming point (Section 10.2).
4. Compare the measured spectrum with the target in the region of generation depth (Section 10.5). *Stop here if they disagree.* Compare the conditions at the toe with the expected toe values.
5. Compare the wave-height statistics with Rayleigh (Section 10.4).
6. Check K_r for plausibility against the structure type (Section 10.6).
7. Check the Step 1 summary: is $V_{\text{background}}/V_{\text{window}}$ small (Eq. 10.37), and does the record contain enough events for $\overline{q}$ to be meaningful?
8. For flow depth and velocity, check the number of cells and of line samples per event and the volume check of Step 4 (Section 10.10).
9. Repeat with a finer mesh and a smaller time step, and if the quantity depends on rare events, with other seeds; confirm that the quantity of interest has converged.

None of these steps is performed automatically, and none can be skipped on the grounds that the software produced a plausible-looking number.

Chapter 13

Questions and Troubleshooting

This chapter answers the questions that come up when COAST is used, grouped by stage. Each answer points to the section where the subject is treated in full.

13.1 Installation and start

`streamlit` is not found, or Python reports `ModuleNotFoundError: python_tools`. Install the packages with `pip install -r requirements.txt` (or `./install.sh`), and start COAST from the repository root after `export PYTHONPATH="$(pwd):$(pwd)/python_tools"` (Section 2.3).

Can COAST run on a server without a screen? Yes: start it on the server and open it from a local browser through an SSH tunnel on ports 8501 and 8502 (Section 2.4). The simulations run in tmux sessions on the server and continue when the browser, the SSH connection or the application is closed.

Can two users, or two instances, run COAST on the same machine? Each instance needs its own port: start the second one with `streamlit run coast_app.py -server.port 8601`. The file server that opens the case reports always uses port 8502; with two instances, the report button of one of them opens the files of the other. The tmux socket of a case is `/tmp/tmux_<casename>.sock`: two cases with the same name on the same machine, even in different folders, use the same socket. Give every case a distinct name.

***Launch Simulation* reports that tmux is not found.** Run `./install.sh`, which builds tmux in `tools/tmux/`, or set `export TMUX_BIN=/path/to/tmux` before starting COAST. COAST looks for tmux in this order: `TMUX_BIN`, `tools/tmux/bin/tmux`, then the `PATH`.

The run stops at once with "solver not found". The job script did not find OpenFOAM or waves2Foam. Source both in `~/.bashrc`, or set `COAST_OPENFOAM_BASHRC` and `COAST_WAVES2FOAM_BASHRC` before starting COAST (Section 2.3). The message is in `TMUX_OUT.log`.

13.2 Inputs and set-up

I reloaded the page and my inputs are gone. The inputs are kept in the browser tab only, until the launch (Section 4.2). Nothing can be recovered from a session that was closed before *Launch Simulation*.

How do I set up a case like a previous one? Open its `run_info.json` and enter the values again; the keys are the names of the inputs (Appendix A). Five inputs are not in this file and are read from the case dictionaries (Section 3.5). Use a new case name.

How do I enter a case of the EurOtop database? Table 5.2 gives the input that receives each column, Table 4.2 an example, and Section 5.5.1 the porous layers.

How do I model a vertical wall? Set the slope cotangent to 0: $\Delta x = \Delta z \cot \alpha = 0$ gives a vertical face. A wall on top of a sloping structure is a crown wall (next question) or an obstacle (Section 5.4).

How do I add a crown wall? Set $R_c > A_c$: a wall of thickness `t_wall` rising to R_c above the SWL is placed at the landward end of the crest. With $R_c = A_c$ there is no wall, and `t_wall` is not used.

The berm is above the SWL. Enter a negative `h_b`: it is a water depth above the berm, measured at mid-berm.

How do I get a sloping foreshore, or the flat bed of a flume? `m` is the cotangent of the foreshore slope between h_{deep} and h. `m` = 1000 selects the flat flume, with a short ramp between the two depths; with $h_{\text{deep}} = h$ there is no ramp at all.

How many armour layers should I put, and how thick? Read γ_f of the database record in Table 5.3: it gives the number of rock layers n_{layers} and the type of core. The armour is *one* COAST layer of vertical thickness $d = n_{\text{layers}}\, k_t\, D_{n50} / \cos\alpha$; the steps are in Section 5.5.1.

The porous layer of a zone is missing in the section. A zone that does not exist in the geometry is skipped: for example `crest` with $G_c = 0$, or `berm` with $B = 0$. `back_slope` exists only without crown wall and without collector. The message under *Generate STL* lists the zones that were built.

Is the thickness of a layer measured normal to the slope? No: it is the *vertical* thickness d (Eq. 5.10).

How long is the relaxation zone, and how do I choose n_{pre}? In the present version, the inlet relaxation zone is one deep-water wavelength L_0 long, from the offshore end of the domain, and the outlet zone of breakwater mode is $1.5\,L_0$ long (constants `n_relax_inlet` and `n_relax_outlet` in `coast_app.py`). The zones are written in `constant/waveProperties.input`. n_{pre} must therefore be larger than 1, to leave a length of free propagation between the end of the zone and the structure, in which the gauges of the reflection analysis are placed (Section 6.3). The demonstration campaign used $n_{\text{pre}} = 2.5$, and the example run of Chapter 4 2.

How fine must the mesh be? There is no fixed rule: the free surface needs enough cells per wave height in the interface band, the overtopping layer enough cells over its depth, and the only proof is a repetition with a finer mesh and a smaller time step (Section 7.4). The example run of Chapter 4 has about 9 cells per H_{m0} in the bands (1.25 cm), and half of its overtopping events are thinner than one cell.

How long should the simulation be? Long enough for the start-up, plus enough waves, and enough overtopping events, for the quantity of interest (Section 8.8). At low discharges a few large events dominate $\overline{q}$; repeat the run with other random-phase seeds to see the spread.

`WRITE_INTERVAL` or line write interval? `WRITE_INTERVAL` writes the fields and the free surface: it is the sampling interval of the gauges and of the snapshots of the event figures. The line write interval writes the sampling line `x01`, used by Steps 2 and 4. The overtopping flux is written at every time step in any case. Both intervals are fixed before the run, and both set the disk space (next question).

How much disk space does a case need? The example run of Chapter 4 (1000 s, fields every 0.1 s, line every 0.02 s, 8 processes) occupies 42 GB, mostly the time folders of the fields in `processor*` and the 50 000 folders of the line. Halving an interval doubles the corresponding part.

RANS or laminar? Both run from the same set-up. RANS (k-ω SST) is the standard choice; laminar is a reference without turbulence model, to check whether the turbulence model damps the waves (Section 8.4).

Which wall roughness K_s? K_s is the Nikuradse equivalent sand roughness of the surface; it acts in RANS only (Section 8.5).

Some segments cannot be selected for a force. Segments inside a porous zone are excluded: the porous model already accounts for the forces there (Section 5.6).

13.3 Running

How long does a run take, and with how many processes? It depends on the mesh, the duration and the computer. The example run of Chapter 4 took 5.1 h of solver time for 1000 s on 8 processes, while other runs were using the same machine. Choose `N_PROCS` at most equal to the number of physical cores available.

How do I follow a run? *Monitor* tab (press *Refresh*), the command `joblist` in a terminal, or the tmux session itself (Section 4.8).

The status is `DEAD?`. The tmux session has disappeared while the status still said `RUNNING`: the machine was restarted, or the session was killed outside COAST. Look at the end of `TMUX_OUT.log` and of the solver log.

The status is `FINISHED_1` (or another code). The job script stopped on an error. `TMUX_OUT.log` shows the step that failed; then look at its log (`log.snappy`, `log.topoSet`, `log.waveFoam`, ...; Section 12.2).

How do I stop a run? *Kill* in the *Monitor* tab, or `killcase <name>` from the repository root (Section 2.6). The status becomes `KILLED`; the results written so far are kept.

Can a stopped run be continued? Not from the interface, and not by running `job_Run.sh` again: the script rebuilds the mesh and decomposes the case again, which erases the `processor*` folders. A run can be continued by hand with OpenFOAM (`startFrom latestTime` in `system/controlDict`, then the solver alone in parallel). The overtopping flux of the continued run is then written to `postProcessing/overtopping/<starttime>/`, which Step 1 does not read: the two files must be joined into `postProcessing/overtopping/0/overtopping.dat` first.

How do I delete cases? `rmcase <case> [<case> ...]` deletes case folders faster than `rm -r` (Section 2.6).

13.4 Post-processing

The extraction of a gauge fails or skips many times. A gauge reads the points of the stored free surface within ±0.01 m of its x (and near its y). Times without such a point are skipped, and if no time has one the extraction fails: check that x is inside the domain and in the water, and that `postProcessing/freeSurface/` exists. Gauge files are named with two decimals of x: two positions closer than 0.005 m share one file.

Which gauge should I use to check the waves? A gauge between the end of the inlet relaxation zone and the toe, where the depth is the generation depth (Section 10.5). A gauge in front of a reflective structure measures incident plus reflected waves, so its H_{m0} exceeds the target; the incident waves alone are given by the *Reflection* tab (Section 4.13).

The automatic trimming gives a strange cut. The test compares windows of the record with its end (Section 10.2.1); it can return a cut near zero when the start-up is short, or a late cut on a short or drifting record. Check the *Trimming* tab and use *MANUAL* when in doubt.

How do I choose the Welch segment length? Longer segments resolve the peak better; shorter segments give a smoother spectrum (Section 10.5). With the automatic setting, a larger divisor N gives shorter segments. The example run of Chapter 4 used $N = 12$.

K_r looks wrong. Keep the analysis band around the peak, keep *Detrend signals* on, and check the placement of the gauges (Section 10.6).

The discharge differs from the measured one. Work through the verification sequence (Section 12.2): incident waves first (spectrum, reflection), then the record length and the number of events, then the mesh and the time step, then the porous parameters (Chapter 11 lists the usual sources of difference).

How do I choose `q_start` and `q_end`? They set which flow counts as an event; check that the events carry nearly all the overtopped volume (volume budget of Step 1, Section 10.7). For the example run of Chapter 4 they carry 98 %.

Step 1 does not finish. On a desktop session, its figures open in windows and Step 1 waits until they are closed; start COAST with `MPLBACKEND=Agg` (Section 2.4).

My flume is not 1 m wide. Step 1 divides the flux by the width `Ly` set at the top of `OV_Test.py`, not by the width of the case: edit it in `Rans_OF_Main/OV_Test.py` before running Step 1 (Section 10.7).

Where are the settings of Steps 2 and 3? At the top of `Rans_OF_Main/kinetic.py` and `merge_OV_energy.py`. The interface runs these two scripts from `Rans_OF_Main/` without a copy: a change applies to every case analysed afterwards. Steps 1 and 4 are copied into the case with the settings of the interface.

The velocity of many events is empty in Step 4. The water layer is thinner than $H_{\min}$ or has no point with $\alpha \geq 0.5$: its velocity is not resolved by the mesh (Section 10.10). Refine the run-up band if these events matter.

How do I get $h_{2\%}$ and $u_{2\%}$? Enter the number of incident waves N_w in `N_WAVES` (Step 4, *Parameters*). N_w is the number of waves of the *Wave Detection* tab at a gauge in front of the structure, over the same time window as Step 1.

The event figures fail or show no flux on the faces. They need the mesh, the time folders of the fields (not deleted), and fields written in ASCII (`writeFormat ascii`, as in the templates). Without `alphaPhi0.water` at the output times, the faces of the section are drawn without their flux, and the interface says so (Section 10.10.5).

The HTML report has few figures. The report collects the figures of the tabs visited since the page was opened: visit every tab of interest, then generate it (Section 10.12).

A case is missing in the *Compare* tab, or gives a warning. The tab lists the cases of the repository root, and each needs the file used by the chosen signal (Section 10.12).

Where are all the files? Appendix B lists every file written by COAST, with the step that writes it.

Chapter 14

Repository Structure

```
COAST/
  coast_app.py                    Main Streamlit app (case generation, monitoring)
  pages/
    postprocessing.py             Analysis page (10 tabs)
  theme.py                        Day/night theme, CSS, sidebar
  docs_content.py                 Text of the notebook documentation (not
                                  shown by the interface)
  Database_Eurotop_v5.xlsx        EurOtop database (records of the tests)
  requirements.txt                Python dependencies
  install.sh                      Local tmux, Python packages, PATH set-up
  LICENSE                         CeCILL Free Software License Agreement v2.1
  NOTICE                          Copyright, licence summary, author's notice
  README.md

  python_tools/                   Shared utilities
    clash_cad.py                  EurOtop geometry -> 2D polygon -> 3D STL,
                                  porous layers, obstacle union
    clash_plot.py                 Cross-section plotting with EurOtop labels
    canvas_tools.py               STL outline import for obstacles
    domain_tools.py               Domain sizing from STL bounding box and Tp
    Mesh_plot.py                  Refinement-band preview
    foam_case_tools.py            Writing and editing of OpenFOAM dictionaries
    tmux_run_case.py              Job-script writer and tmux launcher
    case_report.py                run_info.json and HTML case report
    pdf_report.py                 PDF report (ReportLab)

  Rans_OF_Main/                   RANS template (k-omega SST) and analysis scripts
    0.orig/  constant/  system/   OpenFOAM template dictionaries
    extract_from_freesurface.py   Gauge extraction from the stored iso-surfaces
    OV_Test.py                    Step 1: discharge and events
    kinetic.py                    Step 2: kinetic and potential energy flux
    merge_OV_energy.py            Step 3: event energy and energy-volume fits
    flow_depth_velocity.py        Step 4: flow depth and velocity per event
    event_figures.py              Figures of one overtopping event (snapshots
                                  and overtopping section)
    wave_analysis_py_functions/   Wave-analysis library
      io.py                       Signal and wave-dictionary reading
      stationarity.py             Stationarity detection and trimming
      waves.py                    Zero-crossing wave detection, statistics
      stats.py                    PDF, CDF, PoE, Rayleigh
      spectrum.py                 Welch PSD, spectral moments, JONSWAP
      reflection.py               Mansard-Funke three-gauge separation
      Report_wave_analysis.py     HTML wave-analysis report
      plotting.py, utils.py, domain_tools.py, surface_on_stl.py,
      make_video.py, docs.py      Plotting and helper functions

  Laminar_OF_Main/                Laminar template (no turbulence closure)
```

```
bin/
  joblist                       CLI: list and monitor cases
  killcase                      CLI: terminate a case
  rmcase                        CLI: fast deletion of case directories
tools/                          tmux installer and local tmux build
STL_Geos/                       STL files written by the app
docs/
  Documentation Manual.pdf      This manual
  figures/                      Figures of the manual; gui/: screenshots
```

Chapter 15

Licence and Disclaimer

15.1 CeCILL v2.1

COAST is released under the **CeCILL Free Software License Agreement, version 2.1** (dated 21 June 2013). The authoritative text is the `LICENSE` file at the root of the repository. It is not reproduced here: the licence provides that it may be reproduced and published only if it is not altered and no provision is added or removed, and retypesetting a legal text of this length into this manual would risk exactly that. This chapter describes it; `LICENSE` governs.

CeCILL was written by **CEA, CNRS and Inria** to give French public research a free-software licence drafted under French law, conformant with it both as regards the law of torts and intellectual property law. The family has three members: **CeCILL**, the copyleft licence used here and compatible with the GNU GPL; CeCILL-C, a weak copyleft for software components; and CeCILL-B, a permissive licence with a strong obligation of credit.

The licence exists in French and English, and the two versions are equally authentic (Article 11.5). The English text is distributed with the repository; the French text is published at `https://cecill.info/licences/Licence_CeCILL_V2.1-fr.txt`, and the English text at `https://cecill.info/licences/Licence_CeCILL_V2.1-en.txt`. The SPDX identifier is `CECILL-2.1`.

15.2 Author's additional notice

The following notice accompanies the repository in the file `NOTICE`. It adds nothing to the licence and removes nothing from it; Articles 8 and 9 govern. It exists only to state plainly what those articles mean for this particular software.

No responsibility for results

The author gives no assurance as to the accuracy, reliability, completeness or fitness for any purpose of any output produced by this software: generated geometries, meshes, OpenFOAM case files, simulation results, mean overtopping discharges, wave statistics, spectra, reflection coefficients, energy fluxes, forces, reports, figures, or any derived quantity.

Numerical results depend on the mesh, the numerical schemes, the turbulence closure, the porous-medium calibration, the wave generation and absorption settings, the record length, and the choices made by the user. All results must be independently verified before any use. The verification sequence of Section 12.2 is a minimum, not a guarantee.

Not for design

This software is **not** a design tool and must **not** be used as the sole or primary basis for the design, assessment, certification or operation of any coastal structure, flood defence or other engineering work, nor for any decision affecting public safety, property or the environment. Any

such use is at the user's own risk and remains the responsibility of the user and of the qualified engineer signing off on the work.

Software behaviour

COAST creates, modifies and deletes files and directories, launches and terminates processes and terminal-multiplexer sessions, and consumes CPU, memory, disk and cluster resources. The user is responsible for reviewing the code, for backing up their data, and for operating the software within the rules of their computing environment.

Third-party software and data

COAST orchestrates independent third-party software that it neither includes nor licenses to the user, in particular OpenFOAM, the waves2Foam library, and the Python scientific stack. Each is governed by its own licence and its own warranty terms. COAST is not approved or endorsed by OpenCFD Ltd (ESI Group), producer of the OpenFOAM software and owner of the OPENFOAM trademark, nor by any other third party.

The EurOtop experimental database and the EurOtop manual are the property of their respective owners and subject to their own terms of use. They are referenced for comparison purposes only and are not licensed under the terms above.

15.3 How to cite

If COAST contributes to published work, please cite the journal article that describes it:

> Mahdi, A., Turki, I., Réveillon, J., Duret, B. and Demoulin, F.-X. (2026). COAST: An open-source framework for automated wave overtopping analysis and simulation of coastal structures. *Results in Engineering*, 32, 113720. doi:10.1016/j.rineng.2026.113720

Where the implementation details matter, this manual may be cited as well:

> Mahdi, A. COAST: Coastal Overtopping Analysis and Simulation Toolkit. User and Technical Manual, Version 1.0.

Please also cite, where relevant, the underlying tools: OpenFOAM, waves2Foam (Jacobsen et al., 2012), and the EurOtop manual (EurOtop, 2018).

Appendix A

Input Parameter Reference

This appendix lists the inputs of the interface, with the name under which they appear, the symbol used in this manual, and their role. Default values are shown in the interface (Section 1.2).

Table A.1: Inputs of the case-generation page.

Input (interface)	Symbol	Unit	Role
Sidebar			
Case name	–	–	Name of the case folder and of the STL files
Flow model	–	–	`rans` (k-ω SST) or `laminar`; selects the template
Moon / sun button	–	–	Night or day theme of the interface
Geometry: core depths, slopes, crest			
SWL	z_{SWL}	m	Still-water level
m (foreshore slope)	m	–	Foreshore slope cotangent; 1000 selects the flume configuration
h_deep, h, h_t	h_{deep}, h, h_t	m	Water depths offshore, at the toe, above the toe berm
B_t, h_b, B	B_t, h_b, B	m	Toe-berm width, depth above the berm (mid-berm), berm width
tan_alpha_B	$\tan\alpha_B$	–	Berm slope (0 = horizontal)
cot_alpha_d, cot_alpha_u	$\cot\alpha_d$, $\cot\alpha_u$	–	Lower and upper seaward slopes
A_c, G_c, R_c, t_wall	A_c, G_c, R_c, t_{wall}	m	Crest level, crest width, freeboard (crown wall if $R_c > A_c$), wall thickness
Geometry: structure type and placement, CAD ramps			
Breakwater mode	–	–	Water on both sides, outlet relaxation zone, no collector
x_toe, x_margin	x_{toe}, x_{margin}	m	Toe position; flat bed added seaward of the foreshore
Ly	L_y	m	Spanwise width of the extrusion
ZMIN_THICKNESS	Δz_{min}	m	Solid thickness below the deepest bed level
L_back	L_{back}	m	Length of the back slope or of the collector
USE_COLLECTOR	–	–	Adds a collector box behind the crest
`dx_foreshore_ramp`, `dx_toe_transition`, `dx_berm_connector`	–	m	Connector ramps (Chapter 5)
Geometry: porous structure			
Enable porous zones, Active zones	–	–	Porous layers and the surface zones that carry them
back_slope: crest wedge only	–	–	With `back_slope` active: fill only the wedge under the crest end (Section 5.5)
Number of layers, Layer name	–	–	Concentric layers, from the outside inward
Default thickness d, zone thickness overrides	d	m	*Vertical* layer thickness (Eq. 5.10)

continued on next page

Table A.1 continued.

Input (interface)	Symbol	Unit	Role
porosity, d50, KC, alpha, beta, gammaAddedMass	n, d_{50}, KC, α_{vG}, β_{vG}, γ_{AM}	–, m	Resistance parameters of a layer (Section 8.6); zone overrides for n and d_{50}
Solid core; core porosity, d50, alpha, beta	–	–	Impermeable core, or porous core with its own parameters
Geometry: obstacles			
Add a new obstacle	–	–	Shows the interactive view and the obstacle inputs
Add obstacle as: Rectangle (Left x, Right x, Bottom z, Top z)	–	m	Rectangle obstacle; button *Add rectangle obstacle*
Add obstacle as: Polygon vertices	–	m	One `x, z` pair per line, at least three; button *Add polygon obstacle*
Import STL obstacle: Upload .stl file, Y-plane for cross-section	–	m	Outline of an STL on a plane y = const; button *Import obstacle from STL*
Width, Height, Center X, Center Z	–	m	Resizing and moving an existing obstacle (*Apply dimensions*); *Remove* deletes it
Circular-arrow button next to *Obstacles*	–	–	Redraws the obstacle view
Wave and Domain			
Hm0_deep, Tp_deep, Tm_deep	$H_{m0,\mathrm{deep}}$, $T_{p,\mathrm{deep}}$, $T_{m,\mathrm{deep}}$	m, s	Target spectrum at the generation boundary; L_0
Random phase seed	–	–	Seed of the random phases; default 1234 (Section 6.1)
Hm0_toe, Tp_toe, Tm_toe, Tm10_toe	$H_{m0,\mathrm{toe}}$, …	m, s	Expected toe conditions; $H_{m0,\mathrm{toe}}$ sizes the interface band
Sizing mode	–	–	Automatic ($n \times L_0$) or manual (metres)
n_pre, n_post	n_{pre}, n_{post}	–	Upstream and downstream lengths in L_0
Inlet length, Outlet length	–	m	Manual mode lengths from the ends of the STL
top_clearance	$\Delta z_{\mathrm{clear}}$	m	Air height above the structure
Mesh			
dx_target, dz_target, Ny	$\Delta x_{\mathrm{target}}$, $\Delta z_{\mathrm{target}}$, N_y	m, –	Background mesh
LEVEL_INTERFACE, LEVEL_RUNUP	$\ell_{\mathrm{interface}}$, ℓ_{runup}	–	Refinement levels (Eq. 7.2)
`Vertical_distance_plus_for_refinement_interface`	Δz_e	m	Buffer of the interface band
`RUNUP_X_START_OFFSET`, `RUNUP_X_END`, `RUNUP_TOP_ABOVE_STL`	$\Delta x_{\mathrm{offset}}$, x_{re}, Δz_{up}	m	Extent of the run-up band
Simulation and Job			
END_TIME, DELTA_T, WRITE_INTERVAL	–	s	Duration, initial time step, field output interval
MAX_CO, MAX_ALPHA_CO	–	–	Courant limits
Line write interval	Δt_l	s	Output interval of the sampling line `x01`

continued on next page

Table A.1 continued.

Input (interface)	Symbol	Unit	Role
Ks, Cs	K_s, C_s	m, –	Wall roughness (RANS)
N_PROCS, DECOMP_METHOD	–	–	Number of processes; `scotch` or `simple`
Create new force/impact patch, segments, SOURCE_PATCH	–	–	Force segments (Section 5.6); `SOURCE_PATCH` is the structure patch the faces are taken from
Run and Monitor			
Build Case, Launch Simulation	–	–	Write the case; start it in tmux (Section 4.8)
Refresh, Scan root	–	–	Search the scan directory for cases with `TMUX_SOCKET.txt`
Select case for details / kill	–	–	Details, solver log, report, post-processing and *Kill* button of one case

The number of points of the sampling line and its y-position are not exposed in the interface; they are recorded in `run_info.json`.

Table A.2: Inputs of the post-processing page.

Input (interface)	Symbol	Role
Select case, Case directory (full path)	–	Case to analyse; any folder with *enter path manually*
Gauge X-positions, y_target, SWL z	x_g, y_g, z_{SWL}	Gauges to extract (Section 10.1)
Force re-extraction	–	Extract again the gauges whose file exists
Periods to show	–	Length of the window drawn in the *Wave Detection* tab
Trim method, t_cut	t_{cut}	AUTO or MANUAL trimming (Section 10.2)
Welch nBlocks, Auto divisor N, Fixed value	N_{seg}, D	Segment length of the Welch spectrum
Max frequency	–	Upper limit of the spectrum plot
Gauge 1–3 x, Water depth h	x_1, x_2, x_3, h	Reflection array and its depth
f_min, f_max, Detrend signals	f_{min}, f_{max}	Reflection analysis band; removal of mean and trend
Window mode, Vthr_abs, t_1, t_2	V_{thr}, t_1, t_2	Averaging window of Step 1
q_start, q_end, min_duration, min_volume	q_{start}, q_{end}	Event detection of Step 1
H_MIN, N_WAVES	H_{min}, N_w	Velocity resolution limit and number of incident waves for Step 4
Order of the events, Event, Number of snapshots, Top of the speed colour bar, Choose the snapshot times (First snapshot, Time between snapshots), Set the windows of the figures	–	Figures of one overtopping event (Section 10.10.5)
Cases to compare, Signal	–	Multi-case comparison
User Name	–	Name printed on the PDF report

Settings that are not exposed in the interface are kept at the top of the scripts: the flume width and the `STATIONARY` window parameters of `OV_Test.py`, the weighting options and fallback event detection of `kinetic.py`, the minimum event volume and histogram bins of `merge_OV_energy.py`, the stationarity parameters of `stationarity.py`, the extraction window of `extract_from_freesurface.py`, and the colours, arrow length, figure width and image resolution of `event_figures.py`.

Appendix B

Output File Reference

All paths are relative to the case folder.

Table B.1: Files written by COAST.

File	Written by	Content
Set-up and run		
`STL_Geos/<case>.stl` (repository)	Generate STL	Structure (with merged obstacles)
`STL_Geos/<case>_solid.stl`, `_core.stl`, `_<layer>_<zone>.stl`	Generate STL	Porous cases: impermeable part, porous core, one STL per layer and zone
`0.orig/`, `constant/`, `system/`, `job_Run.sh`	Build Case	OpenFOAM case and job script
`figures/impact_patch_segments.png`	Build Case	Selected force segments
`report_assets/`	Launch	Logos of the case report
`run_info.json`	Launch	All input parameters
`run_report.html`, `figures/`	Launch	Case report and set-up figures
`TMUX_STATUS.txt`, `TMUX_OUT.log`	Job	Run status; complete job output
`postProcessing/overtopping/0/overtopping.dat`	Solver	Water flux $\boldsymbol{Q}(t)$ through the overtopping section, every time step
`postProcessing/line/<time>/x01_*.xy`	Solver	s, α, p_{rgh}, ρ, $\boldsymbol{U}$ on the sampling line
`postProcessing/freeSurface/<time>/`	Solver	$\alpha = 0.5$ iso-surface
`postProcessing/forceImpact_<patch>/0/force.dat`	Solver	Pressure and viscous forces on a force patch
Wave analysis		
`Wave_Analysis/data/eta_from_freeSurface_x_<pos>m.txt`	Gauge extraction	t, η at one gauge (<pos>: x with two decimals, `p` for the point)
`Wave_Analysis/output/eta_vs_time_x_<pos>m.png`	Gauge extraction	Quick plot of the extracted signal
`Wave_Analysis/figures/*.png`	Tabs 1–6	`eta_x_*`, `trim_x_*`, `wave_detection_x_*`, `pdf_x_*`, `cdf_x_*`, `poe_x_*`, `spectrum_x_*`, `reflection_timeseries`, `reflection_spectra`
`Wave_Analysis/reflection_results.json`	Tab 6	$H_{m0,\mathrm{inc}}$, $H_{m0,\mathrm{ref}}$, K_r, gauges, depth, band
`Wave_Analysis/wave_report.html`	Tab 9	Wave-analysis report
`<case>_report.pdf`	Tab 9	PDF report
`force_Fx_<dir>.png`	Tab 8	Force time series
Step 1 (`OV_Test.py`)		
`OV_Test.py`	Step 1	Copy of the script with the settings of the interface
`overtopping_events.csv`	Step 1	event_id, t_start, t_end, duration, volume, q_peak
`overtopping_summary.txt`	Step 1	Window, $\overline{q}$, events, volume budget
`overtopping_fig01_cumulative_volume.png`	Step 1	$V^{+}(t)$ and averaging window

continued on next page

Table B.1 continued.

File	Written by	Content
`overtopping_fig02_rate_and_events.png`	Step 1	$q^+(t)$ and events
`overtopping_fig03_background_samples.png`	Step 1	Samples outside the events
`overtopping_fig04_stationary_diagnostic.png`	Step 1	`STATIONARY` mode only
Step 2 (`kinetic.py`)		
`energyFlux_components.dat`	Step 2	t, F_{KE}, F_{PE}, F_{tot}
`energyFlux_events.dat`	Step 2	Event windows and E_e, $E_{\mathrm{KE},e}$, $E_{\mathrm{PE},e}$
`energyFlux_components.png`, `energyFlux_cumulative.png`, `energyFlux_events_hist.png`	Step 2	Fluxes, cumulative energy, histogram of event energies
Step 3 (`merge_OV_energy.py`)		
`overtopping_energy_merged.csv`	Step 3	Per event: V_e, q_{peak}, energies, e_e, mean and peak flux, classification, fit results
`merge_EKE_vs_V.png`, `merge_EPE_vs_V.png`, `merge_Etotal_vs_V.png`	Step 3	Event energies versus volume
`merge_event_energy_components.png`	Step 3	E_{KE} and E_{PE} of every event
`merge_event_energy_vs_volume_median.png`	Step 3	Median regression
`merge_power_law_scaling.png`	Step 3	Power law
`merge_energy_per_volume_vs_rank.png`	Step 3	Specific energy by rank
`merge_event_energy_pdf.png`, `merge_event_energy_cdf.png`	Step 3	Distribution of event energies
Step 4 (`flow_depth_velocity.py`)		
`flow_depth_velocity.py`	Step 4	Copy of the script with $H_{\min}$ and N_w of the interface
`flow_depth_velocity_timeseries.csv`	Step 4	t, h_f, q_l, u_f
`flow_depth_velocity_events.csv`	Step 4	Per event: window, V_e, number of line samples, $h_{f,\max}$, $u_{f,\max}$, time of $h_{f,\max}$, V_{line}, $h_{f,\max}$ in cells
`flow_depth_velocity_summary.txt`	Step 4	Summary, checks, $h_{2\%}$ and $u_{2\%}$
`fdv_fig01_timeseries.png`	Step 4	h_f, u_f, q_l with event windows
`fdv_fig02_events_vs_volume.png`	Step 4	Per-event maxima versus volume
`fdv_fig03_exceedance.png`	Step 4	Exceedance of the per-event maxima
Figures of one event (`event_figures.py`)		
`event_figures/event<N>_snapshots.pdf`, `.png`	Overtopping tab	Snapshots of event N with q^+, h_f, u_f (Figure 10.11)
`event_figures/event<N>_section.pdf`, `.png`	Overtopping tab	Overtopping section at the peak of event N (Figure 10.12)
`event_figures/event<N>_info.txt`	Overtopping tab	Event values, snapshot times and windows of the figures

Bibliography

Brackbill, J.U., Kothe, D.B., and Zemach, C. (1992). A continuum method for modeling surface tension. *Journal of Computational Physics*, 100(2):335–354.

Chen, W., Warmink, J.J., van Gent, M.R.A., and Hulscher, S.J.M.H. (2021). Numerical modelling of wave overtopping at dikes using OpenFOAM. *Coastal Engineering*, 166:103890.

CIRIA, CUR, CETMEF (2007). *The Rock Manual. The use of rock in hydraulic engineering.* Second edition, C683, CIRIA, London.

DNV (2021). *Environmental conditions and environmental loads.* Recommended Practice DNV-RP-C205.

EurOtop (2018). *Manual on wave overtopping of sea defences and related structures: an overtopping manual largely based on European research, but for worldwide application.* Second edition. Van der Meer, J.W., Allsop, N.W.H., Bruce, T., De Rouck, J., Kortenhaus, A., Pullen, T., Schüttrumpf, H., Troch, P., and Zanuttigh, B. www.overtopping-manual.com

Goda, Y. and Suzuki, Y. (1976). Estimation of incident and reflected waves in random wave experiments. *Proceedings of the 15th International Conference on Coastal Engineering*, pp. 828–845.

Hasselmann, K., et al. (1973). Measurements of wind-wave growth and swell decay during the Joint North Sea Wave Project (JONSWAP). *Ergänzungsheft zur Deutschen Hydrographischen Zeitschrift*, Reihe A, Nr. 12.

Jacobsen, N.G., Fuhrman, D.R., and Fredsøe, J. (2012). A wave generation toolbox for the open-source CFD library: OpenFOAM. *International Journal for Numerical Methods in Fluids*, 70(9):1073–1088.

Jensen, B., Jacobsen, N.G., and Christensen, E.D. (2014). Investigations on the porous media equations and resistance coefficients for coastal structures. *Coastal Engineering*, 84:56–72.

Larsen, B.E. and Fuhrman, D.R. (2018). On the over-production of turbulence beneath surface waves in Reynolds-averaged Navier–Stokes models. *Journal of Fluid Mechanics*, 853:419–460.

Larsen, B.E., Fuhrman, D.R., and Roenby, J. (2019). Performance of interFoam on the simulation of progressive waves. *Coastal Engineering Journal*, 61(3):380–400.

Losada, I.J., Lara, J.L., Guanche, R., and Gonzalez-Ondina, J.M. (2008). Numerical analysis of wave overtopping of rubble mound breakwaters. *Coastal Engineering*, 55(1):47–62.

Mansard, E.P.D. and Funke, E.R. (1980). The measurement of incident and reflected spectra using a least squares method. *Proceedings of the 17th International Conference on Coastal Engineering*, pp. 154–172.

McCormick, M.E. (2010). *Ocean Engineering Mechanics with Applications.* Cambridge University Press.

Menter, F.R. (1994). Two-equation eddy-viscosity turbulence models for engineering applications. *AIAA Journal*, 32(8):1598–1605.

Moretto, F., et al. (2020). Numerical modelling of wave interaction with permeable coastal structures. *Coastal Engineering Proceedings*.

Rusche, H. (2002). *Computational fluid dynamics of dispersed two-phase flows at high phase fractions.* Ph.D. thesis, Imperial College London.

van Gent, M.R.A. (1995). *Wave interaction with permeable coastal structures.* Ph.D. thesis, Delft University of Technology.

Vanneste, D.F.A. and Troch, P. (2015). 2D numerical simulation of large-scale physical model tests of wave interaction with a rubble-mound breakwater. *Coastal Engineering*, 103:22–41.

Welch, P.D. (1967). The use of fast Fourier transform for the estimation of power spectra. *IEEE Transactions on Audio and Electroacoustics*, 15(2):70–73.

Zanuttigh, B. and van der Meer, J.W. (2008). Wave reflection from coastal structures in design conditions. *Coastal Engineering*, 55(10):771–779.

Zelt, J.A. and Skjelbreia, J.E. (1992). Estimating incident and reflected wave fields using an arbitrary number of wave gauges. *Proceedings of the 23rd International Conference on Coastal Engineering*, pp. 777–789.